AI Overview



HUAWEI

Foreword

- Artificial intelligence (AI) is driving progress and industrial upgrades at an unprecedented speed. As a field of research in human intelligence simulation, AI has not only profoundly changed our lifestyles but has also showcased its immense value across various fields such as research, industrial production, healthcare, financial services, and education and entertainment.
- This section describes the history, key technologies, and applications of AI in various fields. It provides insights into the application fields and future of AI technologies based on their definition and scope.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completing this course, you will be able to understand:
  - Basic AI concepts
  - AI technologies and their history
  - AI applications in various technologies and fields
  - Future AI trends

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **AI Overview**
2. AI Technologies
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI in the Eyes of Researchers

"I propose to consider the question, 'Can machines think?'"

-- Alan Turing in 1950

Make machines behave like humans.

-- John McCarthy in 1956

"Artificial intelligence is the science of making machines do things that would require intelligence if done by men."

-- Marvin Minsky

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- "The branch of computer science concerned with making computers behave like humans." -- A widespread early definition of AI proposed by John McCarthy at the Dartmouth Conference in 1956. However, it seems that this definition ignores the possibility of strong AI. According to another definition, AI is the intelligence (weak AI) demonstrated by artificial machines.
- Alan Turing discussed the question "Can machines think" in his paper Computing Machinery and Intelligence.

What Is Intelligence?

- Professor Howard Gardner proposed the theory of multi-intelligence, and listed eight capabilities that reflect multi-intelligence:
  - Verbal/Linguistic
  - Logical/Mathematical
  - Visual/Spatial
  - Bodily/Kinesthetic
  - Musical/Rhythmic
  - Inter-personal/Social
  - Intra-personal/Introspective
  - Naturalist

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Gardner, American educator and psychologist
- 1. Linguistic intelligence

It refers to the ability to express thoughts and understand others orally and literally, to grasp speeches, semantics, and grammars masterly, and to think in words, express in words, and appreciate the deep meaning of languages. They are competent for jobs such as political activists, presenters, lawyers, speakers, editors, writers, journalists, and teachers.

- 2. Logical-mathematical intelligence

It refers to the ability to calculate, measure, infer, conclude, classify, and carry out complex mathematical operations. This type of intelligence possesses sensitivity to logical means and relationships, statements and propositions, functions, and other related abstract concepts. They are competent for jobs such as scientists, accountants, statisticians, engineers, and computer software R&D personnel.

- 3. Spatial intelligence

It refers to the ability to accurately perceive the visual space and surroundings and present the perception in the form of graphics. This type of intelligence possesses sensitivity to colors, lines, shapes, forms, and spatial relationships. They are competent for jobs such as interior designers, architects, photographers, painters, and pilots.

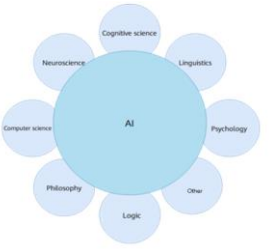
- 4. Bodily-kinesthetic intelligence

It refers to the ability to express thoughts and feelings with the body and to make or operate

objects with hands. This type of intelligence possesses special physical skills such as balance, coordination, agility, strength, flexibility and speed, and abilities triggered by tactile sensation. They are component for jobs such as athletes, actors, dancers, surgeons, jewellers, and mechanics.

What Is AI?

- Artificial intelligence can be interpreted as "artificial" plus "intelligence". "Artificial" means being designed, created, and made by humans. "Intelligence" means thinking and behaving like humans.
- Artificial intelligence is a new technical science that deals with the research and development of theories, methods, techniques, and application systems for simulating and extending human intelligence. In 1956, the term "Artificial intelligence" was first coined by John McCarthy, who defined it as "the science and engineering of making intelligent machines." The purpose of AI is to make machines intelligent and give them human thoughts.
- AI has become an interdisciplinary subject that overlaps with various fields.

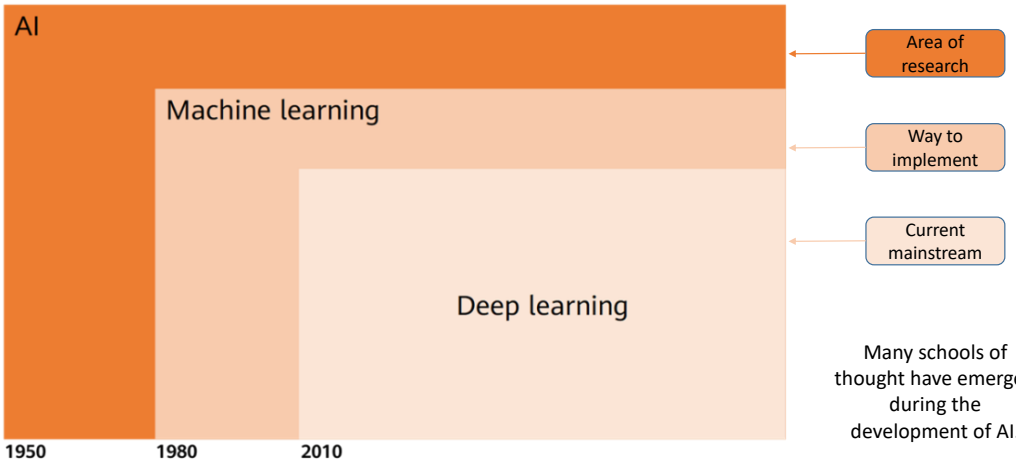


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Machine learning can be understood from multiple aspects. Tom Mitchell, known as the "godfather of global machine learning", defined machine learning as: For a task T and performance metric P, if the performance of a computer program measured by P on T self-improves with experience E, the computer program is learning from experience E. These definitions are simple and abstract. However, as we deepen our understanding of machine learning, we will find that the connotation and extension of machine learning are changing over time. Because a variety of fields and applications are involved and machine learning develops rapidly, it is not easy to define machine learning simply and clearly.

Relationships Among AI, Machine Learning, and Deep Learning (1)



Many schools of thought have emerged during the development of AI.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Relationships Among AI, Machine Learning, and Deep Learning (2)

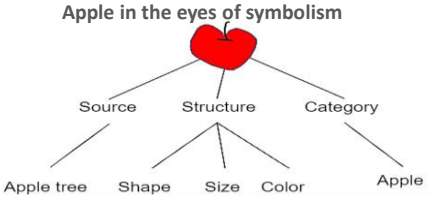
- AI is a new technical science that studies and develops theories, methods, techniques, and application systems to simulate and extend human intelligence.
- Machine learning is the study of how computers simulate or replicate human learning processes to acquire new knowledge or skills and restructure existing knowledge for continuous performance improvement.
- Deep learning is a concept that originates from the research of artificial neural networks. It is an ongoing field of machine learning research that mimics the human brain's mechanism to interpret data.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- The term "artificial intelligence" was previously used to describe machines that imitate and demonstrate "human" cognitive skill related to human thinking.
- A machine learning algorithm mainly builds a model based on sample data (referred to as trained data), so that prediction or decision-making can be made without explicit programming.
- Deep learning is a type of machine learning, and machine learning is a one and only path for implementing artificial intelligence. The concept of deep learning originates from the study of artificial neural networks. A multi-layer perceptron containing multiple hidden layers is a deep learning structure. Deep learning uses higher level features derived from the lower level features to form a hierarchical representation. The motivation of deep learning research is to establish a neural network that simulates the human brain for analysis and learning. The neural network simulates the mechanism of the human brain to interpret data, such as images, sounds, and texts.

Major Schools of AI - Symbolism

- Symbolism is also called Logicism, Psychologism, or Computerism.
- Symbolism believes that AI is built upon mathematical logic. Followers of this school of thought believe that symbols are the cognitive primitives of humans, and the human cognition process is based on inferring and calculating a variety of symbols. In their opinion, both humans and computers use physical symbol systems, so computers can be used to simulate the intelligent behavior of humans.



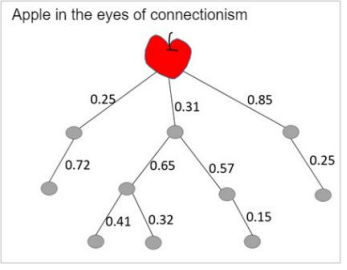
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- After 1956, symbolism developed heuristic algorithms, expert systems, and knowledge engineering theories and technologies, and made significant progress in the 1980s.
- The successful launch and application of the expert system is of great significance for leading AI to engineering application and linking AI theory with practice.
- After other AI schools emerged, symbolism is still a mainstream of AI.

Major Schools of AI - Connectionism

- Connectionism is also called Bionicsism or Physiologism.
- Connectionism believes that AI originates from bionics, particularly the research on human brain models. The primitives of human thinking are neurons, rather than symbolic processes. Researchers in this school of thought began by looking at neurons and then neural network models and brain models. This has successfully opened another path to artificial intelligence.





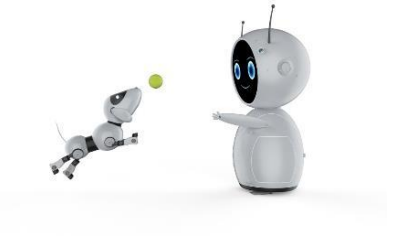
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Units in a network can represent neurons, and connections can represent synapses, just as in the human brain.
- In 1986, Rumelhart et al. proposed a backpropagation (BP) algorithm in a multi-layer network. Since then, connectionism has gradually emerged, from models to algorithms, and from theoretical analysis to engineering implementation.

Major Schools of AI - Behaviorism

- Behaviorism is also called Evolutionism or Cyberneticsism.
- Behaviorism believes that AI originates from cybernetics. In this school of thought, intelligence depends on perception and action; and knowledge, representation, and inference are not involved. AI can evolve like human intelligence. Intelligent behavior can only be manifested through interaction with surrounding environments in the real world.

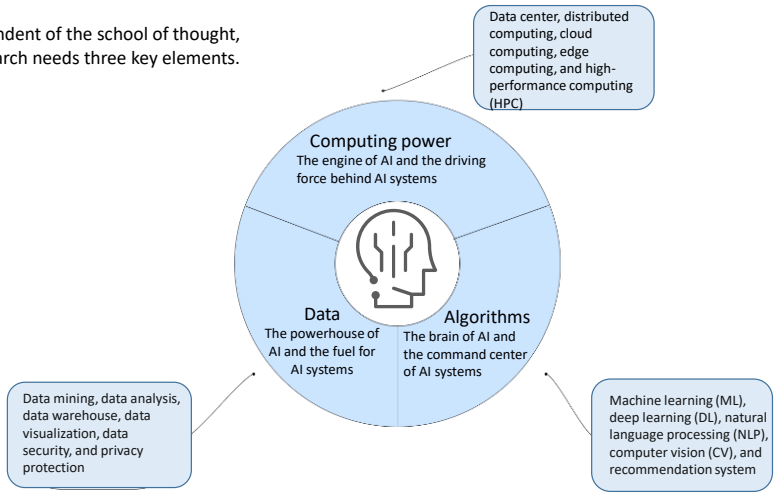


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Behaviorism concerns more about application practices and how to learn from the environment continuously to make corrections.
- School masterpiece: Brooks' hexapod walking robot, a control system that simulates insect behavior based on the perception-action pattern.
- This school is a somewhat similar to an adaptive control system, which collects data using sensors (environments) and acts on the system.

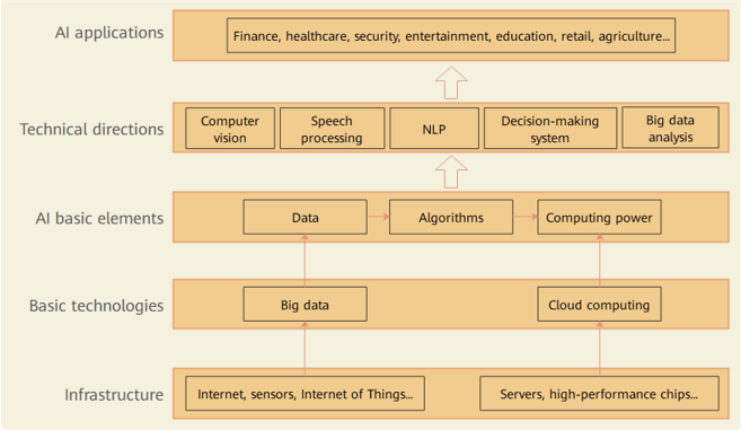
Three Elements for AI Development

Independent of the school of thought, AI research needs three key elements.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Industry Ecosystem



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Types of AI

- Strong AI
  - The strong AI theory is that it is possible to create intelligent machines capable of reasoning and problem solving, and that such machines will be self-aware. They will be able to think independently and develop the best solution to a problem. In addition, such machines will have their own values and worldview, as well as the instincts of living things, such as the need for survival and safety. In a sense, the machine with human thoughts can be regarded as a new civilization.
- Weak AI
  - The weak AI theory holds that it is impossible to create an intelligent machine capable of reasoning and problem solving. Such a machine only looks intelligent but does not have human intelligence or self-awareness.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

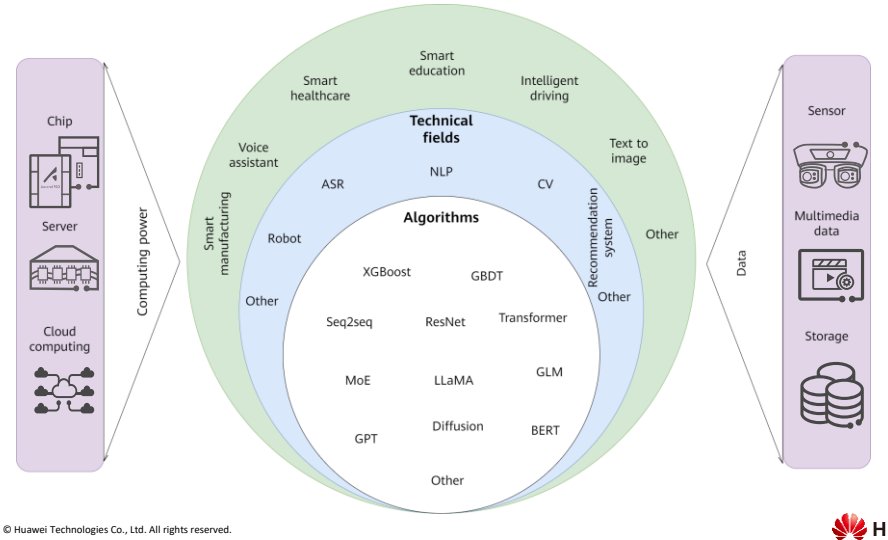
- An important indicator of AI is to achieve a superhuman level in challenging fields through self-learning without any prior knowledge.
- Strong AI refers to AI that can compete with humans in all aspects. Therefore, strong AI is not limited to a specified field, but makes robots comparable to humans in all aspects. Strong AI can think, plan, solve problems, abstract thinking, understand complex concepts, quickly learn, and learn from experience. Currently, it is believed that if the human brain can be simulated and all neurons and synapses in the human brain can be imitated on the same scale, strong AI will naturally occur.
- Now we are in the weak AI phase. Weak AI alleviates human intellectual labor, similar to advanced bionics. AI outperforms humans only in some aspects.

Contents

1. AI Overview
2. **AI Technologies**
   - AI Technologies
   - NLP
   - CV
   - Foundation Model
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. Debates and Future of AI

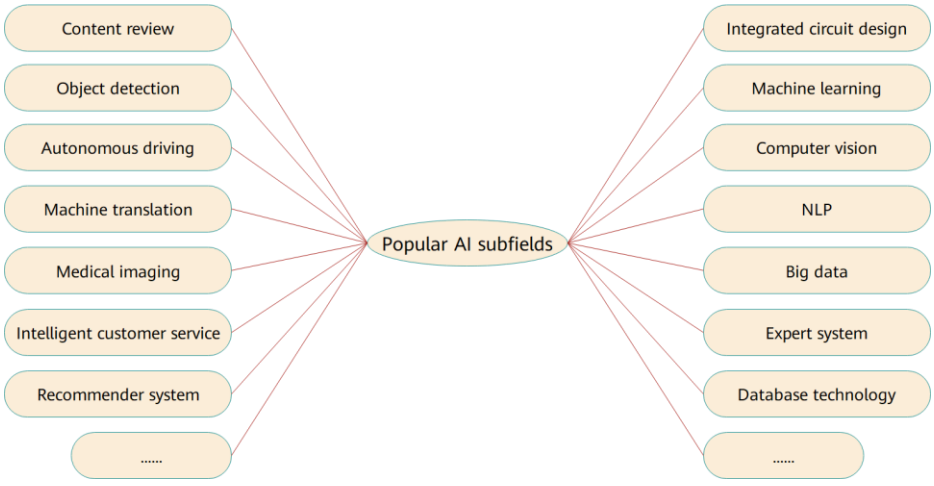
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Overview of AI Technologies & Application Fields



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Popular AI Subfields



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Distribution of AI Applications

- AI is commonly applied in the following technical fields:
  - Computer vision
    - How can we enable computers to "see" and "see" faster and more accurately?
  - Natural language processing
    - How can we enable computers to understand and use natural languages?
  - Multimodal
  - Other



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. **AI Technologies**
   - AI Technologies
   - NLP
   - CV
   - Foundation Model
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

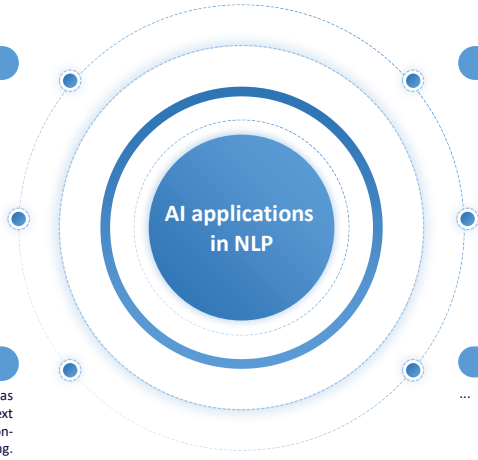
Natural Language Processing

- Natural language processing (NLP) is a branch of computer science and artificial intelligence. It studies theories and methods for implementing effective communication between humans and computers through natural languages. NLP is intended to enable computers to understand and generate human languages.
  - Development: (1) Rules and linguistic theories; (2) Machine learning and deep learning; (3) Large language models (LLMs)
- Applications:
  - Sentiment analysis
  - Text mining
  - Machine translation
  - Chatbots
  - Other



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Applications in NLP



Text classification

Automatically classifies text into preset categories by using algorithms, and analyzes text features to manage information efficiently and support precise decision-making.

Text summarization

Automatically extracts core information from text and generates a more concise version while retaining the original semantic associations and key elements.

Information extraction

Automatically extracts key information, such as entities and relationships, from massive text data to support data analytics and decision-making.

Machine translation

Automatically converts a language into another language (for example, Chinese to English, or French to Japanese) using algorithm models.

Smart writing

Assists in generating high-quality text, such as news releases and advertising copies, improving content creation efficiency.

...

...

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

NLP Task - Text Classification

- Definition
  - Input:
    - An article d
    - Fixed category set $C = \{C_1, C_2, \dots, C_j\}$
  - Output:
    - Predicted category for the article $c \in C$

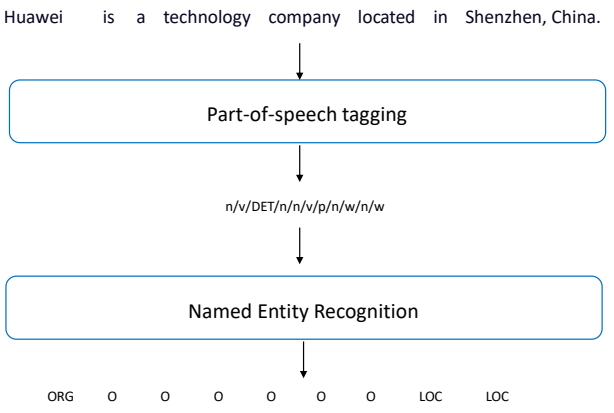
I wasn't happy to hear about a price cut a few days after this purchase. The flash memory score is just over 500 points. ☹

Pleasing appearance and amazing sound. Now, most electronic products are made by Huawei. 😀

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

NLP Task - Sequence Labeling

- Sequence labeling is using a model to label each position of a given input sequence (usually a sentence, character, or word in text) to form a label sequence. This process may be viewed as sequence-to-sequence mapping.



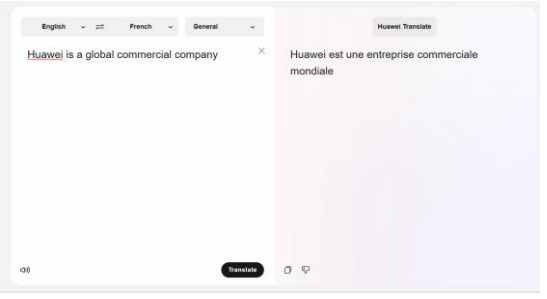
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- DET represents determiner, n represents noun, v represents verb, p represents preposition, and w represents punctuation.

NLP Task - Text Generation

- Text generation is a process in which an algorithm automatically generates grammatically sound and meaningful text content.
- This technology can simulate the human writing process and generate various types of text, such as news reports, emails, articles, social media posts, poems, novel chapters, and machine translations.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Knowledge Graph

- A knowledge graph is a graphical data model used to represent and organize structured information. It represents entities (such as people, places, and things) in the real world and their relationships (such as friendships and affiliations) in the form of graphs to construct a large-scale, multi-domain knowledge base.
- NLP provides the means to build a knowledge graph. NLP helps extract information (such as entities, relationships, and attributes) from text to build and enrich knowledge graphs.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. **AI Technologies**
   - AI Technologies
   - NLP
   - CV
   - Foundation Model
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Computer Vision

- Computer Vision (CV) is a technology that uses computers and algorithms to simulate the process of human vision. It involves extracting information from images or videos, analyzing and comprehending content, and making decisions accordingly.






Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Applications in CV

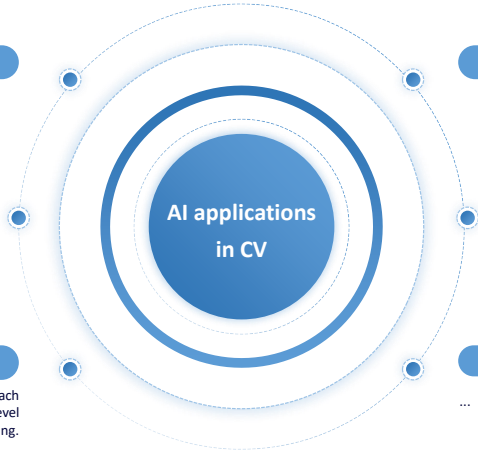


Image classification

Uses extensive amounts of unlabeled data to improve the representation capability of models for applications in various general scenarios.

Object detection

Identifies and locates objects in images or videos, supporting automatic analysis and decision-making.

Semantic segmentation

Assigns a predefined class label to each pixel in an image, implementing pixel-level image understanding.

Video inpainting

Restores and enhances video quality using deep learning-based techniques.

Intelligent driving

Enables vehicles to perceive surroundings, make decisions, and perform driving operations like human drivers.

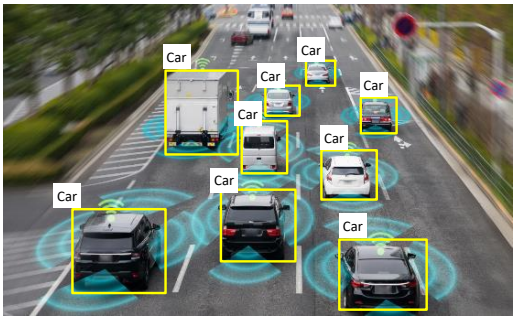
...

...

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CV Task - Object Detection

- Object detection is one of the fundamental technologies for image processing and computer vision in the AI field. It has a wide range of applications, such as traffic monitoring, image search, facial recognition, and human-computer interaction. Object detection technologies can help detect objects in an image for further processing with intelligent algorithms.



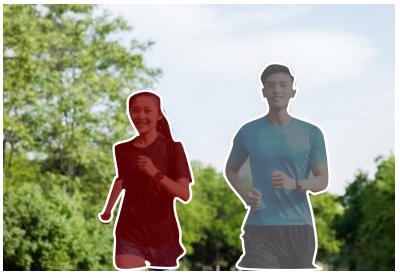
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Due to different appearances, shapes, and arrangements of objects, as well as lighting and shading during imaging, object detection has always been the largest challenge in the field of computer vision.
- Despite similar techniques used, object recognition is slightly different from object detection. Given a specific object, target recognition is aimed to find an instance of the object in an image. This is not to classify but to judge whether the object appears in the image, and if so, the object is located. For example, a real-time image taken by a security camera is monitored to identify a face of a person.

CV Task - Image Segmentation

- Image segmentation is the process of partitioning an image into multiple segments based on the problem to be solved.
- There are many algorithms and application methods for image segmentation. Common ones include connected component segmentation, motion segmentation, and object segmentation.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Object segmentation: semantic segmentation and instance segmentation.
  - Semantic segmentation refers to pixel-level image recognition, that is, marking the object category to which each pixel in the image belongs.
  - Instance segmentation is a task of identifying each instance of an object from an image and performing **pixel-level** labeling on each of the instances.
- Segmentation is a pixel-level description of an image, and makes each pixel category (instance) count. It is applicable to scenarios with high requirements for understanding, for example, segmentation between roads and off-roads in unmanned driving.
- Input and output:
  - Input: image.
  - Output: segmented images with the same resolution as the input image and label of each pixel category.

CV Task - Object Tracking

- Object tracking is a core research area in computer vision and has a wide range of applications, such as intelligent transportation, security monitoring, human-machine interaction, and autonomous driving.
- Tracking algorithms can obtain the trajectories of target objects in terms of time to analyze their movement behaviors.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CV Task - OCR

- Optical Character Recognition (OCR) is the process of recognizing characters in images or scans and converting them into editable ones using image processing technologies such as character object detection and object classification. OCR improves service efficiency by sparing manual information input. OCR can be used to identify characters in ID cards, driving licenses, vehicle licenses, invoices, customs forms, general tables, and general text.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CV Task - Image Generation

- Image generation refers to generating a wholly new image, modifying a part of an image, or generating a new image based on an existing one.
  - Super resolution is a process of estimating a high-resolution image from a low-resolution counterpart and predicting image features at different magnifications.
  - Style migration is a process of applying the style of one domain or several images to other domains or images. For example, an abstract style is applied to a realistic image.
  - Image inpainting is the process of fixing images, for example, restoring damaged black-and-white photos and videos.





Image inpainting

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Input and output:
  - Input: realistic image and random noise
  - Output: generated image

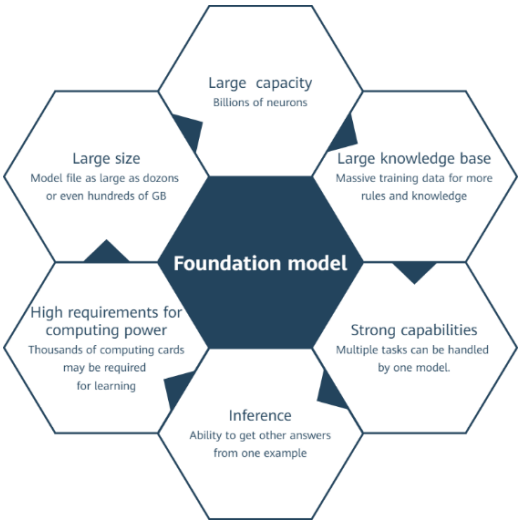
Contents

1. AI Overview
2. **AI Technologies**
   - AI Technologies
   - NLP
   - CV
   - Foundation Model
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

What Is a Foundation Model (1)?

- A foundation model refers to a model that is trained on large-scale data, has a massive number of parameters, and has powerful functionalities.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

What Is a Foundation Model (2)?

Principles Behind Large Models

Scaling Law

As the model size increases exponentially, the model performance increases linearly.

Chinchilla Law

The model size and the number of training tokens should scale at equal rates.

Emergent Abilities

When the model reaches a certain size and scale, it shows a sudden and unexpected improvement in performance and generalization capability.

What makes a model "large":

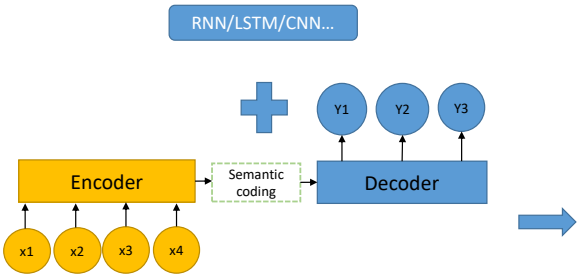
- ✓ Large-scale data: Extensive data to cover nearly all real-world scenarios
- ✓ Large-scale model: High capacity to adapt to almost any scenario
- ✓ Large-scale compute: Robust computing resources for handling intricate calculations

Three problems of large models:

- ✓ How many parameters does a large model have?
- ✓ How can we develop complex algorithms?
- ✓ How does a model process the input data and produce the output?

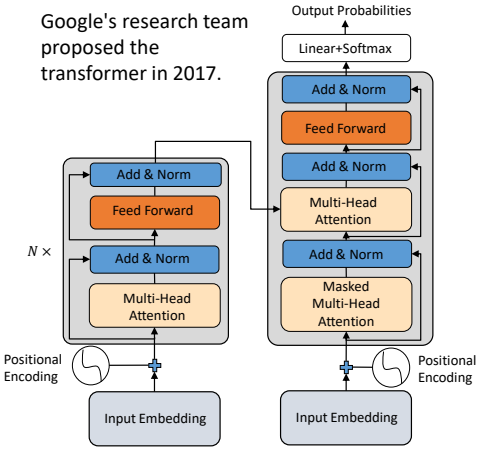
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Where Does a Foundation Model Come from? (1)



In 2014, researchers in machine translation proposed the Seq2Seq model. This was a new way to implement end-to-end machine translation based on the RNN architecture.

Google's research team proposed the transformer in 2017.

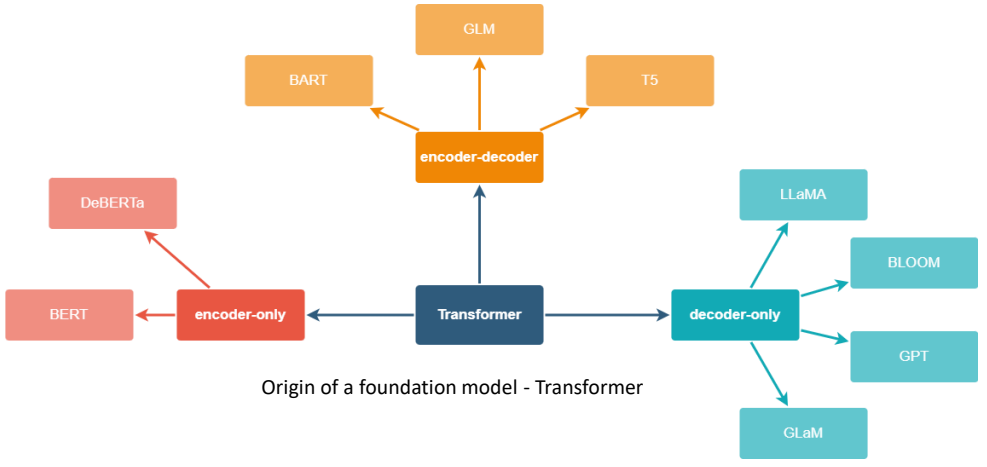


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The core idea of the Seq2Seq model is to use an encoder network to encode an input sequence into a vector in a fixed dimension or a series of hidden states, and then the decoder network generates a target sequence word by word.

Where Does a Foundation Model Come from? (2)



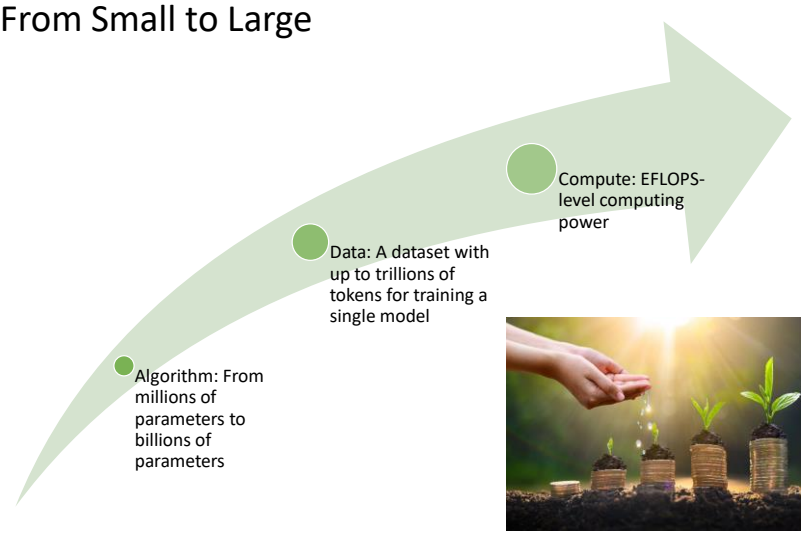
Origin of a foundation model - Transformer

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

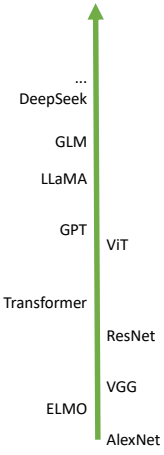
HUAWEI

- With Transformer as the infrastructure, foundation models are developed in different branches.

From Small to Large







Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Sizes of Foundation Models

- A foundation model usually has hundreds of millions of parameters and has powerful representation and generalization capabilities.

Model	Parameters (million)
ResNet101	44.55
ResNet152	60.19
Yolo v7	74.8
VGG16	138

Model	Parameters (B)	tokens
GPT3	175	3000B
GPT4	17,60 (estimated)	13T
LLaMA1	7–70	1.4T
LLaMA2		2T
LLaMA3	8–70	15T
GLM	6–130	400B
T5	11	34B
PaLM2	340	3.6T
Stable LM 2	21	2T

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Dataset

	Data Scale	Data Diversity
Small models	A dataset usually contains millions or tens of millions of data records, suitable for resource-constrained scenarios (such as mobile devices or edge devices).	The data is usually used for a specific task or domain (such as medical image classification and speech keyword recognition), and a wide coverage is not needed. The data is more centralized to optimize the model performance in specific scenarios.
Large models	Large models need to process and learn more complex relationships and patterns, and therefore require large-scale datasets. For example, the Common Crawl dataset used to train GPT-3 contains hundreds of billions of tokens, a significant increase compared with the dataset used to train a small model.	The datasets involve multiple domains and topics, including but not limited to science, technology, culture, art, history, and business. Datasets are no longer limited to a single modality or a specific domain, but present multi-modal and cross-domain characteristics.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Large Model and Small Model



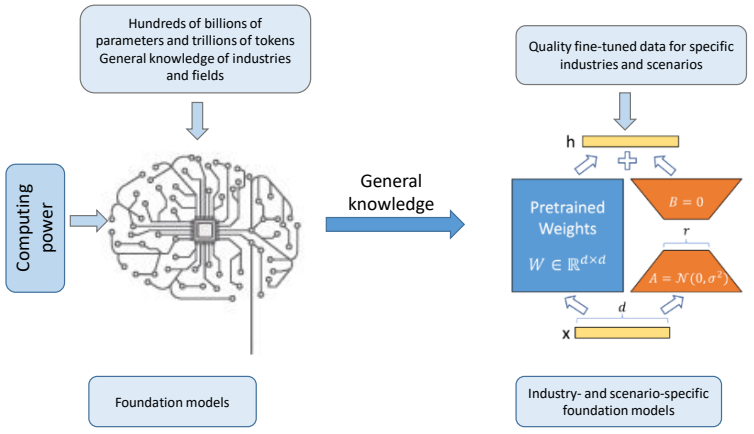
Small models are light and highly efficient.
They are suitable when resources are limited and only one task needs to be performed, for example, a watch that is only used to check the time.



Large models feature higher processing capabilities and accuracy. They are suitable when high complexity and accuracy are required.
For example, a smartwatch can be used to check the time, heartbeat, sleep, and other parameters.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

How Does a Foundation Model Learn?



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Foundation models require massive data for learning. It is costly to train models for each industry and scenario from scratch. Therefore, the learning of foundation models is based on a basic model pre-trained on massive data and then fine-tuning is performed based on industry data.

Foundation Models That Can Speak and Paint

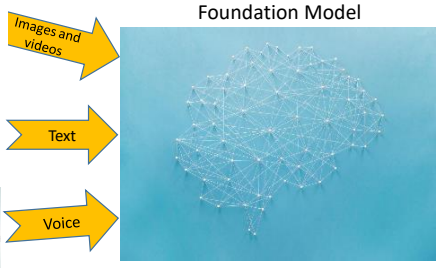
- Unlike conventional single-modal models (which process only one type of information such as text, images, or audio), a foundation model can integrate multi-modal data (text, image, video, and audio) for comprehensive understanding and inference.
- Cross-modal understanding: The relationship between different types of data can be understood. For example, information is extracted from an image and described in text, or an image or video is generated based on a text description.



Comprehensive model pipeline

Provides an end-to-end tool chain for model development, training, and inference, together with seamless coordination among DataOps + MLOps + DevOp, making development 50% more efficient.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Breakthroughs Driven by Foundation Models - Emergence

- When a complex system consists of many micro individuals, these micro individuals are combined and interact with each other. Integrating many micro individuals can result in a special phenomenon at the macro level that cannot be explained using a single unit. This phenomenon is called "emergence".
- Currently, two task types are considered to experience emergence:
  - In-context learning (ICL): ICL helps algorithms understand the meaning and relationships among words in a sentence and recognize the relationships among different objects in an image.
  - Chain-of-thought (CoT): Users write down step-by-step inference processes and provide them for a large language model (LLM) so it can perform some complex inference tasks.

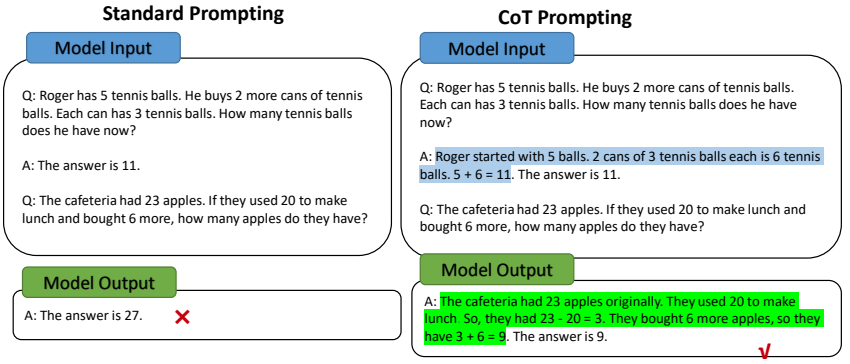


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Emergence also exists in daily life, for example, snowflake formation, traffic jams, animal migration, and vortex formation. Snowflakes are used as an example for explanation. A snowflake is composed of small water molecules. However, if a large number of water molecules interact with each other under the premise of external temperature change, a regular, symmetric, and beautiful snowflake will be formed at the macro level.
- When the model size is not large enough, tasks cannot be processed properly. However, when the model size exceeds a threshold, those tasks can be properly performed in a sudden.

Stepwise Thinking - CoT

- COT is an improved prompting strategy that helps LLMs perform better in complex inference tasks, such as arithmetic, common sense, and symbolic inference. Unlike ICL, where prompts are constructed using simple input-output pairs, CoT **combines intermediate inference steps**, which can import the final output to prompts.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

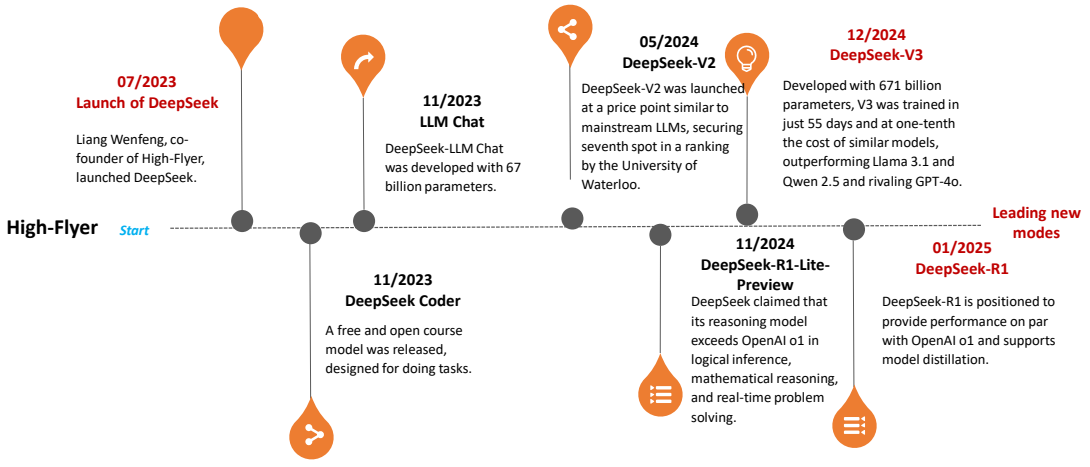
- Compared with the traditional in-context learning (which uses x1, y1, x2, y2, ... xtest as the input to enable the foundation model to provide the output ytest), the chain of thought introduces intermediate inference prompts.
- https://bbs.huaweicloud.com/blogs/406077.

Contents

1. AI Overview
2. AI Technologies
3. **Overview of DeepSeek and Its Influence on AI Development**
4. AI Applications
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

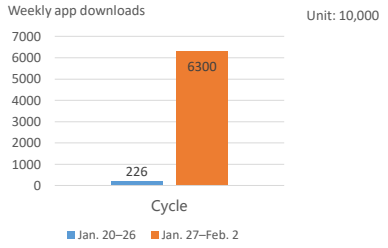
A History of DeepSeek



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

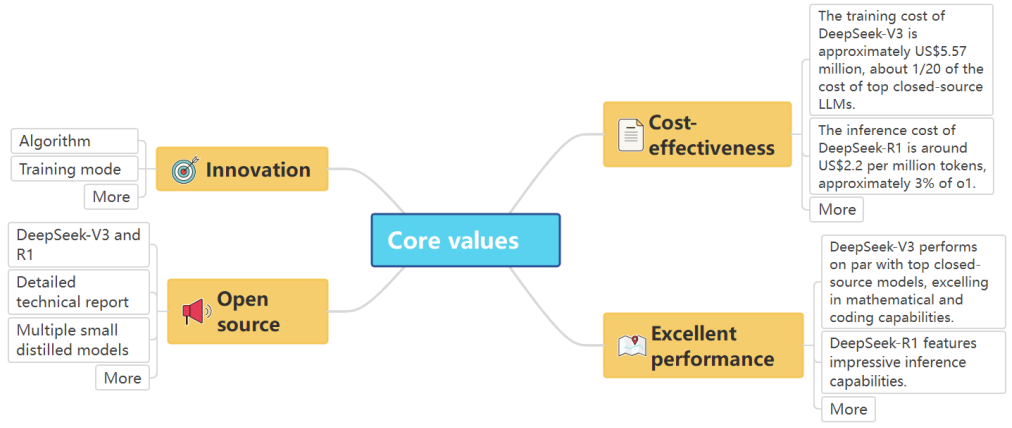
How Popular Is DeepSeek?

- On January 31, 2025, it was reported DeepSeek AI topped the list for most downloaded mobile app in 140 global markets. This led to big US companies like Microsoft, NVIDIA, and Amazon to embrace and interconnect with DeepSeek.
- As of February 9, 2025, DeepSeek had seen 110 million downloads—up from the 2.26 million downloads between January 20 and 26 and 63 million downloads the following week—with nearly 97 million weekly active users. This resulted in a week-on-week growth of 2,700%, a remarkable feat achieved without significant marketing investment.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

What Makes DeepSeek So Popular?

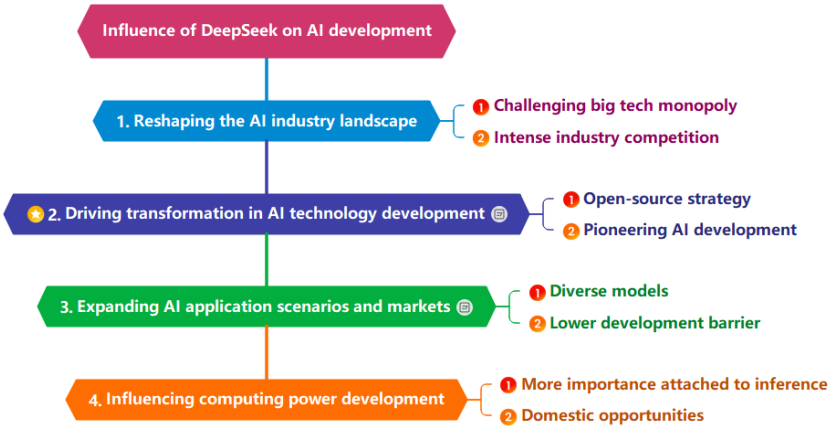


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- DeepSeek provides performance that rivals GPT-4o, but at a much lower training cost. While ChatGPT-4 ran up costs reaching US$78–100 million, DeepSeek-V3 was launched with just US$5.576 million (2.788 million H800 GPU hours)—just 5–10% of similar models.
- DeepSeek uses the new Mixture of Experts architecture, Multi-Head Latent Attention, auxiliary loss-free load balancing, and Multi-Token Prediction.
- V3 uses an innovative approach for training.

Influence of DeepSeek on AI Development



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

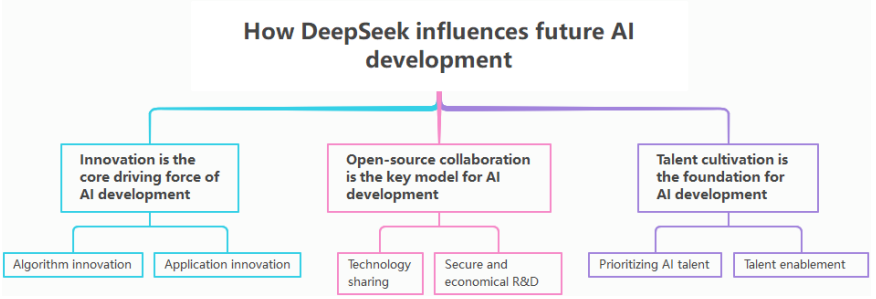
HUAWEI

- Challenging big tech monopoly: For a long time, the AI field has been dominated by a few tech giants that boast countless resources and other advantages. With DeepSeek, there is legitimate competition to the existing monopoly. DeepSeek boasts strong capabilities and a unique open-source strategy that have combined into various breakthroughs and widespread uptake in major markets. Many users and developers have moved to DeepSeek for both personal and work issues, disrupting the previous balance and prompting rival companies to reassess their market strategies.
- Intense industry competition: DeepSeek's success has inspired more companies and research institutions to invest in AI. To gain a competitive edge, major companies are investing more in R&D by introducing more cutting-edge technologies, which have not only improved technical performance but also reduced costs and expanded application scenarios. In the LLM market, this has sparked a price war, forcing other companies to lower prices to remain competitive. This greater competition has catalyzed faster technological iterations and ensured a baseline for AI development costs.
- Open-source strategy: DeepSeek is open-source, meaning it is available for anyone to download, copy, and build upon, marking a break in the typical method of closed-source AI. As part of their drive for sharing and collaboration, DeepSeek is available for developers for secondary development of new, sector-specific technologies, advancing

the application of AI in various fields.

- Pioneering AI development: Prior to DeepSeek's emergence, AI development required huge investment in both money and computing resources. This notion has changed with DeepSeek, which demonstrated that achievements can be made even under limited resource conditions through using innovative algorithms and efficient training methods. Now, AI development is not simply reliant on computing power and model scale, but on innovations in algorithms, resources, and applications, paving the way for new avenues of AI development.
- Diverse models: DeepSeek delivers powerful language understanding, text generation, mathematical reasoning, and code processing, meaning it can be applied to meet diverse needs in many different industries and domains. In finance, it can be used for risk assessment and investment decisions; in healthcare, it can assist in diagnosis and drug development; and in education, it can help develop personalized learning and intelligent tutoring. The countless benefits have seen wider support and popularity for AI in everyday life.
- Lower development barrier: Open-source models have reduced the barrier for enterprises and developers to develop AI. Instead of starting from scratch, developers instead have free tools to quickly build complex AI models or applications. Now, SMEs or other small-scale teams can develop and train applications quicker and more economically.
- More importance attached to inference: Since development costs are set to decrease with each iteration, the focus will shift to balancing cost and performance in the inference phase. This will likely lead to changes in computing power structures. The importance of inference computing power will increase significantly, driven by user needs.
- Domestic opportunities: Numerous chip manufacturers have adapted their products to DeepSeek, influencing global chip design considerations. This is advantageous for major Chinese processor companies, as it can stimulate a wider ecosystem and provide more opportunities for domestic computing power.

How DeepSeek Influences Future AI Development



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Algorithm innovation: DeepSeek achieves efficient training and powerful performance with limited resources thanks to its excellent algorithms, an enhanced Transformer architecture, and a unique attention mechanism and training framework. Since algorithms are crucial for advancing AI, companies and research institutions should increase investment accordingly, aiming to explore new paths for enhanced system performance and efficiency.
- Application innovation: Another notable aspect of DeepSeek in various fields demonstrates the potential for deep integration of AI across industries. Companies are looking to explore AI in their practices and business models to improve production and gain tangible value.
- Technology sharing: Open-source models can be shared and used by global developers, encouraging wider technology sharing and collaboration. By open-sourcing code and models, ecosystem users can work together to improve AI technology, while companies can actively participate in open-source projects to share their achievements and hear from others' experience.
- Secure and economical R&D: Open-source projects can lower R&D costs and risks. SMEs and startups can leverage open-source AI to create their products or services without

substantial investment or uncertainty from starting from scratch. Further feedback from the open-source community can help refine and enhance products for added market competitiveness.

- Prioritizing AI talent: The success of DeepSeek can be attributed to the exceptional AI team working behind the scenes. With the demand for AI growing every year, there is an equal demand for talented personnel who have a solid foundation in mathematical and computer science, expertise in core AI technologies, and a drive for innovation. Governments, universities, and enterprises should collaborate to provide initiatives that encourage the next generation of AI talent. This includes boosting investments in AI education and nurturing more professionals based on industry demands.
- Talent enablement: Enterprises and research institutions need to create a favorable environment to attract and retain excellent AI personnel. This means providing competitive compensation, development opportunities, a supportive atmosphere, and encouraging innovation and exploration. It is also important to ensure objective and fair talent evaluation that prioritizes actual achievements.

DeepSeek in the Industry

- Major companies inside and outside China are moving to use DeepSeek or integrate it with their existing applications.
- Owing to its powerful features and open-source nature, the AI has been widely adopted in many sectors, catalyzing significant technological advancements in all fields.



Henan Yunfei Technology Development Co., Ltd. partnered with DeepSeek to explore its potential in agricultural plant protection thanks to robust data analysis. This led to better capabilities in identifying patterns of and predicting the risk of disease and pest outbreaks. The AI can recommend agricultural plant protection tailored to farms and crops based on real-time field data.

On February 5, Huawei officially linked its Celia app with DeepSeek-R1 (beta). This strategic partnership has enhanced Celia for programmers and coders, students struggling with complex math calculations, or professionals needing logical reasoning analysis. Now, users can access the DeepSeek-R1 model through the Celia app to quickly receive accurate answers.





On February 7, Geely Auto announced its self-developed Xingrui AI large model will integrate with DeepSeek.

On February 7, Dongfeng Motor announced its entire passenger vehicle lineup is integrated with DeepSeek LLMs. Wholly-owned brands it has including MHERO, eπ, Aeolus, and Nammi will soon adopt the AI-powered system.

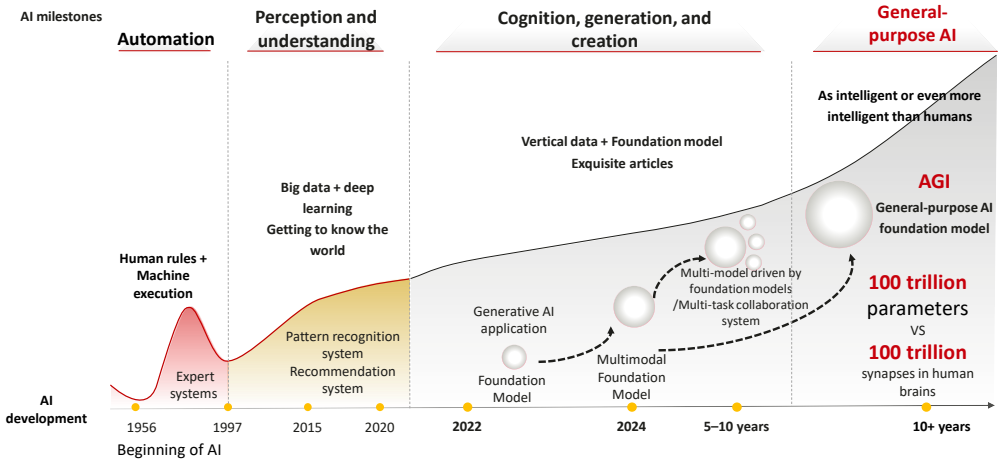
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. AI Technologies
3. Overview of DeepSeek and Its Influence on AI Development
4. **AI Applications**
   - AI Application History
   - AI Application Cases
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Application History - From Perception and Understanding to Creation and Generation



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Weighty Application - ChatGPT

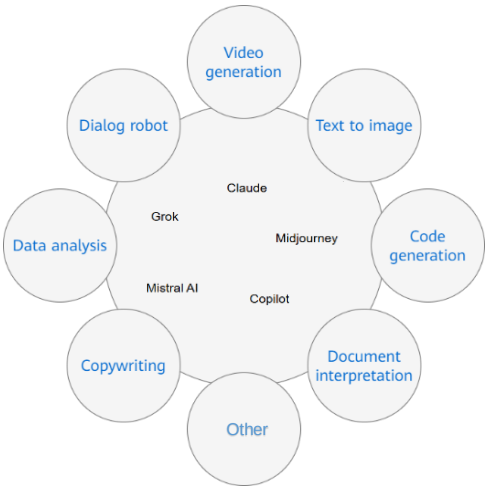
- ChatGPT is a next-generation dialog-based AI system developed by OpenAI based on its GPT language model.
  - ChatGPT had over 100 million active users only two months after its launch, making it the fastest growing consumer-level application in history.
- ChatGPT is acclaimed for human-like understanding and inference capabilities in language, context, intent, and logic. It has outstanding performance in high-quality, multi-round, and long-text human-machine interaction, and has achieved preliminary "intelligent emergence."
- ChatGPT is currently in its GPT4-o version with multi-modal capabilities and multi-domain expertise.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

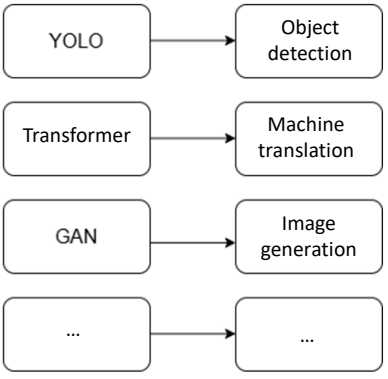
Applications Similar to ChatGPT

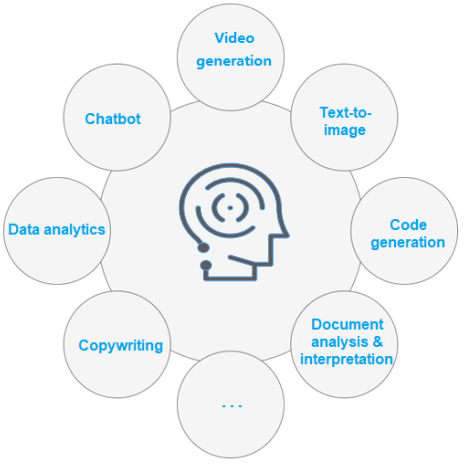
- After the release of ChatGPT, many similar applications have been released in China and globally.
- Today, these applications have integrated multiple functions, such as text, code, video, and image generation.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

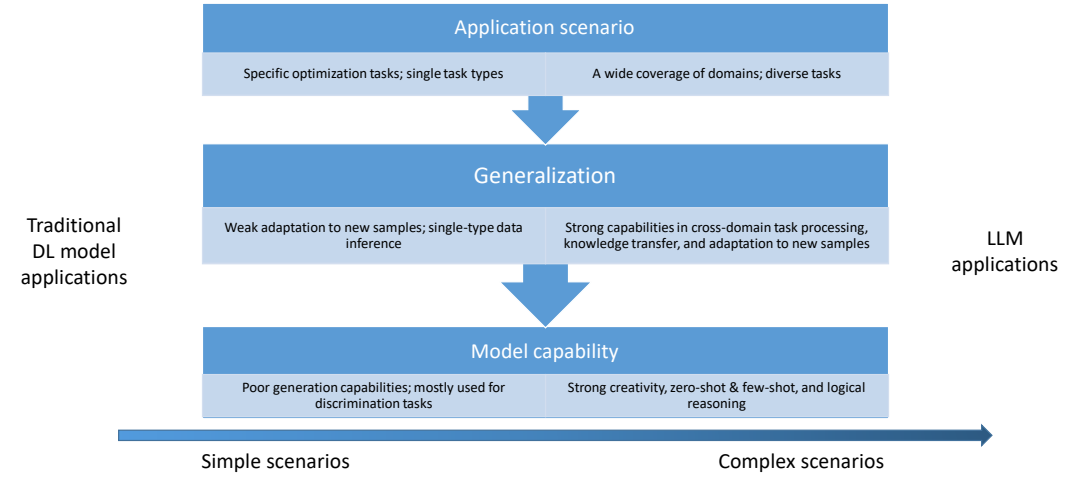
From Single Tasks to Multiple Tasks





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

From Simple Tasks to Complex Tasks



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. AI Technologies
3. Overview of DeepSeek and Its Influence on AI Development
4. **AI Applications**
   - AI Application History
   - AI Application Cases
5. Debates and Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Intelligent Mining

Scenario 1: Electromechanical chamber - storing power distribution and communications equipment

Objective: **To implement unattended automatic inspection**

Challenge 1: 24-hour monitoring of the chamber environment



Challenge 2: Scheduled manual inspection



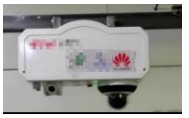
Scenario 2: Underground intelligent monitoring

	Challenges
Coal conveyor belt	It's unsafe and time-consuming to inspect a coal conveyor belt longer than **20 km**. Multiple belt segments need to collaborate to prevent coal accumulation.
Behavior protocol violations	Not wearing safety helmets, smoking, passing under equipment, walking in laneways, and sitting on belts
Operational violations	Improper operations during gas drainage and water inspection and drainage

Solution for scenario 1: Automatic inspection robot

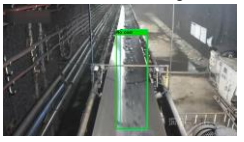
Video surveillance: **AI** machine vision for 24-hour monitoring, alarms, and sensitive area inspection

Sound detection: **AI** intelligently identifies whether machines are running properly



Scenario 2

Belt tear monitoring



Belt deviation monitoring



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://e.huawei.com/cn/material/wireless/4d008289c5424b31a055c71eaaed5790

AI Helps Protect Nature - Preserving Chile's Biodiversity

- The Nature Guardian project uses Huawei Cloud and AI to research and protect Darwin's foxes in the Nahuelbuta Mountains. The Nature Guardian is an acoustic monitoring system developed by Rainforest Connection (RFCx) and has been effective in several projects.
- It consists of solar devices equipped with microphones and antennas. These devices collect sound data from the surrounding environment and transmit it to the cloud through wireless networks for AI data analysis. Each device can cover three square kilometers around the clock.
- The Nature Guardian can capture animal sounds as well as illegal noise made by poachers' gunshots or trucks and electric saws of illegal loggers.
- The trained AI model can identify the sounds of different animals, enabling experts to study the distribution and behavior of species and helping with environmental protection through adaptive management.
- If a threat is identified, the system sends a real-time alarm to the ranger's mobile application for a fast response.



Darwin's Fox

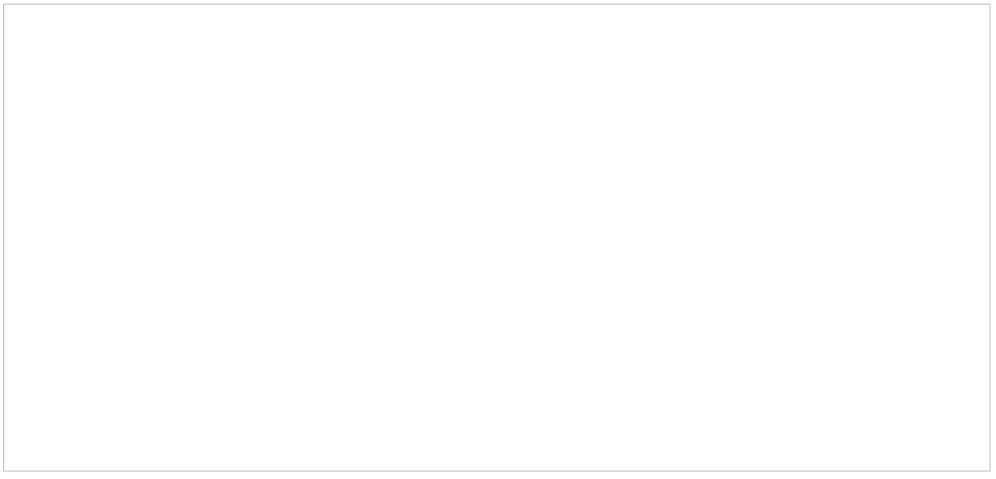
Source:https://www.huawei.com/cn/tech4all/stories/nature-guardians-for-biodiversity-in-chile

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://www.huawei.com/cn/tech4all/stories/nature-guardians-for-biodiversity-in-chile

AI Safeguards Nature - Protecting Wildlife in Greece with a Shield of Sound



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- https://www.huawei.com/cn/tech4all/stories/wildlife-greece

Scaling Law in AI for Science

Multi-disciplinary and multi-scale data to train four fundamental science models: life, materials, fluids, and electromagnetism


Atomic orbitals

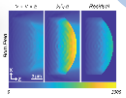

Crystal structure

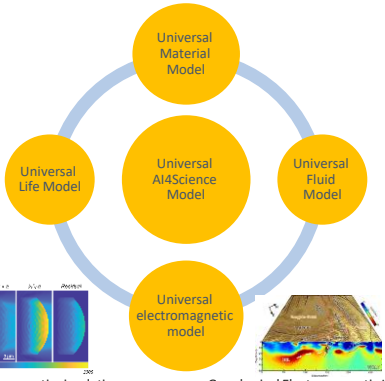

Small molecule

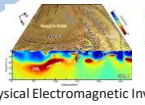

Proteins


Cell

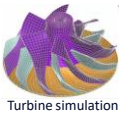

Brain

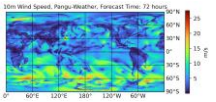

Electromagnetic simulations




Geophysical Electromagnetic Inversion




Turbine simulation

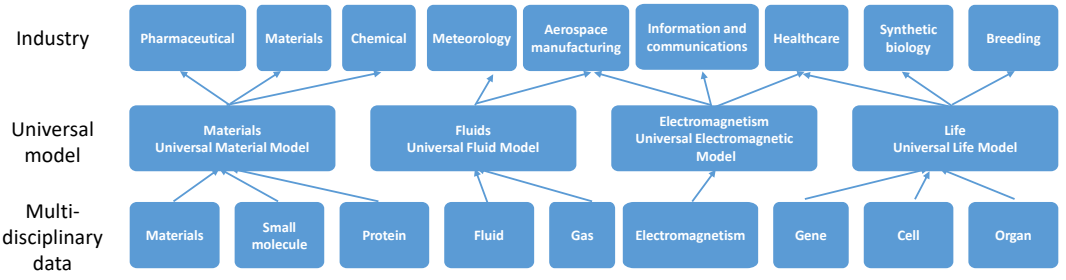

Weather forecasting

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

AI4Science

Emergent intelligence based on massive data in various industries



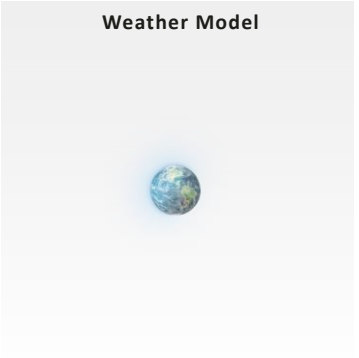
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI4Science - AI + Meteorology

Weather models for fast, accurate weather forecasting

80

typhoons worldwide each year



3D weather model | Hierarchical temporal aggregation

More accurate

Forecasting accuracy **20%** ↑

Faster and greener

Compute used for generating a 10-day weather forecast

Traditional Numerical Weather Prediction (NMP)	Weather model inference
3,000 servers	1 device
5 hours	10 seconds

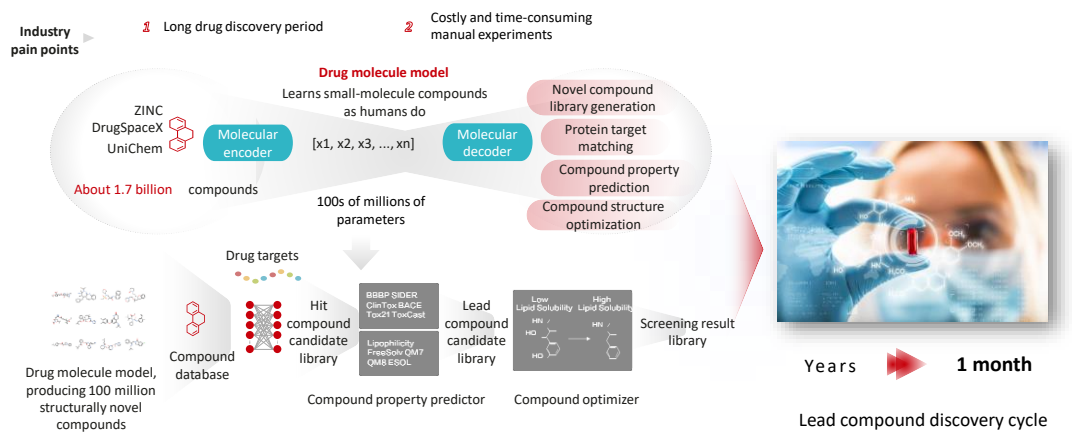
Forecasting speed **10,000x** ↑

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

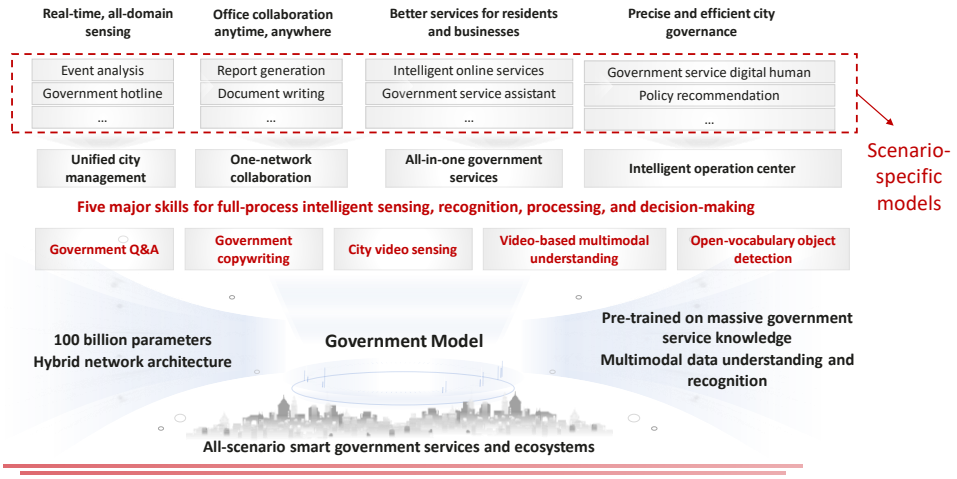
AI4Science - AI + Drugs

Drug molecule models for much higher drug design efficiency



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Smart Government



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI For Coding

How to format a large number to two decimal places without scientific notation in ArkTS?

Solution for Formatting Large Numbers in ArkTS

Implementation Approach

1. **String-Based Processing**
   Directly manipulate numeric values as strings to prevent precision loss and avoid automatic scientific notation conversion.
2. **Scientific Notation Handling**
   Implement custom logic or integrate libraries like `bignumber.js` (recommended) for accurate scientific notation conversion.
3. **Decimal Precision Control**
   - Split the number into integer and decimal parts
   - Truncate decimals to 2 digits using `.slice(0, 2)`
   - Ensure two decimal places with `.padEnd(2, '0')`

Core Implementation

typescript

```typescript
// Formatting function
function formatLargeNumber(value: string): string {
  // Handle scientific notation (custom logic required)
  let numStr = value.includes('e') ? convertScientificNotation(value) : value;

  // Split integer and decimal parts
  const [integerPart, decimalPart = '00'] = numStr.split('.');

  // Format decimals
  const formattedDecimal = decimalPart.slice(0, 2).padEnd(2, '0');
```

3. **Library Integration**

typescript

```typescript
import BigNumber from "bignumber.js";
// Recommended for precise large number handling
const x = new BigNumber('1234567890123456789O.1234567890');
```

HarmonyOS Component Integration

typescript

```typescript
@Entry
@Component
struct DemoPage {
  @State inputValue: string = '';
  @State displayValue: string = '';

  build() {
    Column() {
      TextInput({ text: this.inputValue })
        .onChange((value: string) => {
          this.displayValue = formatLargeNumber(value);
        })
      Text(`Formatted: ${this.displayValue}`)
    }
  }
```

Best Practices

- Follow HarmonyOS performance specifications
- Use `@State` decorators for UI updates
- Validate input formats in `onChange` events
- Combine with locale-aware formatting when displaying currencies/measurements

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. AI Technologies
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. **Debates and Future of AI**
   - Debates in AI
   - Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Debates in AI

- There are many debates around AI technologies, especially as they evolve:
  - Is what we see or hear true?
  - How do we solve the ethical issues in AI?
  - Will everyone lose their jobs in the future?
  - Will our privacy be protected? How do we protect our privacy?
  - Will AI be controllable in the future?
  - ...



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- What you see may not be true.
- Ethical issues will be controlled by policies and regulations.
- Partial unemployment may be caused thereby.
- Privacy may possibly be leaked. However, privacy can be protected depending on technologies such as differential privacy training, model fingerprint, and model encryption.
- Controllability.

Is Seeing Still Believing?

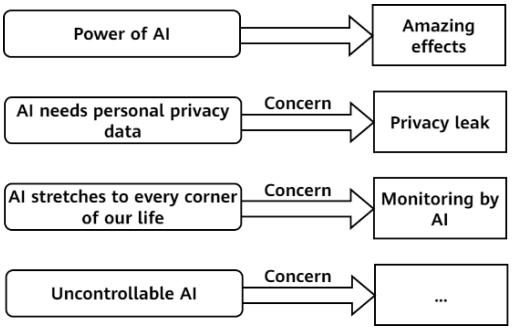
- As AI technologies develop, we begin to question the credibility of images, audio, and video. Now, technologies such as multi-modal large models and generative adversarial network (GAN) can be used to produce false images and videos, making it difficult to distinguish between what is true and what is false.
  - For example, Lyrebird is a tool that can automatically imitate human voices from several minutes of recording samples.
  - Deepfake can generate videos with fake faces.

True?  False?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Opinions on Ethical Issues in AI

- Discrimination and stigmatization: Threatens social fairness and justice and restricts people's freedom of choice.
- Autonomous decision-making: This mechanism endangers human autonomy. Results cannot be traced back and effective measures cannot be taken to correct decisions.
- Disadvantages: The objectives, methods, and decisions of AI systems cannot be explained.
- Harm to human dignity and mind, and physical damage: People may face unsafe environments (for example, coexistence with robots) and vulnerability to the malicious use of robots, privacy leaks, and susceptibility to deception and manipulation.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Will You Lose Your Job?

- As a society, we have always been in pursuit of higher efficiency. The steam engine reduced the need for horses. Every step in achieving automation will change our life and work. In the era of AI, what jobs will be replaced by AI?
- AI may replace:
  - Repetitive work
  - Hazardous work
  - ...



AI

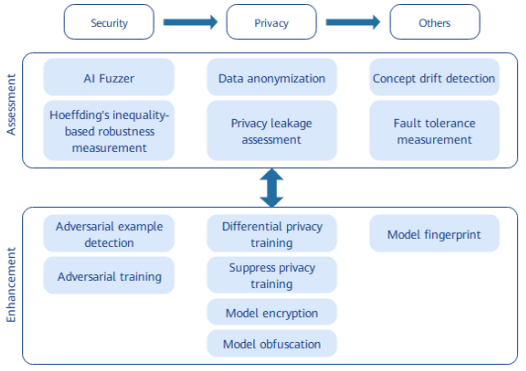
What opportunities and jobs will AI bring?



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Privacy and Data Security

- Big data and AI allow us to obtain information more conveniently and efficiently. However, our behavior is also recorded and the data is used all the time. How can we protect private data?
  - 1. Users: Before obtaining the user's information, mobile applications must obtain the user's consent (**privacy policy**). Users can refuse to provide the information required by an application for proactive protection.
  - 2. Society: **Actively promote the formulation of privacy protection clauses, implement basic industry rules**, and strengthen supervision.
  - 3. Developers: **Confidential computing, model privacy protection, federated learning, or adversarial learning** can be used to prevent reverse attacks and protect user privacy. Alternatively, a well-established privacy protection framework.



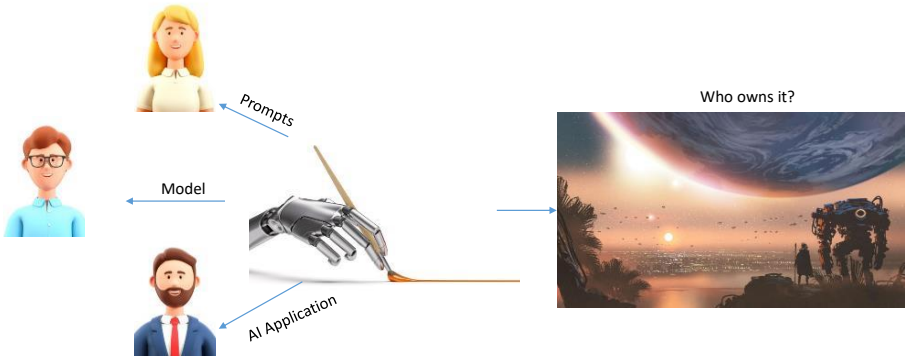
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Confidential computing: Data transmission and computing are confidential, and privacy protection is costly.
- Model privacy protection: Commonly used methods include data anonymization, differential privacy training, model encryption, and model obfuscation. (Differential privacy can measure and control the leakage of trained data by models.)
- Federated learning: Generally, multi-party machine learning is performed without sharing data. It is essentially a distributed machine learning framework with limited data access. Computing nodes share only gradients but not raw data.

Problems to Be Solved

- Are AI-created works protected by copyright laws?
- Who gives authority to robots? What rights shall be given to robots?
- ...



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Overview
2. AI Technologies
3. Overview of DeepSeek and Its Influence on AI Development
4. AI Applications
5. **Debates and Future of AI**
   - Debates in AI
   - Future of AI

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Value - Wide Application

- AI will see wider application as vertical fields integrate the technology.
  - Smart transportation: highways, airports, railways, logistics, ports
  - Energy: electricity, oil, gas
  - Finance: banking, insurance, and securities
  - And more

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Digital government: smart city, smart government, smart emergency response, sunny kitchen, smart water conservancy, and the like. For example, sunny kitchen uses AI machine vision to identify food types on meal plates and calculate prices and calories. Smart water conservancy and management can solve problems in the following aspects: 1. black water management and sewage monitoring (AI video algorithm for sewage identification, satellite remote sensing, and water pump gate scheduling); 2. reservoir (enhanced video, intelligent power supply, and integrated pole site for water conditions) 3. rivers and lakes (spectral water quality analysis, and 5G access to drones/ships).
- Smart mining: underground 5G network, security monitoring, and violation detection.
- Smart transportation: airports (IOC queuing management – sensing an extra long passenger queue: AI + video analysis; stand scheduling: operation optimization AI algorithm + big data + IoT + GIS + video + simulation...), and highways (holographic intersection, unmanned driving...).

Value - Growing Market

Market scale

The global AI market was expected to reach USD638.2 billion in 2024 (USD538.1 billion in 2023).

Total AI IT investment

In 2022, the total global investment in AI IT was USD128.8 billion, which is estimated to rise to USD423.6 billion in 2027, representing a five-year compound annual growth rate (CAGR) of about 26.9%.

Foundation Model Solutions

In the first half of 2024, the market scale of AI foundation model solutions in China was CNY1.38 billion. The estimated CAGR is 56.2% from 2024 to 2028, reaching a market scale of CNY21.1 billion by 2028.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Value - Higher Production Efficiency

- AI innovation plays a prominent role in advancing the manufacturing industry. Smart manufacturing can significantly improve product quality and production efficiency. Statistics show that AI reconstruction helps speed up R&D in factories by 20.7% and improve production efficiency by 34.8%.
- Labor-intensive and technology-intensive enterprises should make full use of AI innovation. The former can use AI for higher labor productivity, while the latter has strong R&D capabilities and is somewhat ready for AI innovation.



A group of people work together.

A few people control a group of machines.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Value - Industry-specific AI Applications

- In addition to offering general capabilities, AI technologies are also customized for vertical industries to solve pain points in specific domains and provide differentiated solutions.
- Industry-specific customization unlocks the true value of AI technologies. In the future, competition will increasingly hinge on a deep understanding of vertical scenarios.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Technology - Artificial General Intelligence

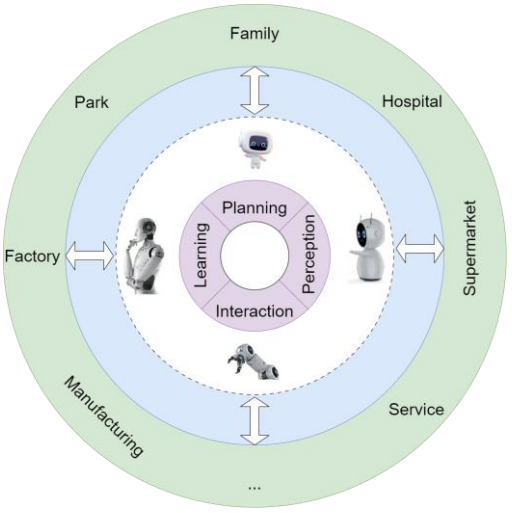
- AGI is a computer system that can implement any intelligent human activity. It features universal intelligence, including self-learning, self-improvement, self-adjustment, and cross-domain and cross-modal learning, inference, and decision-making capabilities.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Technology - Embodied Artificial Intelligence

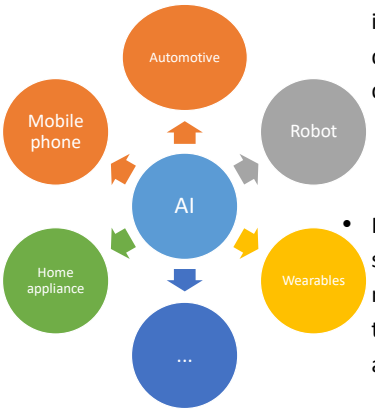
- Embodied artificial intelligence is an intelligent system that implants AI in physical objects. This gives objects capabilities like autonomous perception, learning, decision-making, and action in physical environments, so they can flexibly adapt to physical environments and tasks.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Devices

- Smartphones will integrate AI, enabling natural interactions and becoming truly smart. They will deliver a brand-new experience together with wearable devices.
- Smart home appliance will provide recipe recommendations, optimize energy usage, and offer language interaction for a more comfortable experience.

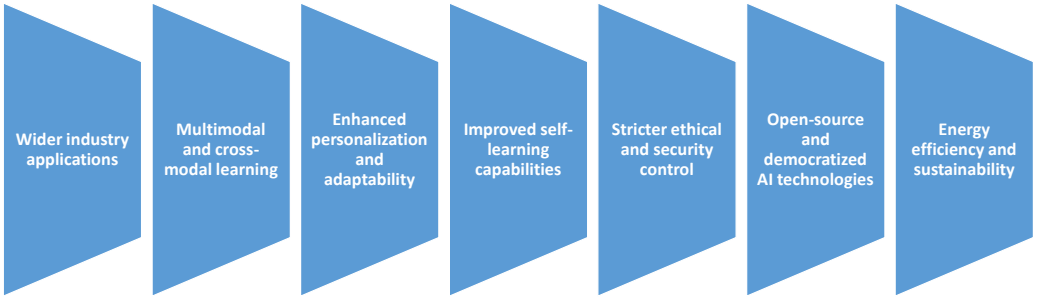


- Intelligent vehicles will achieve intelligent driving in a real sense, delivering safe and comfortable driving experiences.
- Intelligent robots will feature stronger sensing and decision-making capabilities, ready to take on jobs like housekeeping and nursing.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Future of AI Applications

- The future of AI applications is expected to see wider technology integration, deeper industry penetration, and continuous ethical and security considerations.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Wider industry applications: As technologies become more mature and adaptable, large models are poised to find broader applications in traditional industries, such as manufacturing, energy, and logistics, to support more efficient problem-solving and decision-making.
- Multimodal and cross-modal learning: Future large models will not be limited to a single domain such as text or images. Instead, they will be able to process and integrate multiple types of data (such as text, images, videos, and sounds) to provide a more comprehensive AI application experience.
- Enhanced personalization and adaptability: Large models will better adapt to specific needs of individual users and provide customized services. For example, in the education and health fields, models can provide personalized suggestions based on individuals' learning progress or health status.
- Improved self-learning capabilities: Future large models will have stronger self-learning capabilities, allowing them to learn new patterns and knowledge through continuous interaction and feedback. This reduces the reliance on large-scale labeled data.
- Stricter ethical and security control: As large models become more widespread, ethical, privacy, and security issues will receive more attention. Future development will focus

on ensuring the transparency, explainability, and fairness of models while strengthening data protection.

- Open-source and democratized AI technologies: The development and application of large models may become more democratized. More research institutions and companies may make their models and technologies open-source to promote global technology sharing and innovation.
- Energy efficiency and sustainability: Given the enormous energy consumption required to train large models, future research will focus on improving the energy efficiency of models and reducing the carbon footprint to support sustainable development.

Summary

- This chapter covered the definition and history of AI, its applications and sectors, and controversial topics and future trends in the field.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Multi-choice question) Which of the following are applications of computer vision?
   A. Knowledge graph
   B. Semantic segmentation
   C. Intelligent driving
   D. Video analysis

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answer:
  - BCD

Recommendations

- Huawei Talent Online Website
  - https://e.huawei.com/cn/talent/#/home
- Huawei Cloud
  - https://www.huaweicloud.com/intl/en-us/

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

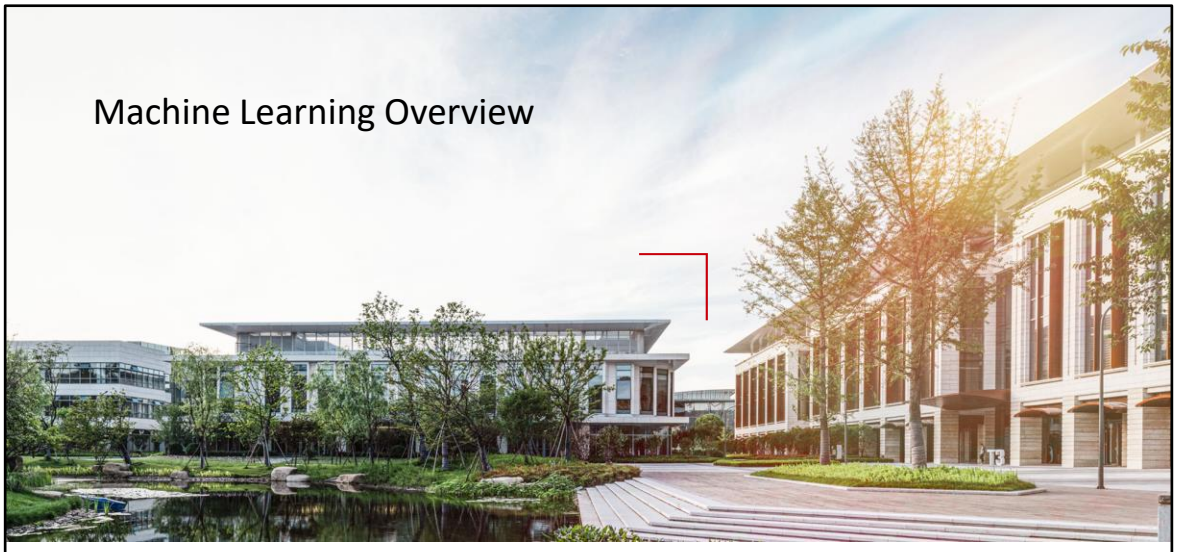
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2025 Huawei Technologies Co., Ltd. All rights reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Machine Learning Overview



HUAWEI

Objectives

- Upon completion of this course, you will understand:
  - Learning algorithm definitions and machine learning process
  - Related concepts such as hyperparameters, gradient descent, and cross-validation
  - Common machine learning algorithms

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

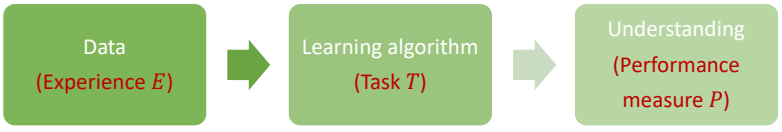
Contents

1. **Machine Learning Algorithms**
2. Types of Machine Learning
3. Machine Learning Process
4. Important Machine Learning Concepts
5. Common Machine Learning Algorithms

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Machine Learning Algorithms (1)

- Machine learning is often combined with deep learning methods to study and observe AI algorithms. A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

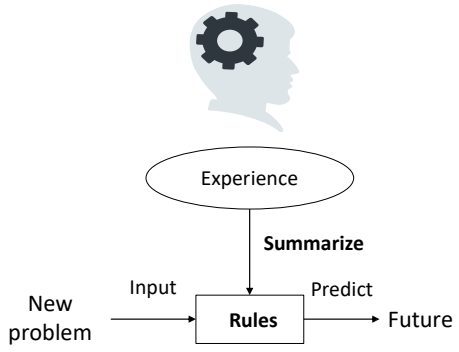


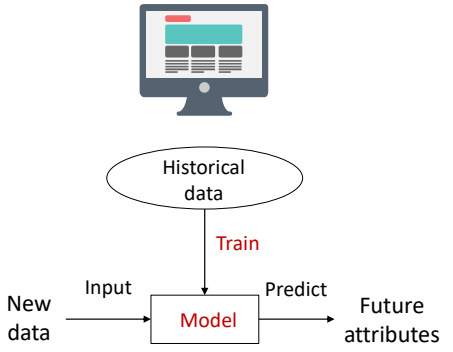
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Deep learning is a sub-field of machine learning. To understand deep learning, you need to first understand the fundamentals of machine learning.
  - Task T represents how the machine learning system should process a sample.
  - Performance measure P is used to evaluate aspects of machine learning such as accuracy and error rate.
  - Experience E: Most learning algorithms can be understood as being allowed to experience an entire dataset. Some machine learning algorithms do not just experience a fixed dataset. For example, reinforcement learning algorithms interact with an environment, so there is a feedback loop between the learning system and its training process. Machine learning algorithms can be broadly categorized as unsupervised or supervised by what kind of experience they are allowed to have during the learning process.
- To learn the game of Go:
  - Experience E_1 :Game with itself- Unsupervised and indirect learning
  - Experience E_2 :Inquiring Humans during a game with itself – Semi-supervised learning
  - Experience E_3 :Learning from human historical games - Supervised and direct learning

- Handwriting recognition problem:
    - Task T: Recognize handwritten text
    - Performance measure P: classification accuracy
    - Experience E: Classified sample library (supervised and direct learning)
- Robots' desire to advance: Look for new games and practice their skills through tiny changes in the same situation, enriching their training examples.

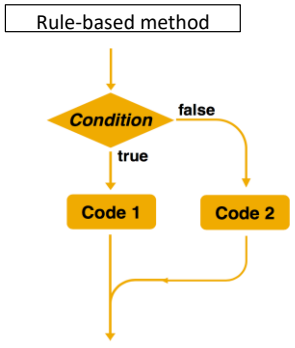
Machine Learning Algorithms (2)



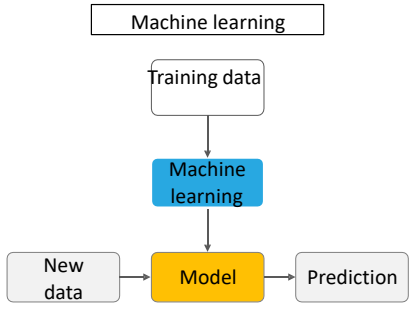


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Differences Between Machine Learning Algorithms and Traditional Rule-based Methods



- Explicit programming is used to solve problems.
- Rules can be manually determined.



- Models are trained on samples.
- Decision-making rules are complex or difficult to describe.
- Machines automatically learn rules.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

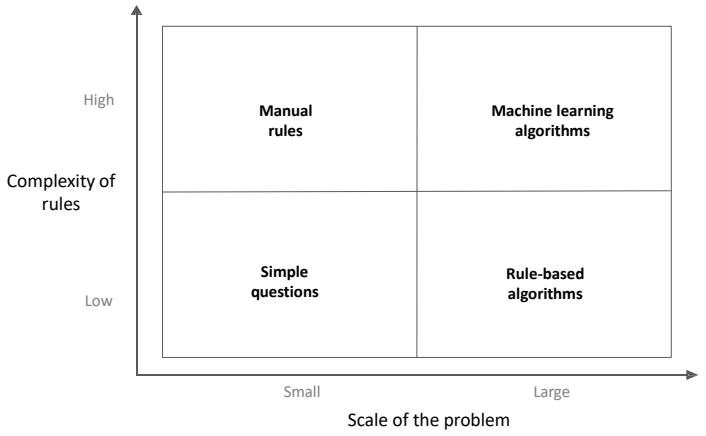
When to Use Machine Learning (1)

- Machine learning provides solutions to complex problems, or those involving a large amount of data whose distribution function cannot be determined.
- Consider the following scenarios:

Rules are complex or difficult to describe, for example, speech recognition.	Task rules change over time, for example, part-of-speech tagging, in which new words or word meanings can be generated at any time.	Data distribution changes over time and programs need to adapt to new data constantly, for example, sales trend forecast.
		

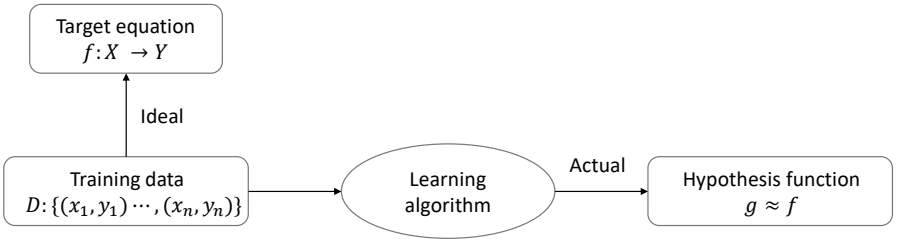
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

When to Use Machine Learning (2)



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Rationale of Machine Learning Algorithms



- The objective function f is unknown, and the learning algorithm cannot obtain a perfect function f.
- Hypothesis function g approximates function f, but may be different from function f.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Main Problems Solved by Machine Learning

- Machine learning can solve many types of tasks. Three most common types are:
  - **Classification**: To specify a specific one of the *k* categories for the input, the learning algorithm usually outputs a function $f: R^n \rightarrow (1,2,\dots,k)$. For example, image classification algorithms in computer vision solve classification tasks.
  - **Regression**: The program predicts the output for a given input. The learning algorithms usually output a function $f: R^n \rightarrow R$. Such tasks include predicting the claim amount of a policy holder to set an insurance premium or predicting the security price.
  - **Clustering**: Based on internal similarities, the program groups a large amount of unlabeled data into multiple classes. Same-class data is more similar than data across classes. Clustering tasks include search by image and user profiling.
- **Classification and regression are two major types of prediction tasks. The output of classification is discrete class values, and the output of regression is continuous values.**



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Machine Learning Algorithms
2. **Types of Machine Learning**
3. Machine Learning Process
4. Important Machine Learning Concepts
5. Common Machine Learning Algorithms

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Types of Machine Learning

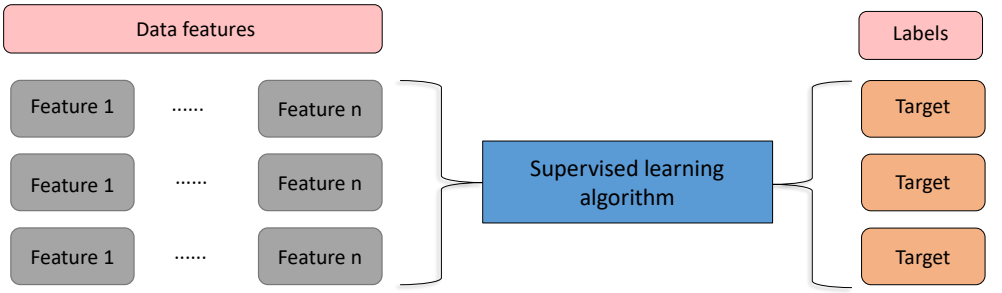
- **Supervised learning**: The program takes a known set of samples and trains an optimal model to generate predictions. Then, the trained model maps all inputs to outputs and performs simple judgment on the outputs. In this way, unknown data is classified.
- **Unsupervised learning**: The program builds a model based on unlabeled input data. For example, a clustering model groups objects based on similarities. Unsupervised learning algorithms model the highly similar samples, calculate the similarity between new and existing samples, and classify new samples by similarity.
- **Semi-supervised learning**: The program trains a model through a combination of a small amount of labeled data and a large amount of unlabeled data.
- **Reinforcement learning**: The learning systems learn behavior from the environment to maximize the value of reward (reinforcement) signal function. Reinforcement learning differs from supervised learning of connectionism in that, instead of telling the system the correct action, the environment provides scalar reinforcement signals to evaluate its actions.
- Machine learning evolution is producing new machine learning types, for example, self-supervised learning, contrastive learning, generative learning.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Supervised learning: We give a computer a bunch of choice questions (training samples) and provide standard answers. The computer tries to adjust its model parameters to make their predictions close to standard answers. In this way, the computer learns how to deal with this type of problem. Then the computer can help us reply to choice questions whose answers are not given (test samples).
- Unsupervised learning: We give a computer a bunch of choice questions (training samples), but do not provide standard answers. The computer tries to analyze the relationships between these questions and classify them. It does not know the answers to these questions, but it thinks that the answers to the questions in the same category should be the same.
- Semi-supervised learning: Traditional supervised learning uses a large number of labeled training samples to create a model for predicting the labels of new samples. For example, in a classification task, a label indicates the category of a sample while in a regression task, a label is a real-value output of the sample. As data storage capabilities are developing, we can easily obtain a large amount of unlabeled data from many tasks. However, labeling the data is labor-consuming and time-consuming. For example, for web page recommendation, users need to label web pages they like, but only a few users are willing to spend a lot of time doing this. Then we get limited labeled web page data and a large amount of unlabeled web page data.

- Reinforcement learning: We give a computer a bunch of choice questions (training samples), but do not provide standard answers. It tries to solve these questions, and we check whether the answers are correct as teachers. If the computer generates more correct answers, we offer more rewards. The computer adjusts its model parameters to make its predictions correct and obtain more rewards. Not strictly speaking, reinforcement learning can be understood as unsupervised learning plus supervised learning (unsupervised learning before supervised learning).

Supervised Learning



Weather	Temperature	Wind Speed
Sunny	High	High
Rainy	Low	Medium
Sunny	Low	Low

Suitable for Exercise
Yes
No
Yes

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Supervised Learning - Regression

- **Regression** reflects the features of sample attributes in a dataset. A function is used to express the sample mapping relationship and further discover the dependency between attributes. Examples include:
  - How much money can I make from stocks next week?
  - What will the temperature be on Tuesday?

Monday	Tuesday
38°	?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

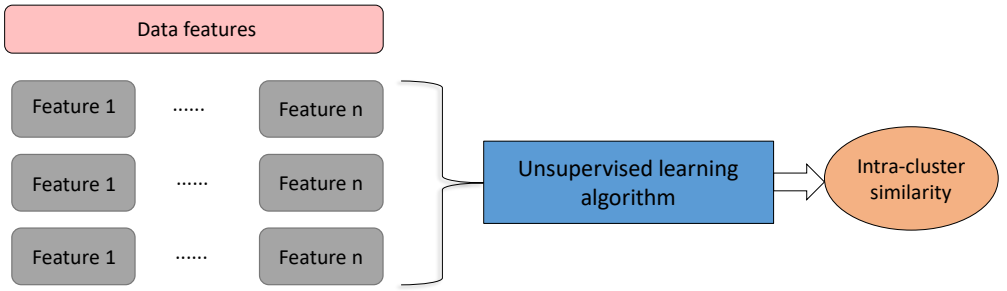
Supervised Learning - Classification

- **Classification** uses a classification model to map samples in a dataset to a given category.
  - What category of garbage does the plastic bottle belong to?
  - Is the email a spam?



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Unsupervised Learning



Monthly Sales Volume	Product	Sale Duration
1000-2000	Badminton racket	6:00-12:00
500-1000	Basketball	18:00-24:00
1000-2000	Game console	00:00-6:00

Category
Cluster 1
Cluster 2

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

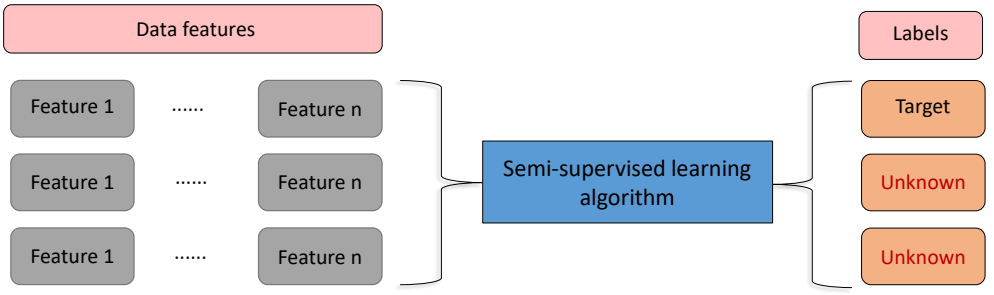
Unsupervised Learning - Clustering

- **Clustering** uses a clustering model to classify samples in a dataset into several categories based on similarity.
  - Defining fish of the same species.
  - Recommending movies for users.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Semi-supervised Learning



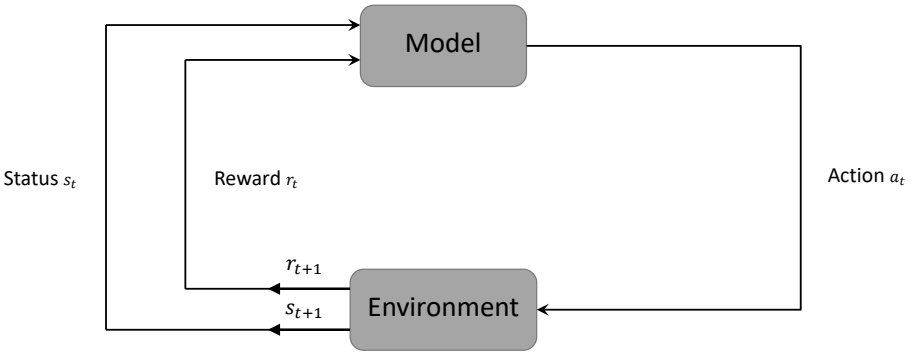
Weather	Temperature	Wind Speed
Sunny	High	High
Rainy	Low	Medium
Sunny	Low	Low

Suitable for Exercise
Yes
/
/

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Reinforcement Learning

- A reinforcement learning model learns from the environment, takes actions, and adjusts the actions based on a system of rewards.

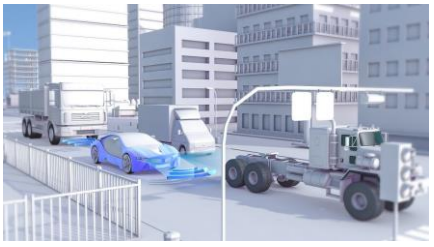


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Reinforcement learning uses a series of actions to maximize the reward function to learn a model.
- Both good and bad behavior can help reinforcement learning models learn.
- Example: Autonomous vehicles learn by continuously interacting with the environment

Reinforcement Learning - Best Action

- **Reinforcement learning** always tries to find the best action.
  - Autonomous vehicles: The traffic lights are flashing yellow. Should the vehicle brake or accelerate?
  - Robot vacuum: The battery level is 10%, and a small area is not cleaned. Should the robot continue cleaning or recharge?





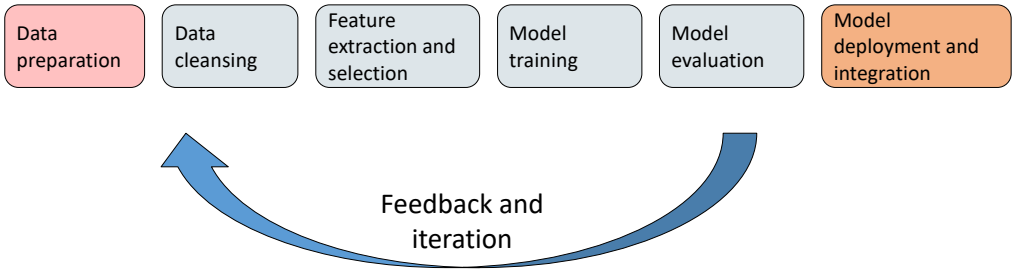
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Machine Learning Algorithms
2. Types of Machine Learning
3. **Machine Learning Process**
4. Important Machine Learning Concepts
5. Common Machine Learning Algorithms

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Machine Learning Process



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Machine Learning Basic Concept - Dataset

- **Dataset**: collection of data used in machine learning tasks, where each piece of data is called a sample. Items or attributes that reflect the presentation or nature of a sample in a particular aspect are called **features**.
  - **Training set**: dataset used in the training process, where each sample is called a training sample. **Learning (or training)** is the process of building a model from data.
  - **Test set**: dataset used in the testing process, where each sample is called a test sample. Testing refers to the process, during which the learned model is used for prediction.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Overview

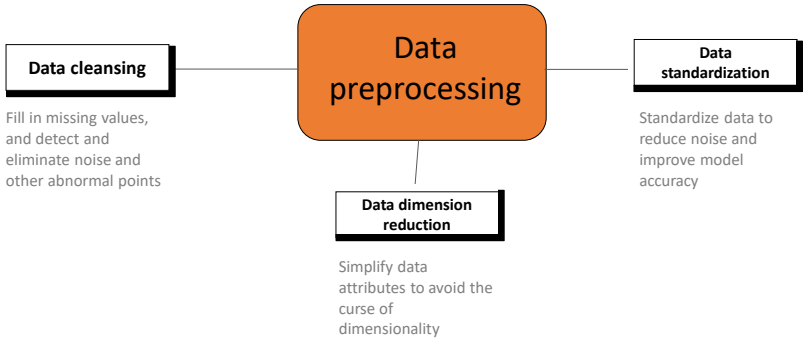
- Typical dataset composition

		Feature 1	Feature 2	Feature 3	Label
	No.	Area	Location	Orientation	House Price
Training set	1	100	8	South	1000
	2	120	9	Southwest	1300
	3	60	6	North	700
	4	80	9	Southeast	1100
Test set	5	95	3	South	850

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Importance of Data Processing

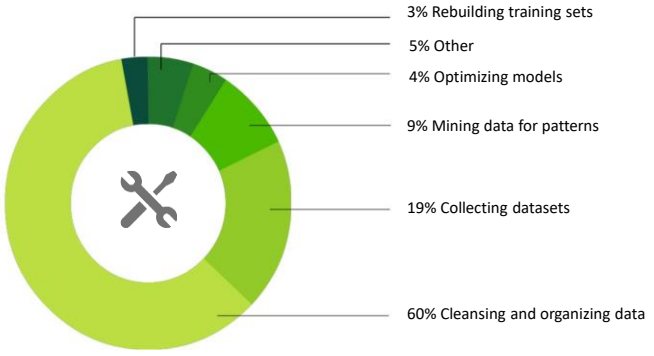
- Data is crucial to models and determines the scope of model capabilities. All good models require good data.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Cleansing Workloads

- What data scientists spend time doing for machine learning:



CrowdFlower Data Science Report 2016

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

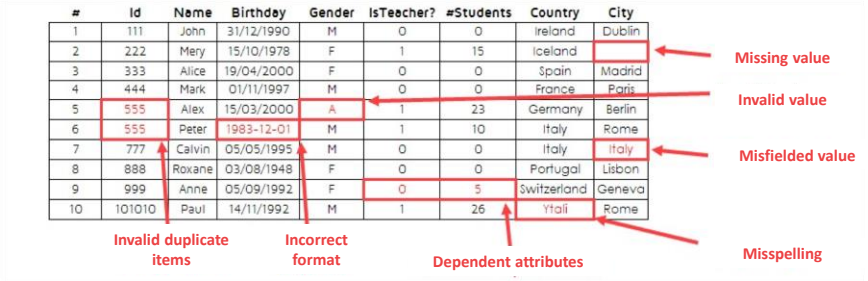
Data Cleansing

- Most machine learning models process features, which are usually numeric representations of input variables that can be used in the model.
- In most cases, only preprocessed data can be used by algorithms. Data preprocessing involves the following operations:
  - Data filtering
  - Data loss handling
  - Handling of possible error or abnormal values
  - Merging of data from multiple sources
  - Data consolidation

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Dirty Data

- Raw data usually contains data quality problems:
  - Incompleteness: Incomplete data or lack of relevant attributes or values.
  - Noise: Data contains incorrect records or abnormal points.
  - Inconsistency: Data contains conflicting records.



#	Id	Name	Birthday	Gender	IsTeacher?	#Students	Country	City
1	111	John	31/12/1990	M	0	0	Ireland	Dublin
2	222	Mery	15/10/1978	F	1	15	Iceland	
3	333	Alice	19/04/2000	F	0	0	Spain	Madrid
4	444	Mark	01/11/1997	M	0	0	France	Paris
5	555	Alex	15/03/2000	A	1	23	Germany	Berlin
6	555	Peter	1983-12-01	M	1	10	Italy	Rome
7	777	Calvin	05/05/1995	M	0	0	Italy	Italy
8	888	Roxane	03/08/1948	F	0	0	Portugal	Lisbon
9	999	Anne	05/09/1992	F	0	5	Switzerland	Geneva
10	101010	Paul	14/11/1992	M	1	26	Ytali	Rome

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

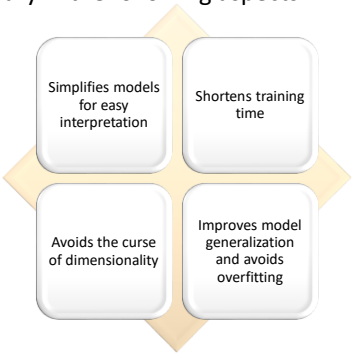
Data Conversion

- Preprocessed data needs to be converted into **a representation suitable for machine learning models**. The following are typically used to convert data:
  - Encoding categorical data into numerals for classification
  - Converting numeric data into categorical data to reduce the values of variables (for example, segmenting age data)
  - Other data:
    - Embedding words into text to convert them into word vectors (Typically, models such as word2vec and BERT are used.)
    - Image data processing, such as color space conversion, grayscale image conversion, geometric conversion, Haar-like features, and image enhancement
  - Feature engineering:
    - Normalizing and standardizing features to ensure that different input variables of a model fall into the same value range
    - Feature augmentation: combining or converting the existing variables to generate new features, such as averages.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Necessity of Feature Selection

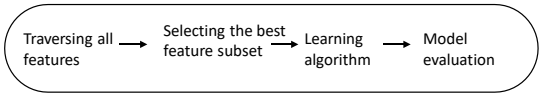
- Generally, a dataset has many features, some of which may be **unnecessary or irrelevant to the values to be predicted**.
- Feature selection is necessary in the following aspects:



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Feature Selection Methods - Filter

- Filter methods are independent of models during feature selection.



Filter method process

By evaluating the correlation between each feature and target attribute, a filter method scores each feature using a statistics measurement and then sorts the features by score. This can preserve or eliminate specific features.

Common methods:

- Pearson correlation coefficient
- Chi-square coefficient
- Mutual information

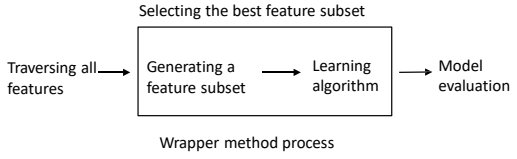
Limitations of filter methods:

- Filter methods tend to select redundant variables because they do not consider the relationships between features.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Feature Selection Methods - Wrapper

- Wrapper methods use a prediction model to score a feature subset.



Wrapper method process

Wrapper methods treat feature selection as a search issue and evaluate and compare different combinations. Wrapper methods use a predictive model to evaluate the different feature combinations, and score the feature subsets by model accuracy.

Common method:

- Recursive feature elimination

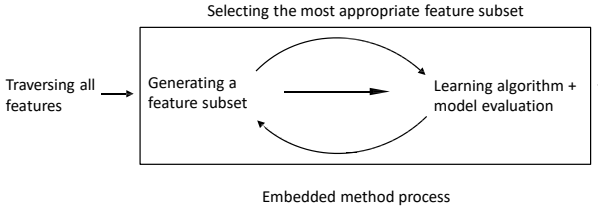
Limitations of wrapper methods:

- Wrapper methods train a new model for each feature subset, which can be **computationally intensive**.
- Wrapper methods usually provide high-performance feature sets for **a specific type of model**.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Feature Selection Methods - Embedded

- Embedded methods treat feature selection as a part of the modeling process.



Embedded method process

Regularization is the most common type of embedded methods.

Regularization methods, also called penalization methods, introduce additional constraints into the optimization of a predictive algorithm to bias the model toward lower complexity and reduce the number of features.

Common method:

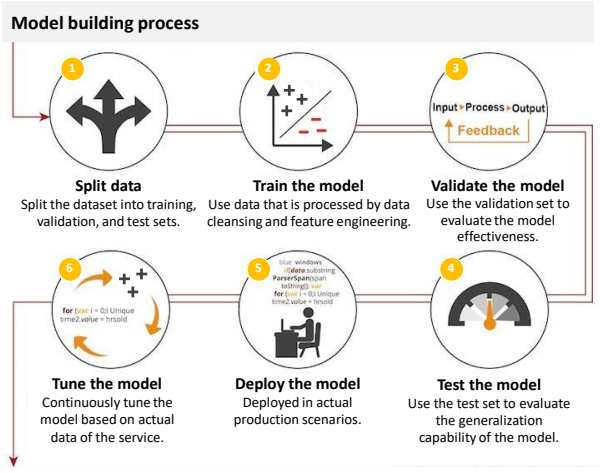
- LASSO regression

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The penalty attribute of the model is highlighted, which is used to eliminate insignificant features.

Overall Process of Building an AI Model



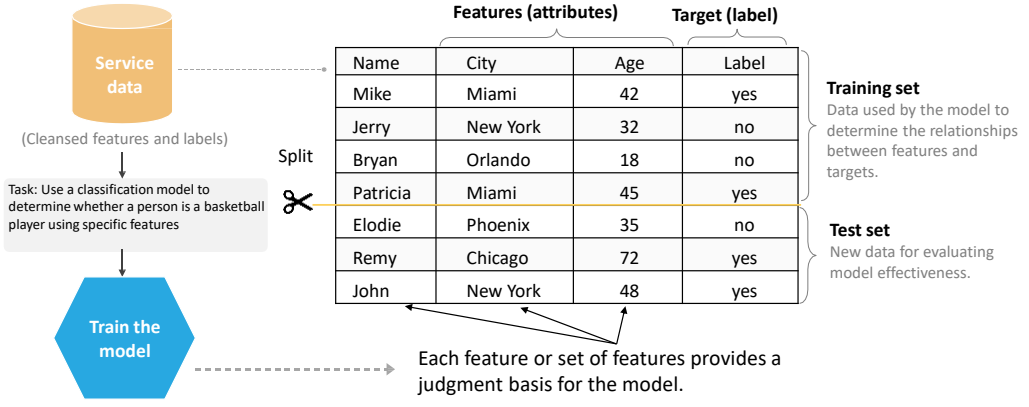
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- After data cleansing and feature extraction, we need to start building the model. The general procedure for building a model is shown above (supervised learning).

Supervised Learning Example - Learning Phase

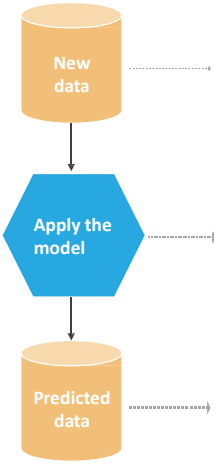
- Use a classification model to determine whether a person is a basketball player based on specific features.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Model example

Supervised Learning Example - Prediction Phase



Name	City	Age	Label
Marine	Miami	45	?
Julien	Miami	52	?
Fred	Orlando	20	?
Michelle	Boston	34	?
Nicolas	Phoenix	90	?

Unknown data
Recent data cannot determine whether they are a basketball player.

IF city = Miami → Probability = +0.7
IF city= Orlando → Probability = +0.2
IF age > 42 → Probability = +0.05*age + 0.06
IF age <= 42 → Probability = +0.01*age + 0.02

Name	City	Age	Prediction
Marine	Miami	45	**0.3**
Julien	Miami	52	**0.9**
Fred	Orlando	20	**0.6**
Michelle	Boston	34	**0.5**
Nicolas	Phoenix	90	**0.4**

Predicted possibility
Use the model against new data to predict the possibility of them being basketball players.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Model example
- The model here is similar to the decision tree model.

What Is a Good Model?



- **Generalization**
  The accuracy of predictions based on actual data
- **Explainability**
  Predicted results are easy to explain
- **Prediction speed**
  The time needed to make a prediction

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Which factors are used to judge a model?
- Generalization capability is the most important.

Model Effectiveness (1)

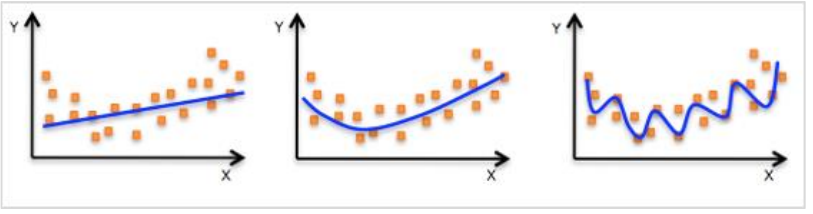
- **Generalization capability**: Machine learning aims to ensure models perform well on new samples, not just those used for training. Generalization capability, also called **robustness**, is the extent to which a learned model can be applied to new samples.
- **Error** is the difference between the prediction of a learned model on a sample and the actual result of the sample.
  - Training error is the error of the model on the training set.
  - Generalization error is the error of the model on new samples. Obviously, we prefer a model with a smaller generalization error.
- **Underfitting**: The training error is large.
- **Overfitting**: The training error of a trained model is small while the generalization error is large.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Once the form of a problem's hypothesis is given, all possible functions constitute a space, which is hypothesis space. The problem of machine learning is searching for a suitable fitting function in a hypothesis space.
- Overfitting: It occurs frequently in complex mathematical models. To prevent overfitting, we can simplify mathematical models, end training before overfitting, or use dropout/weight decay methods.
- Underfitting: It occurs if the mathematical model is too simple or the training time is too short. To solve the underfitting problem, use a more complex model or extend the training time.

Model Effectiveness (2)

- **Model capacity**, also known as model complexity, is the capability of the model to fit various functions.
  - With sufficient capacity to handle task complexity and training data volumes, the algorithm results are optimal.
  - Models with an insufficient capacity cannot handle complex tasks because **underfitting** may occur.
  - Models with a large capacity can handle complex tasks, but **overfitting** may occur when the capacity is greater than the amount required by a task.



Underfitting: features not learned | Good fitting | Overfitting: noises learned

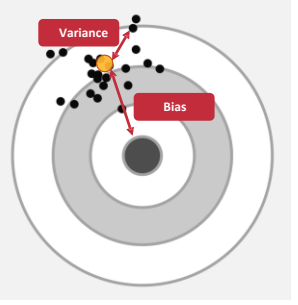
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The effective capacity is restricted by algorithms, parameters, and regularization.

Cause of Overfitting - Errors

- Prediction error = **Bias**^{2} + **Variance** + Ineliminable error
- In general, the two main factors of prediction error are variance an bias.
- Variance:
  - How much a prediction result deviates from the mean
  - Variance is caused by the sensitivity of the model to small fluctuations a training set.
- Bias:
  - Difference between the average of the predicted values and the actual values.

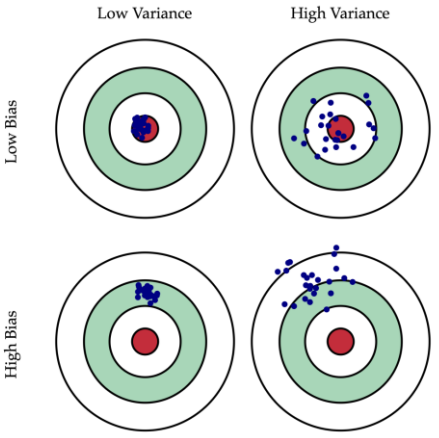


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- About the ineliminable error
  - Theoretically, if there is infinite amount of data and a perfect model, the error can be eliminated.
  - In practice, no model is perfect, and our data volume is limited.

Variance and Bias

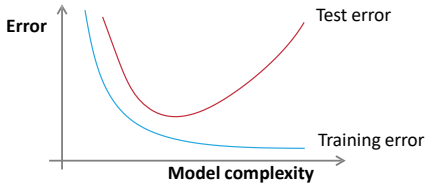
- Different combinations of variance and bias are as follows:
  - Low bias & low variance ➜ good model
  - Low bias & high variance ➜ inadequate model
  - High bias & low variance ➜ inadequate model
  - High bias & high variance ➜ bad model
- An ideal model can accurately capture the rules in the training data and be generalized to invisible (new) data. However, it is impossible for a model to complete both tasks at the same time.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Complexity and Errors of Models

- The more complex a model is, the smaller its training error is.
- As the model complexity increases, the test error decreases before increasing again, forming a convex curve.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Performance Evaluation of Machine Learning - Regression

- Mean absolute error (MAE). An MAE value closer to 0 indicates the model fits the training data better.

$$MAE = \frac{1}{m}\sum_{i=1}^{m}|y_i - \hat{y}_i|$$

- Mean squared error (MSE).

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(y_i - \hat{y}_i)^2$$

- The value range of R^2 is [0,1]. A larger value indicates that the model fits the training data better. TSS indicates the difference between samples, and RSS indicates the difference between the predicted values and sample values.

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^{m}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{m}(y_i - \bar{y}_i)^2}$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Performance Evaluation of Machine Learning - Classification (1)

- Terms:
  - P: positive, indicating the number of real positive cases in the data.
  - N: negative, indicating the number of real negative cases in the data.
  - TP : true positive, indicating the number of positive cases that are correctly classified.
  - TN: true negative, indicating the number of negative cases that are correctly classified.
  - FP: false positive, indicating the number of positive cases that are incorrectly classified.
  - FN: false negative, indicating the number of negative cases that are incorrectly classified.

Actual \ Predicted	Yes	No	Total
Yes	TP	FN	P
No	FP	TN	N
Total	P'	N'	$P+N$

Confusion matrix

- The confusion matrix is an $m \times m$ table at minimum. The entry $CM_{i,j}$ in the first m rows and m columns indicates the number of cases that belong to class i but are labeled as j.
  - For classifiers with high accuracy, most of the cases should be represented by entries on the diagonal of the confusion matrix from $CM_{1,1}$ to $CM_{m,m}$,while other entries are 0 or close to 0. That is, FP and FN are close to 0.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Performance Evaluation of Machine Learning - Classification (2)

Measurement	Formula
Accuracy, recognition rate	$\frac{TP + TN}{P + N}$
Error rate, misclassification rate	$\frac{FP + FN}{P + N}$
True positive rate, sensitivity, recall	$\frac{TP}{P}$
True negative rate, specificity	$\frac{TN}{N}$
Precision	$\frac{TP}{TP + FP}$
F_1 value, harmonic mean of precision and recall	$\frac{2 \times precision \times recall}{precision + recall}$
F_β value, where β is a non-negative real number	$\frac{(1 + \beta^2) \times precision \times recall}{\beta^2 \times precision + recall}$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Performance Evaluation of Machine Learning - Example

- In this example, an ML model was trained to identify an image of a cat. To evaluate the model's performance, 200 images were used, of which 170 of them were cats.
- The model reported that 160 images were cats.

Precision: $P = \frac{TP}{TP+FP} = \frac{140}{140+20} = 87.5\%$

Recall: $R = \frac{TP}{P} = \frac{140}{170} = 82.4\%$

Accuracy: $ACC = \frac{TP+TN}{P+N} = \frac{140+10}{170+30} = 75\%$

Predicted / Actual	yes	no	Total
yes	140	30	170
no	20	10	30
Total	160	40	200

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Machine Learning Algorithms
2. Types of Machine Learning
3. Machine Learning Process
4. **Important Machine Learning Concepts**
5. Common Machine Learning Algorithms

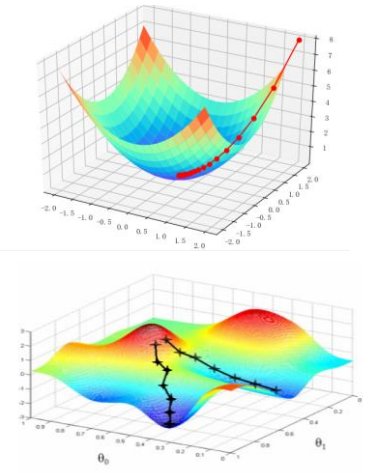
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Machine Learning Training Methods - Gradient Descent (1)

- This method uses the negative gradient direction of the current position as the search direction, which is the fastest descent direction of the current position. The formula is as follows:

$$w_{k+1,i} = w_k - \eta \nabla f_{w_k}(x^i)$$

- η is the learning rate. i indicates the i-th data record. $\eta \nabla f_{w_k}(x^i)$ indicates the change of weight parameter w in each iteration.
- Convergence means that the value of the objective function changes very little or reaches the maximum number of iterations.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Typically, the gradient descent method shows a decreasing descent in a very small range close to the target point, but the gradient may fluctuate in a specific range of the target point.

Machine Learning Training Methods - Gradient Descent (2)

- Batch gradient descent (BGD) uses the sum of gradients of all m samples of the dataset at the current point to update the weight parameter.

$$w_{k+1} = w_k - \eta \frac{1}{m} \sum_{i=1}^{m} \nabla f_{w_k}(x^i)$$

- Stochastic gradient descent (SGD) randomly uses the gradient of a random sample of the dataset at the current point to update the weight parameter in the current gradient.

$$w_{k+1} = w_k - \eta \nabla f_{w_k}(x^i)$$

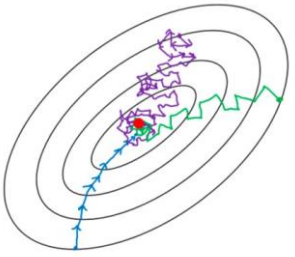
- Mini-batch gradient descent (MBGD) combines the features of BGD and SGD, and chooses the gradients of n samples in a dataset each time to update the weight parameter.

$$w_{k+1} = w_k - \eta \frac{1}{n} \sum_{i=\mathrm{t}}^{t+n-1} \nabla f_{w_k}(x^i)$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Machine Learning Training Methods - Gradient Descent (3)

- Comparison of gradient descent methods
  - SGD randomly chooses samples for each training pass, causing instability. As a result, the loss function fluctuates or even produces reverse displacement during the process of dropping to the minimum.
  - BGD is the most stable, but it consumes too many compute resources. MBGD is a balance between BGD and SGD.



BGD
Use **all** training samples for each training pass.

SGD
One training sample is used for each training pass.

MBGD
A certain number of training samples are used for each training pass.

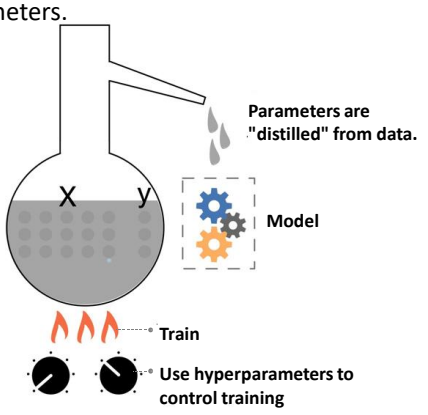
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Reverse displacement is caused by noise data from datasets.
- Some consider BGD and MBSGD as types of SGD.

Parameters and Hyperparameters

- A model contains not only parameters but also hyperparameters. Hyperparameters enable the model to learn the optimal configurations of the parameters.
  - Parameters are automatically learned by models.
  - Hyperparameters are manually set.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Hyperparameters

- Commonly used for model parameter estimation.
- Specified by the user.
- Set heuristically.
- Often tuned for a given predictive modeling problem.

Hyperparameters are configurations outside the model.

- λ of Lasso/Ridge regression
- Learning rate, number of iterations, batch size, activation function, and number of neurons of a neural network to be trained
- C and σ of support vector machines (SVMs)
- k in the k-nearest neighbors (k-NN) algorithm
- Number of trees in a random forest

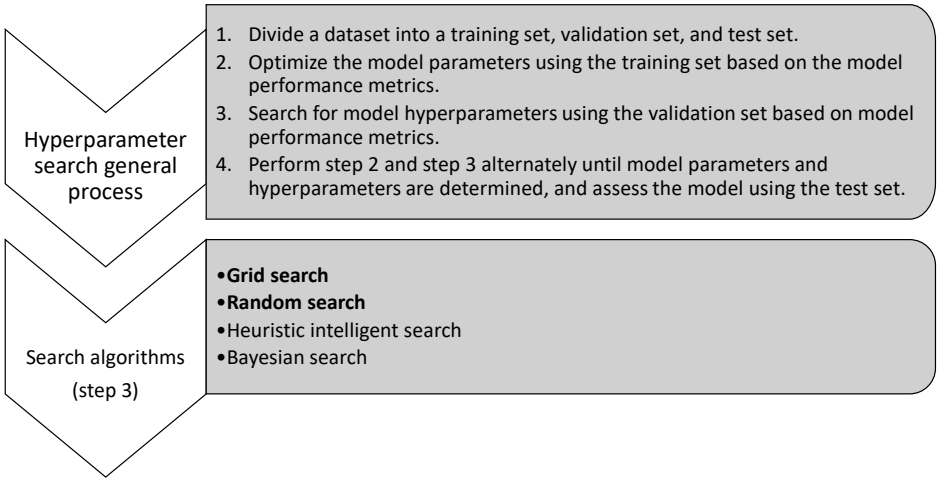
Common hyperparameters

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Parameters are a part of a model that is learned from historical training data and key to machine learning algorithms. They have the following characteristics:
  - They are required by a model to make predictions.
  - Their values define the model functions.
  - They are learned or estimated from data.
  - They are often not set manually by the user.
  - They are often saved as a part of the learned model.
- Examples:
  - Weights in an artificial neural network
  - Support vectors in a SVM
  - Coefficients in linear regression or logistic regression

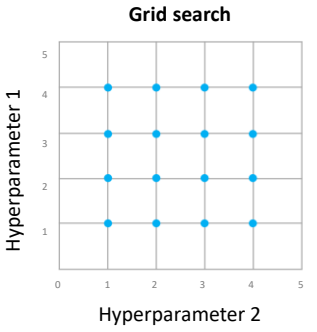
Hyperparameter Search Process and Methods



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Hyperparameter Tuning Methods - Grid Search

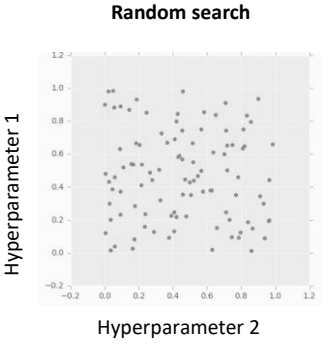
- Grid search performs an **exhaustive search** of all possible hyperparameter combinations to form a hyperparameter value grid.
- In practice, the hyperparameter ranges and steps are manually specified.
- Grid search is expensive and time-consuming.
  - This method works well when there are relatively few hyperparameters. Therefore, it is feasible for general machine learning algorithms, but not for neural networks (see the deep learning course).



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Hyperparameter Tuning Methods - Random Search

- If the hyperparameter search space is large, **random search** is more appropriate than grid search.
- In a random search, each setting item is sampled from possible parameter values to find the most appropriate parameter subset.
- Note:
  - In a random search, a search is first performed within a broad range, and then the range is narrowed based on the location of the best result.
  - Some hyperparameters are more important than others and affect random search preferences.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

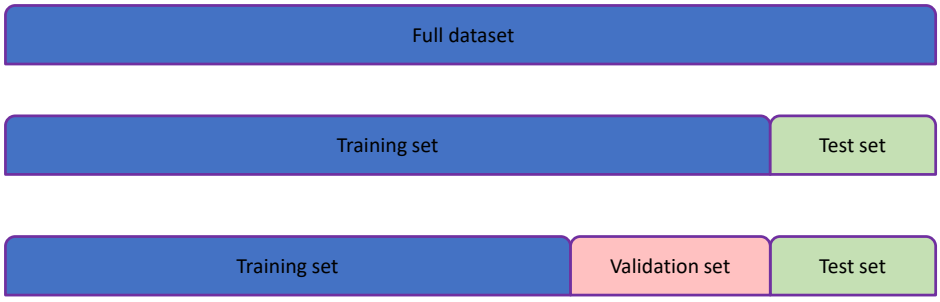
Cross-Validation (1)

- **Cross-validation** is a statistical analysis method used to check the performance of classifiers. It splits the original data into the training set and validation set. The former is used to train a classifier, whereas the latter is used to evaluate the classifier by testing the trained model.
- ***k*-fold cross-validation (*k*-fold CV):**
  - Divides the original data into k (usually equal-sized) subsets.
  - Each unique group is treated as a validation set, and the remaining $k-1$ groups are treated as the training set. In this way, k models are obtained.
  - The average classification accuracy score of the k models on the validation set is used as the performance metric for *k*-fold CV classifiers.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- After a dataset is divided into a fixed training set and a fixed test set, if the test set error is small, there is a problem. A small test set implies statistical uncertainty around the estimated mean test error, making it difficult to claim that A algorithm works better than B algorithm on the given task. When the dataset has hundreds of thousands of samples or more, this is not a serious problem. When the dataset is too small, alternative procedures enable one to use all the samples in the estimation of the mean test error, at the price of increased computational workload.
- In k-fold CV, k is typically greater than or equal to 2. It usually starts from 3 and is set to 2 only when the original dataset is small. k-fold CV can effectively avoid over-learning and under-learning, and the final result is also persuasive.

Cross-Validation (2)



- Note: *k* in *k*-fold CV is a hyperparameter.

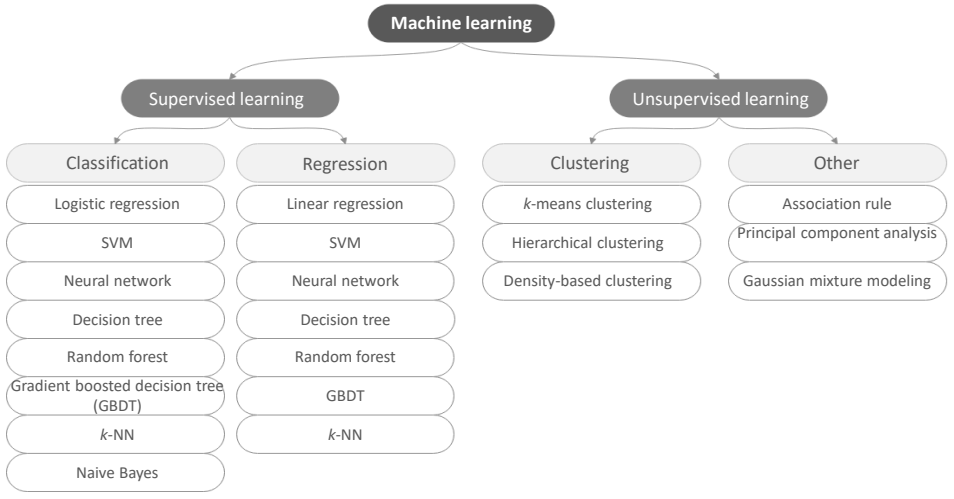
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Machine Learning Algorithms
2. Types of Machine Learning
3. Machine Learning Process
4. Important Machine Learning Concepts
5. **Common Machine Learning Algorithms**

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

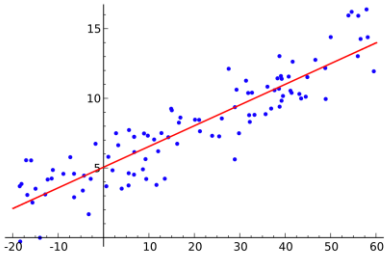
Machine Learning Algorithm Overview



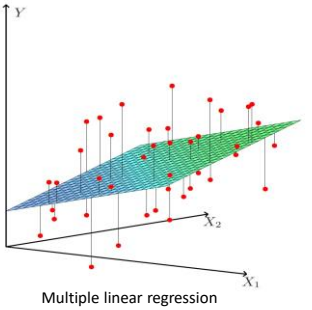
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Linear Regression (1)

- Linear regression uses the regression analysis of mathematical statistics to determine the quantitative relationship between two or more variables.
- Linear regression is a type of supervised learning.



Simple linear regression



Multiple linear regression

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- xSimple linear regression analysis refers to regression analysis where only one independent variable and one dependent variable exist, and their relationship can be approximately represented by a straight line. Multiple linear regression analysis involves two or more independent variables and the relationship between independent variables and dependent variables is linear. The linear regression model is a straight line only when the variable x is one-dimensional. It is a hyperplane when this variable is multi-dimensional. For example, the price of an apartment is determined by a variety of factors such as the area, layout, and location. Prediction of the apartment price based on these factors can be abstracted into a linear regression problem.
- The above figures show polynomial linear regression.

Linear Regression (2)

- The model function of linear regression is as follows, where w is the weight parameter, b is the bias, and x represents the sample:

$$h_w(x) = w^T x + b$$

- The relationship between the value predicted by the model and the actual value is as follows, where y indicates the actual value, and ε indicates the error:

$$y = w^T x + b + \varepsilon$$

- The error ε is affected by many independent factors. Linear regression assumes that the error ε follows normal distribution. The loss function of linear regression can be obtained using the normal distribution function and maximum likelihood estimation (MLE):

$$J(w) = \frac{1}{2m}\sum\left(h_w(x) - y\right)^2$$

- We want the predicted value approaches the actual value as far as possible, that is, to minimize the loss value. We can use a gradient descent algorithm to calculate the weight parameter w when the loss function reaches the minimum, thereby complete model building.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

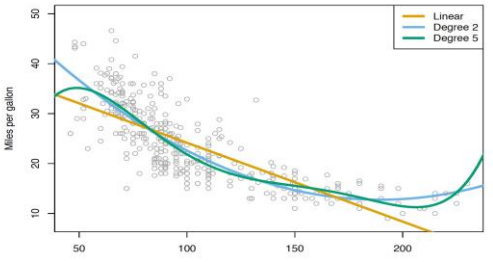
Linear Regression Extension - Polynomial Regression

- Polynomial regression is an extension of linear regression. Because the complexity of a dataset exceeds the possibility of fitting performed using a straight line (obvious underfitting occurs if the original linear regression model is used), polynomial regression is used.

$$h_w(x) = w_1x + w_2x^2 + \cdots + w_nx^n + b$$

Here, n-th power indicates the degree of the polynomial.

Polynomial regression is a type of linear regression. Although its features are non-linear, the relationship between its weight parameters w is still linear.



Comparison between linear and polynomial regression

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Preventing Overfitting of Linear Regression

- Regularization terms help reduce overfitting. The w value cannot be too large or too small in the sample space. You can add a square sum loss to the target function:

$$J(w) = \frac{1}{2m}\sum(h_w(x) - y)^2 + \lambda\|w\|_2^2$$

- Regularization term: This regularization term is called L2-norm. Linear regression that uses this loss function is called **Ridge regression**.

$$J(w) = \frac{1}{2m}\sum(h_w(x) - y)^2 + \lambda\|w\|_1$$

- Linear regression with an absolute loss is called **Lasso regression**.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

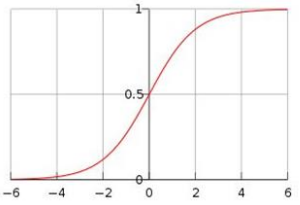
Logistic Regression (1)

- The logistic regression model is a classification model used to resolve classification problems. The model is defined as follows:

$$P(Y=0|x)=\frac{e^{-(wx+b)}}{1+e^{-(wx+b)}}$$

$$P(Y=1|x)=\frac{1}{1+e^{-(wx+b)}}$$

w represents the weight, b represents the bias, and $wx+b$ represents a linear function with respect to x. Compare the preceding two probability values. x belongs to the type with a larger probability value.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Logistic Regression (2)

- Logistic regression and linear regression are both linear models in broad sense. The former introduces a non-linear factor (sigmoid function) on the basis of the latter and sets a threshold. Therefore, logistic regression applies to binary classification.
- According to the model function of logistic regression, the loss function of logistic regression can be calculated through maximum likelihood estimation as follows:

$$J(w) = -\frac{1}{m}\sum\left(y \ln h_w(x) + (1-y)\ln(1-h_w(x))\right)$$

- In the formula, w indicates the weight parameter, m indicates the number of samples, x indicates the sample, and y indicates the actual value. You can also obtain the values of all the weight parameters w by using a gradient descent algorithm.

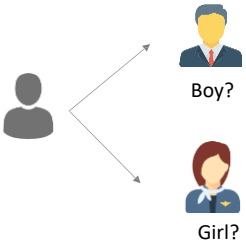
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Overfitting of logistic regression can also be prevented by adding regularization terms.

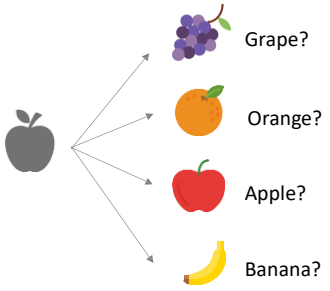
Logistic Regression Extension - Softmax (1)

- Logistic regression mainly applies to binary classification. For multi-class classification, the softmax function is typically used.

Binary classification problem



Multi-class classification problem



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Logistic Regression Extension - Softmax (2)

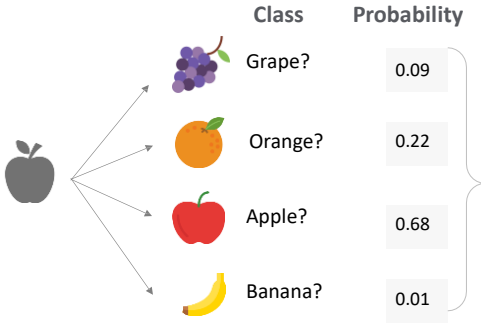
- Softmax regression is a generalization of logistic regression and applies to *k*-class classification.
- The softmax function compresses (maps) a *k*-dimensional vector of arbitrary real values to another *k*-dimensional vector of real values, where each vector element is in (0, 1).
- The Softmax regression probability function is:

$$p(y=k \mid x; w) = \frac{e^{w_k^T x}}{\sum_{l=1}^{K} e^{w_l^T x}}, k = 1, 2 \cdots, K$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Logistic Regression Extension - Softmax (3)

- Softmax assigns a probability value to each class in a multi-class classification problem. The sum of all the probabilities is 1.
  - Softmax may present the generated classes as follows:

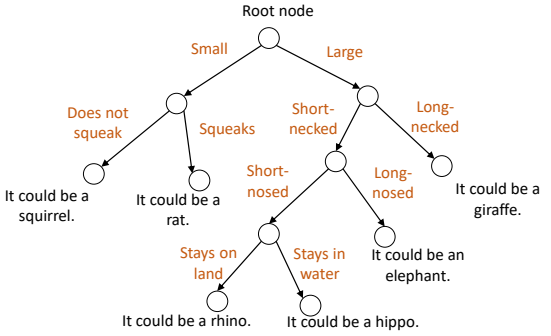


- **Sum of all probabilities:**
  - 0.09 + 0.22 + 0.68 + 0.01 =1
- The most possible object in the image: Apple

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Decision Tree

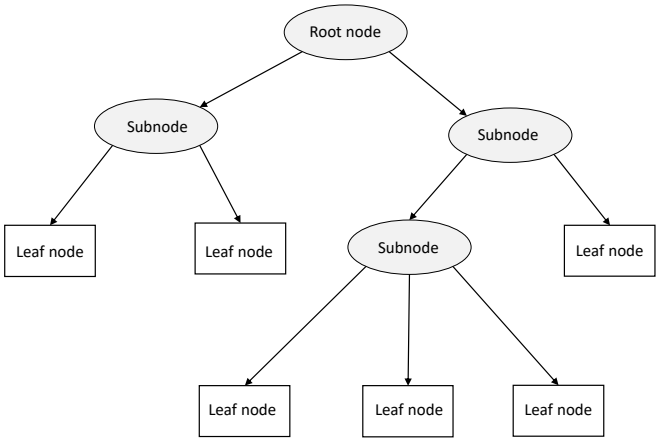
- Each non-leaf node of the decision tree denotes a test on an attribute; each branch represents the output of a test; and each leaf (or terminal) node holds a class label. The algorithm starts at the root node (topmost node in the tree), tests the selected attributes on the intermediate (internal) nodes, and generates branches according to the output of the tests. Then, it saves the class labels on the leaf nodes as the decision results.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- How to construct a decision tree is very important. We should determine the topological structure of feature attributes based on attribute selection measures. The key step is to split attributes. That is, different branches are constructed based on the differences of a feature attribute on a node.
- The common learning algorithms used to generate a decision tree include ID3, C4.5, and CART.

Structure of a Decision Tree



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- All nodes except the root node are called leaf nodes.

Key to Decision Tree Construction

- A decision tree requires feature attributes and an appropriate tree structure. The key step of constructing a decision tree is to divide data of all feature attributes, compare the result sets in terms of purity, and select the attribute with the highest purity as the data point for dataset division.
- Purity is measured mainly through the information entropy and GINI coefficient. The formula is as follows:

$$H(X)= -\sum_{k=1}^{K} p_k \log_2(p_k) \qquad Gini = 1 - \sum_{k=1}^{K} p_k^2$$

$$min_{j,s}[min_{c1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + min_{c2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2]$$

- p_k indicates the probability that a sample belongs to category k (in a total of *K* categories). A larger purity difference between the sample before and after division indicates a better decision tree.
- Common decision tree algorithms include ID3, C4.5, and CART.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Decision Tree Construction Process

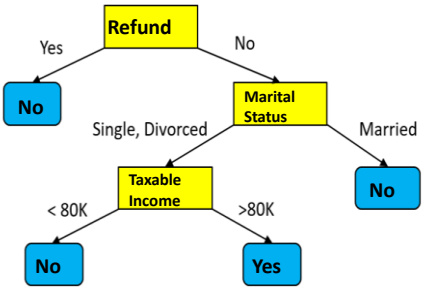
- **Feature selection**: Select one of the features of the training data as the split standard of the current node. (Different standards distinguish different decision tree algorithms.)
- **Decision tree generation**: Generate subnodes from top down based on the selected feature and stop until the dataset can no longer be split.
- **Pruning**: The decision tree may easily become overfitting unless necessary pruning (including pre-pruning and post-pruning) is performed to reduce the tree size and optimize its node structure.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Decision Tree Example

- The following figure shows a decision tree for a classification problem. The classification result is affected by three attributes: refund, marital status, and taxable income.

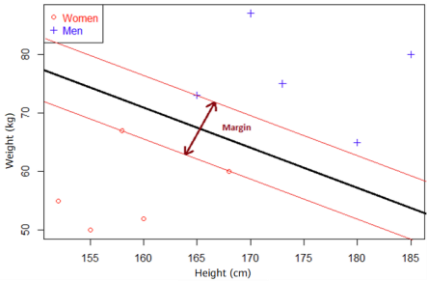
TID	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

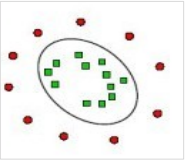


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Support Vector Machine

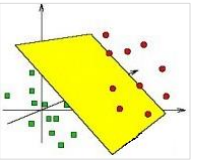
- Support vector machines (SVMs) are binary classification models. Their basic model is the linear classifier that maximizes the width of the gap between the two categories in the feature space. SVMs also have a kernel trick, which makes it a non-linear classifier. The learning algorithm of SVMs is the optimal algorithm for convex quadratic programming.





Difficult to split in a low-dimensional space.

Mapping



Easy to split in a high-dimensional space.

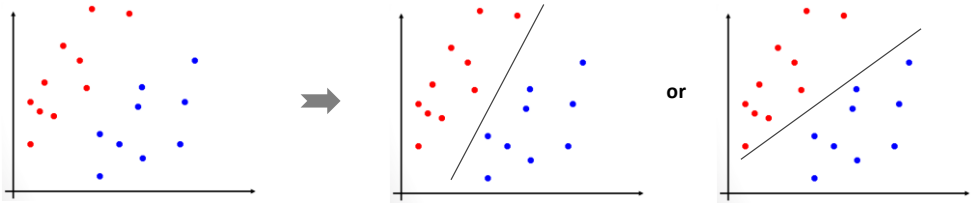
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The main ideas of SVMs are:
  - In the case of linear inseparability, non-linear mapping algorithms are used to convert the linearly inseparable samples of low-dimensional input space into samples of high-dimensional feature space. In this way, samples become linearly separable. Then the linear algorithm can be used to analyze the non-linear features of samples.
  - Based on the structural risk minimization principle, an optimal hyperplane is constructed in the feature space, so that the learner is optimized globally, and the expectation of the whole sample space satisfies an upper boundary with a certain probability.

Linear SVM (1)

- How can we divide the red and blue data points with just one line?



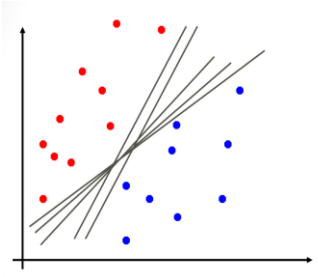
Two-dimensional data set with two sample categories

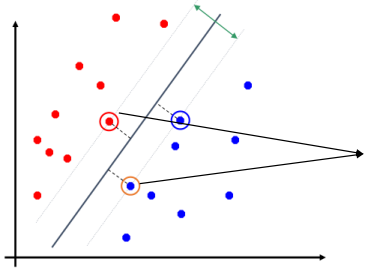
Both the division methods on the left and right can divide data. But which is correct?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Linear SVM (2)

- We can use different straight lines to divide data into different categories. SVMs find a straight line and keep the most nearby points as **far** from the line as possible. This gives the model a strong generalization capability. These most nearby points are called **support vectors**.
- In the two-dimensional space, a straight line is used for division; in the high-dimensional space, a **hyperplane** is used for division.

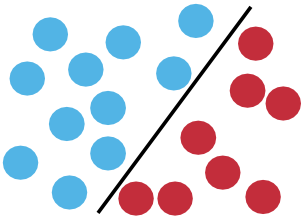




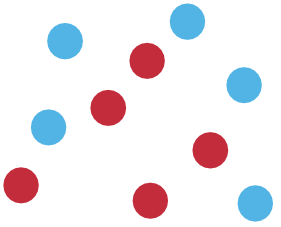
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Non-linear SVM (1)

- How can we divide a linear inseparable data set?



Linear SVM works well on a linear separable data set.



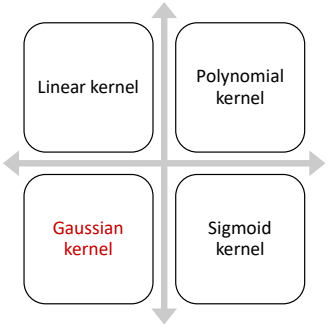
A non-linear data set cannot be divided using a straight line.

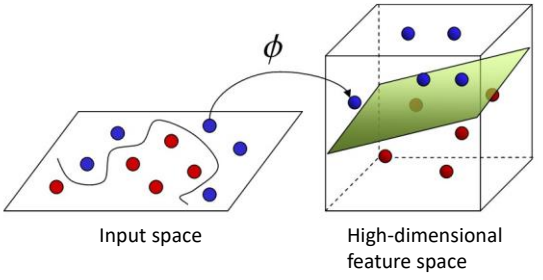
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Non-linear SVM (2)

- Kernel functions can be used to create non-linear SVMs.
- Kernel functions allow algorithms to fit a maximum-margin hyperplane in a transformed high-dimensional feature space.

Common kernel functions





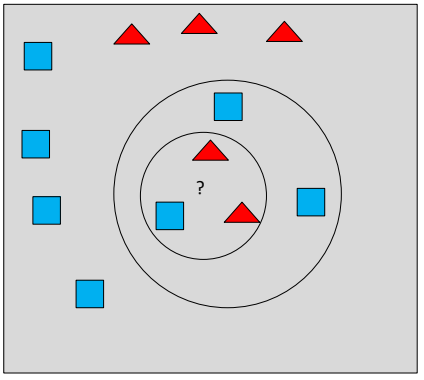
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The Gaussian kernel function is the most frequently used.

k-Nearest Neighbors Algorithm (1)

- The *k*-nearest neighbor (*k*-NN) classification algorithm is a theoretically mature method and one of the simplest machine learning algorithms. The idea of *k*-NN classification is that, if most of *k* closest samples (nearest neighbors) of a sample in the feature space belong to a category, the sample also belongs to this category.

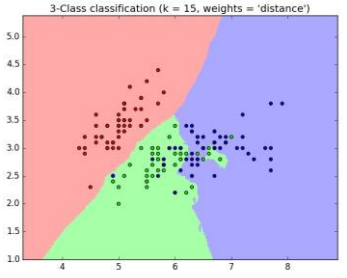


The category of point **?** varies according to how many neighbor nodes are chosen.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

k-Nearest Neighbors Algorithm (2)

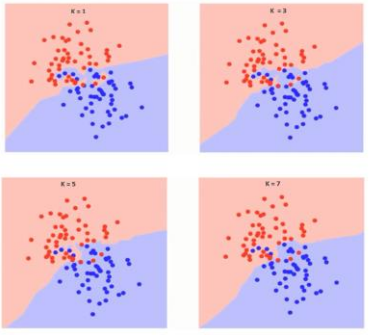
- The logic of *k*-NN is simple: If an object's *k* nearest neighbors belong to a class, so does the object.
- *k*-NN is a non-parametric method and is often used for datasets with irregular decision boundaries.
  - *k*-NN typically uses the **majority voting** method to predict classification, and uses the **mean value** method to predict regression.
- *k*-NN requires a very large amount of computing.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

k-Nearest Neighbors Algorithm (3)

- Typically, a larger *k* value reduces the impact of noise on classification, but makes the boundary between classes less obvious.
  - A large *k* value indicates a higher probability of underfitting because the division is too rough; while a small *k* value indicates a higher probability of overfitting because the division is too refined.



- As seen from the figure, the boundary becomes smoother as the *k* value increases.
- As the *k* value increases, the points will eventually become all blue or all red.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Naive Bayes (1)

- **Naive Bayes** classifiers are a family of simple "probabilistic classifiers" based on **Bayes' theorem** with **strong independence assumptions between the features**. For a given sample feature X, the probability that the sample belongs to category H is:

$$P(C_k \mid X_1,\ldots,X_n) = \frac{P(X_1,\ldots,X_n \mid C_k)P(C_k)}{P(X_1,\ldots,X_n)}$$

  - $X_1, X_2, \ldots, X_n$ are data features, which are usually described by *m* measurement values of the attribute set.
    - For example, the attribute of the color feature may be red, yellow, and blue.
  - C_k indicates that the data belongs to a specific class C.
  - $P(C_k|X_1, X_2, \ldots, X_n)$ is the posterior probability, or the posterior probability of H under condition C_k.
  - $\mathrm{P}(C_k)$ is the prior probability independent of $X_1, X_2, \ldots, X_n$.
  - $P(X_1, X_2, \ldots, X_n)$ is the prior probability of X.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Class conditional independence: The Bayes classifier assumes that the effect of an attribute value on a given class is independent of the values of other attributes. This assumption is made to simplify the calculation and becomes "naive" in this sense.
- Bayes classifiers show high accuracy and speed when applied to large databases.

Naive Bayes (2)

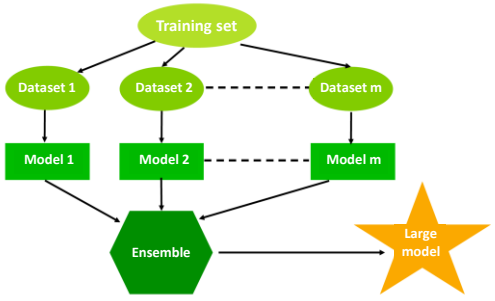
- Feature independent hypothesis example:
  - If a fruit is red, round, and about 10 cm in diameter, it can be considered an apple.
  - A Naive Bayes classifier believes that each of these features independently contributes to the probability of the fruit being an apple, regardless of any possible correlation between color, roundness, and diameter features.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

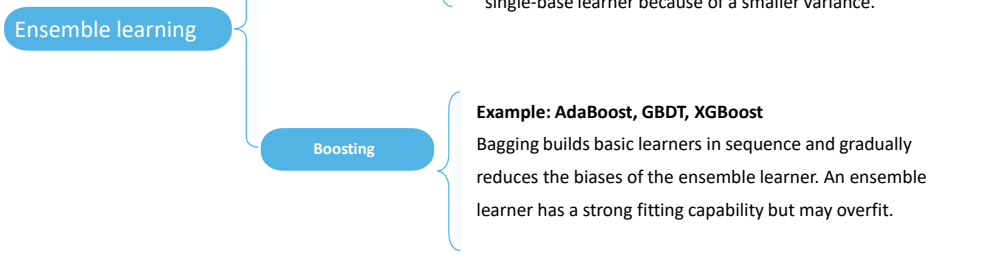
Ensemble Learning

- Ensemble learning is a machine learning paradigm in which multiple learners are trained and combined to resolve an issue. When multiple learners are used, the generalization capability of the ensemble can be much stronger than that of a single learner.
- For example, If you ask thousands of people at random a complex question and then summarize their answers, the summarized answer is more accurate than an expert's answer in most cases. This is the **wisdom of the crowd**.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Types of Ensemble Learning



Example: random forest

- Bagging independently builds multiple basic learners and then averages their predictions.
- On average, an ensemble learner is usually better than a single-base learner because of a smaller variance.

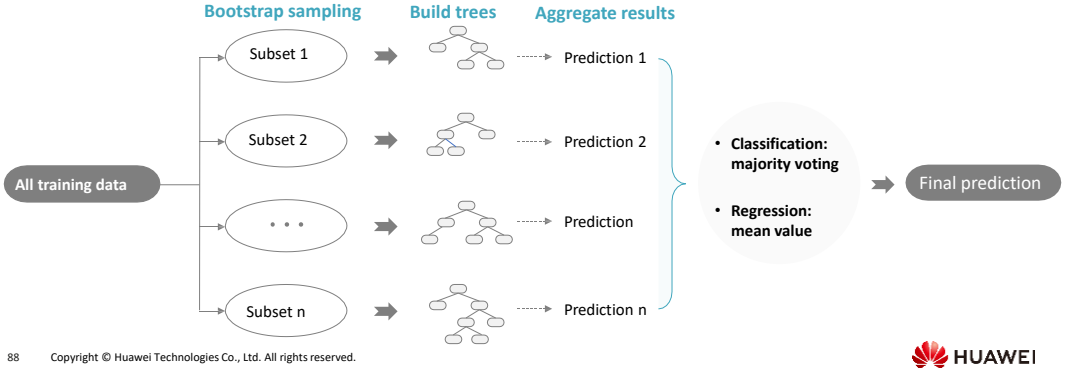
Example: AdaBoost, GBDT, XGBoost

Bagging builds basic learners in sequence and gradually reduces the biases of the ensemble learner. An ensemble learner has a strong fitting capability but may overfit.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Ensemble Learning - Random Forest

- Random forest = Bagging + Classification and regression tree (CART)
- Random forest builds multiple decision trees and aggregates their results to make prediction more accurate and stable.
  - The random forest algorithm can be used for classification and regression problems.

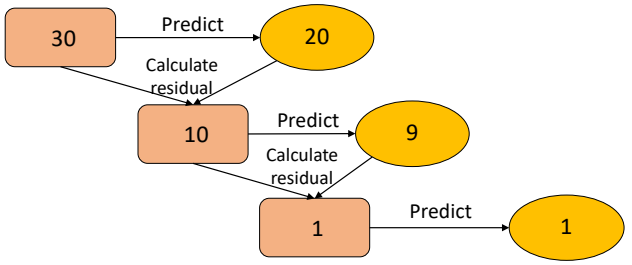


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Bootstrap bagging is based on sampling with replacement.

Ensemble learning - Gradient Boosted Decision Tree

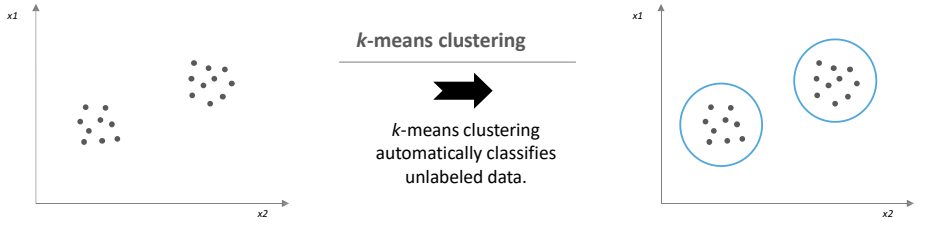
- Gradient boosted decision tree (GBDT) is a type of boosting algorithm.
- The prediction result of the ensemble model is the sum of results of all base learners. The essence of GBDT is that the next base learner tries to fit the residual of the error function to the prediction value, that is, the residual is the error between the prediction value and the actual value.
- During GBDT model training, the loss function value of the sample predicted by the model must be as small as possible.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Unsupervised Learning - *k*-Means Clustering

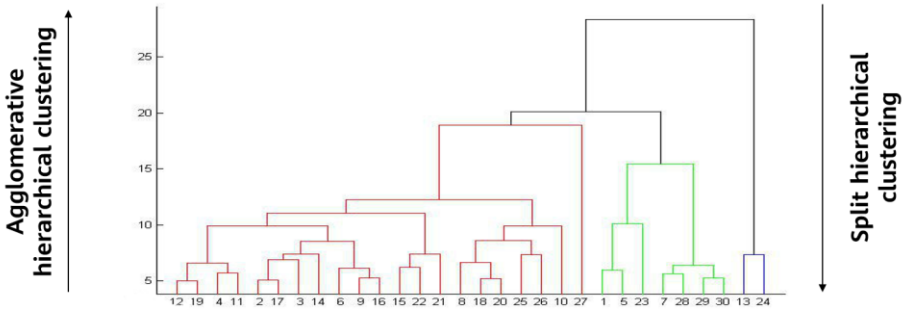
- *k*-means clustering takes the number of clusters *k* and a dataset of *n* objects as inputs, and outputs *k* clusters with minimized within-cluster variances.
- In the *k*-means algorithm, the number of clusters is *k*, and *n* data objects are split into *k* clusters. The obtained clusters meet the following requirements: high similarity between objects in the same cluster, and low similarity between objects in different clusters.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Unsupervised Learning - Hierarchical Clustering

- Hierarchical clustering divides a dataset at different layers and forms a tree-like clustering structure. The dataset division may use a "bottom-up" aggregation policy, or a "top-down" splitting policy. The hierarchy of clustering is represented in a tree diagram. The root is the only cluster of all samples, and the leaves are clusters of single samples.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Summary

- This course first describes the definition and types of machine learning, as well as problems machine learning solves. Then, it introduces key knowledge points of machine learning, including the overall procedure (data preparation, data cleansing, feature selection, model evaluation, and model deployment), common algorithms (including linear regression, logistic regression, decision tree, SVM, Naive Bayes, *k*-NN, ensemble learning, and k-means clustering), and hyperparameters.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Single-answer) Which of the following is not a supervised learning algorithm? ( )

   A. Linear regression

   B. Decision tree

   C. *k*-NN

   D. *k*-means clustering

2. (True or false) Gradient descent is the only method of machine learning. ( )

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Answers: 1. D 2. False

Recommendations

- Huawei Talent
  - https://e.huawei.com/en/talent/portal/#/
- Huawei knowledge base
  - https://support.huawei.com/enterprise/en/knowledge?lang=en

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Basics of Deep Learning and Foundation Models



HUAWEI

Foreword

- In today's digital era, explosive data growth and significant improvements in computing power have enabled unprecedented development opportunities in the AI industry. Deep learning is an important branch of this industry.
- As research develops and technologies progress, a shallow deep learning architecture has been replaced with a deeper one, giving rise to foundation models. Foundation models are extremely large neural networks with a more complex structure. Due to their excellent expressiveness, foundation models have outperformed most other methods of image generation, speech recognition, and text generation.
- This chapter explains the basics of neural networks and foundation models.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completing this course, you will be able to understand:
  - Deep learning basics
  - The basic architecture of convolutional neural networks
  - The basic architecture of recurrent neural networks
  - The transformer architecture
  - The architectures of common foundation models

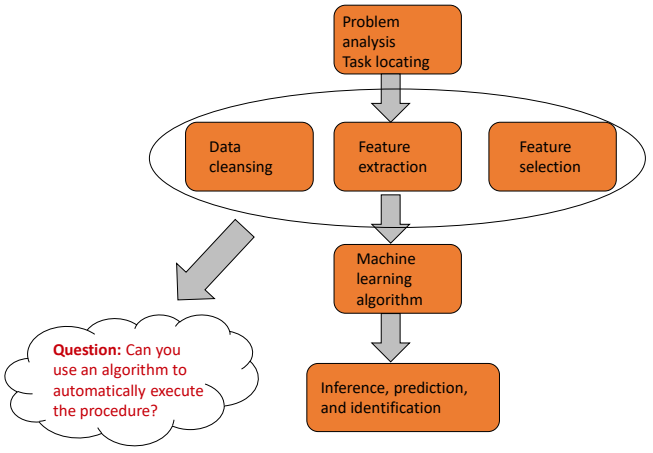
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **Perceptron**
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Traditional Machine Learning



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

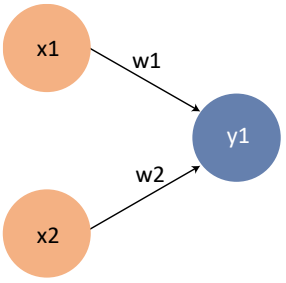
HUAWEI

- Problem analysis and task locating: Locate the problem, convert the problem into a machine learning problem, determine required data, and then collect the required data. This part includes data exploration, scenario abstraction, and algorithm selection. For example, logistic regression can be used to determine whether we can drink coffee, and the Naive Bayes algorithm can be used to distinguish junk mails from normal ones. AI systems do not directly detect but gather data from doctors, such as human body's tolerance to caffeine and whether it causes sleeplessness.
- Data cleansing: Data cleansing is the process of rechecking and validating data to remove duplicate information, correct errors, and ensure data consistency. Adoption (random, systematic, and layered), normalization, noise removal, and data filtering
- Feature extraction: feature abstraction, feature importance evaluation, feature derivation, and feature dimension reduction
- Feature selection
- Model training
- Inference, prediction, and identification
- Generally, features are manually selected in machine learning. More features provide more information and higher identification accuracy.
- However, more features also increase the computation complexity and search space. Training data appears to be sparse in the overall feature vectors,

affecting similarity determination, that is, **dimension exploration**.

What Is a Perceptron?

- A perceptron receives multiple input signals and outputs one signal.



- The figure on the left shows an example of a perceptron that receives two input signals. x1 and x2 are the input signals, y is the output signal, and w1 and w2 are the weights.
- The circles in the figure are neurons or nodes. When the input signals are sent to the neuron, they are multiplied by a fixed weight (w1 x 1 and w2 x 2). The neuron calculates the sum of the input signals and outputs 1 only when the sum exceeds a certain threshold. This is also called neuron activation.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- In 1957, Frank Rosenblatt at the aviation lab of the Cornell University invented the perceptron artificial neural network. Considered a simplest form of a feedforward neural network model, the network is a binary linear classifier. Its activation function is a sign function sign (x). The perceptron is the first practical application of the artificial neural network, marking a new stage of neural network development. After implementing the perceptron with software, Frank Rosenblatt started to build the hardware perceptron MarkI. He used 400 optoelectronic devices as neurons and adjustable potentiometers as synaptic weights. Motors are used to adjust potentiometers to implement weight changes during learning. He built the hardware perceptron in this way and used it for image recognition. Due to technical limitations at that time, perceptrons based on physical implementation are rare. This image recognition system drew attention from many parties and received a large amount of fund from the U.S. Navy.
- Signals mentioned here can be seen as something that has mobility like an electric current or a river. Similar to an electric current in a wire sending electrons forward, perceptron signals form a flow and transmit information forward. The difference from an electric current is that a perceptron signal has only two values, current (1) and no current (0). In this document, 0 indicates that no signal is transmitted, and 1 indicates that signals are transmitted.

Mathematical Representation of a Perceptron

- Input signals of the perceptron have their own inherent weights, which determine the importance of each signal. A larger weight indicates that the signal is more important.

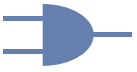
$$y = \begin{cases} -1 & (w_1x_1 + w_2x_2 \leqslant \theta) \\ 1 & (w_1x_1 + w_2x_2 > \theta) \end{cases}$$

A weight is similar to the resistance in an electric circuit. Resistance determines how difficult it is for current to flow. The lower the resistance, the larger the current flowing through. When it comes to the perceptron, a larger weight indicates a larger signal that passes through the perceptron. Resistance and weight play the same role in controlling the signal flow difficulty (or ease).

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Using a Perceptron to Simulate a Simple Logic Circuit

- Forms of logic circuits



AND gate

x_1	x_2	y
0	0	0
1	0	0
0	1	0
1	1	1

(w1, w2, θ) = (0.5, 0.5, 0.7)



NAND gate

x_1	x_2	y
0	0	1
1	0	1
0	1	1
1	1	0

(w1, w2, θ) = (−0.5, −0.5, −0.7)



OR gate

x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	1

(w1, w2, θ) = ?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

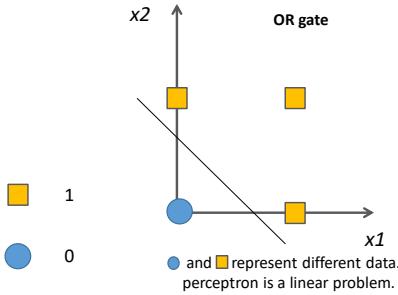
HUAWEI

- In the logic circuit, the output -1 is changed to
- What other value combinations meet different gates?
- AND gate: (0.5, 0.5, 0.8) or (1.0, 1.0, 1.0)

How To Use a Single-layer Perceptron To Solve the XOR Problem

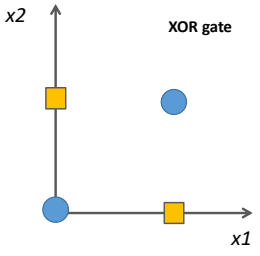
- When the OR gate truth table is expressed in a two-dimensional coordinate system, the single-layer perceptron linearly separates the two types of values.

$$y = \begin{cases} 0 & (-0.5 + x_1 + x_2 \leqslant 0) \\ 1 & (-0.5 + x_1 + x_2 > 0) \end{cases}$$



and represent different data. A single-layer perceptron is a linear problem. We can use a straight line to divide the data and implement the classification function.

- The XOR gate is also referred to as a logical XOR circuit. The output is 1 only when either *x1* or *x2* is 1.
- Can we use a single-layer perceptron to distinguish between 0 and 1 in an XOR gate?



x_1	x_2	y
0	0	0
1	0	1
0	1	1
1	1	0

XOR truth table

and represent the output of the XOR gate. Can we divide the output with a straight line?

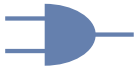
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The two lines may be separated, leading to two layers.

How To Use the AND, NAND, and OR Gates To Implement the XOR Gate

- If a single-layer perceptron cannot solve the XOR problem, can we combine implemented logic circuit gates to implement the XOR gate?



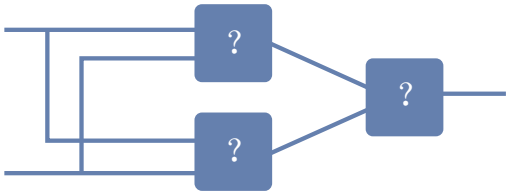
AND gate



NAND gate



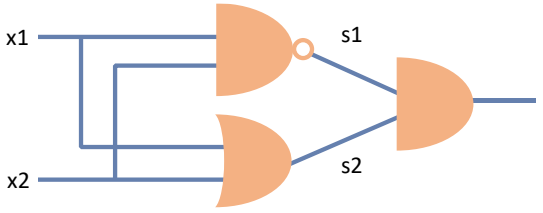
OR gate



Replace the question marks (?) with the AND gate, NAND gate, and OR gate to implement the XOR gate.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Implementation of the XOR Gate — Two-layer Perceptron



The XOR gate can be implemented by combining the AND, NAND, and OR gates.

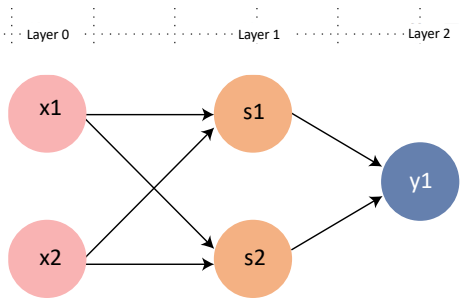
x_1	x_2	s_1	s_2	y
0	0	1	0	0
1	0	1	1	1
0	1	1	1	1
1	1	0	1	0

Truth table of the XOR gate implementation process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Multi-layer Perceptron

- Convert the previous logic gate representation into a perceptron representation:

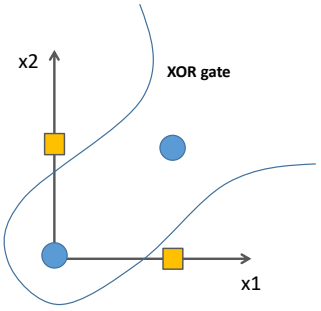


1. The two neurons at layer 0 receive an input signal and send it to a neuron at layer 1.
2. The neuron at layer 1 sends the signal to the neuron at layer 2, which then outputs y.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Nonlinearity

- To solve the XOR problem, a non-linear curve is required to separate 0 and 1 to simulate the XOR gate effect.

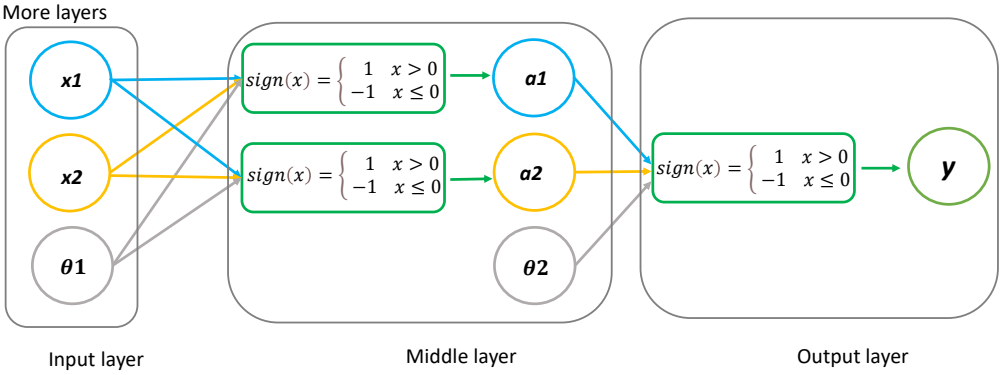


- Question: How does a multi-layer perceptron transform the perceptron function from linear to nonlinear?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Source of Nonlinearity of a Multi-layer Perceptron — More Layers

- A multi-layer perceptron can solve complex problems for the following reasons:
  - More layers

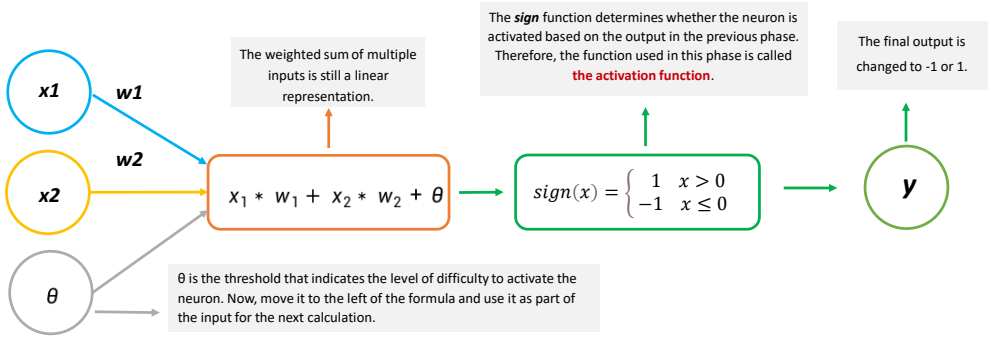


3-layer perceptron

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Source of Nonlinearity of a Multi-layer Perceptron — Nonlinear Activation Function

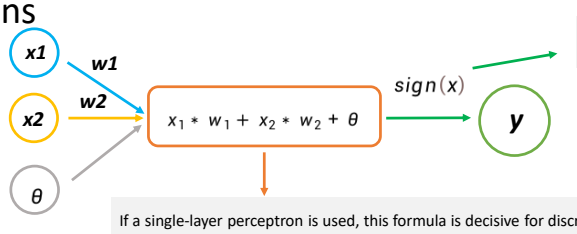
- A non-linear function is used to process the output.



Calculation process of a single-layer perceptron

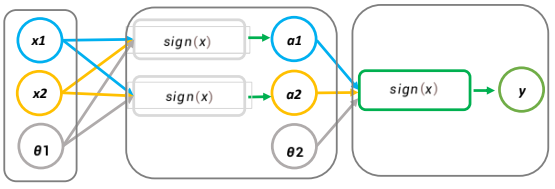
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The Importance of Multi-layer Perceptrons and Nonlinear Activation Functions



The sign function only determines the output type.

If a single-layer perceptron is used, this formula is decisive for discrimination, and it is a linear classifier.



If the sign function is not used at the middle layer:

$a_1 = x_1 * w_1 + x_2 * w_2 + \theta 1$ – linear

$a_2 = x_1 * w_3 + x_2 * w_4 + \theta 1$ – linear

The final output is as follows:

$$a_1 * w_5 + a_2 * w_6 + \theta 2 =$$
$$w_5(x_1 * w_1 + x_2 * w_2 + \theta 1)$$
$$+ w_6(x_1 * w_3 + x_2 * w_4 + \theta 1) + \theta 2$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

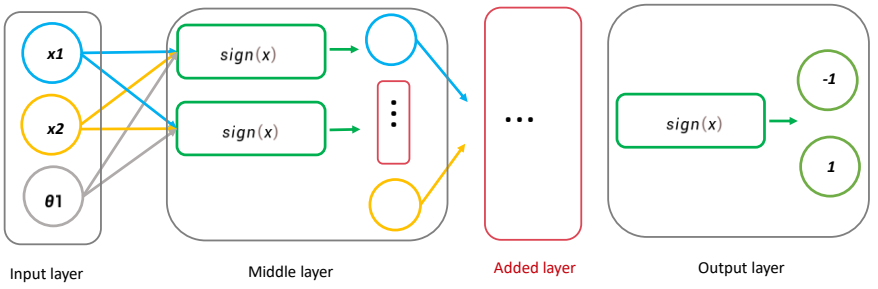
Contents

1. Perceptron
2. **Fully-connected Neural Network and How It Is Trained**
   - Deep Neural Network
   - Common Activation Functions
   - Neural Network Training
   - Optimizers
   - Regularization
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Perceptron Capability Improvement

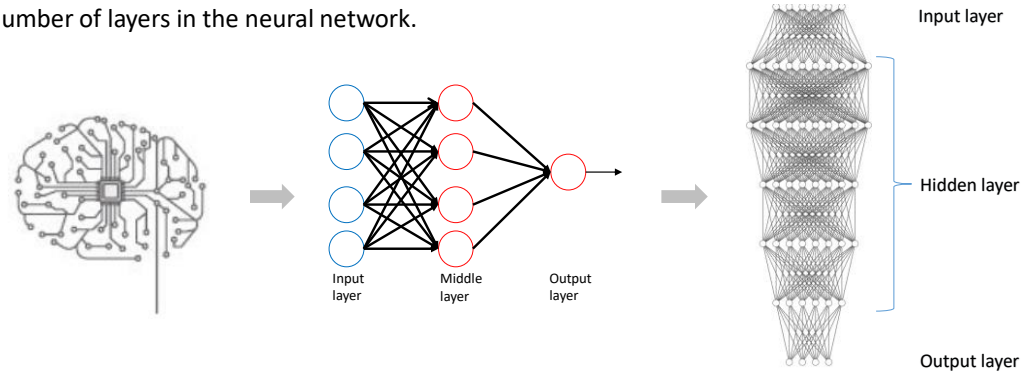
- We have seen the ability of multi-layer perceptrons to solve complex problems (fitting complex functions). However, its middle layer has only two units, and it can only solve binary classification problems.
- Can this ability be further improved if we continue to increase the number of nodes at the middle layer and the number of network layers?



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Deep Neural Network

- Generally, the deep learning architecture is a deep neural network. The depth refers to the number of layers in the neural network.



Human neural network | Perceptron | Deep neural network

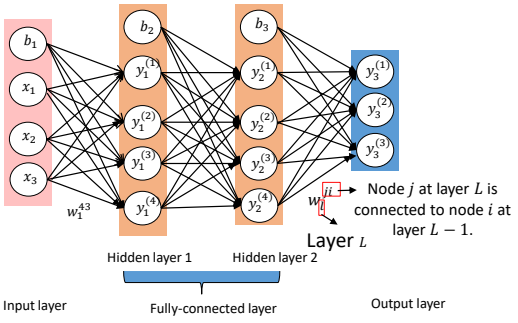
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- What is deep learning? In the previous course, we have learned that deep learning is a subset of machine learning.
- Generally, the deep learning architecture is a deep neural network. The depth refers to the number of layers in the neural network.
- The network is built by simulating the human neural network.
- In the design and application of artificial neural networks (ANNs), the following factors need to be considered: neuron functions, connection modes among neurons, and network learning (training).
- Source of image on the left: http://3ms.huawei.com/km/static/image/detail.html?fid=59876

Feedforward Neural Network Structure

- A Feedforward Neural Network (FNN) is a typical deep learning model. It has the following features:
  - As the input layer, the first layer provides the initial data source for the network.
  - Each neuron at the hidden layer and the output layer receives the weighted sum of all neurons from the previous layer, and provides the output to the next layer through the activation function.
  - A unidirectional multi-layer structure is used. The entire network provides no feedback. Signals are transmitted unidirectionally from the input layer to the output layer. The network can be represented by a directed acyclic graph (DAG).



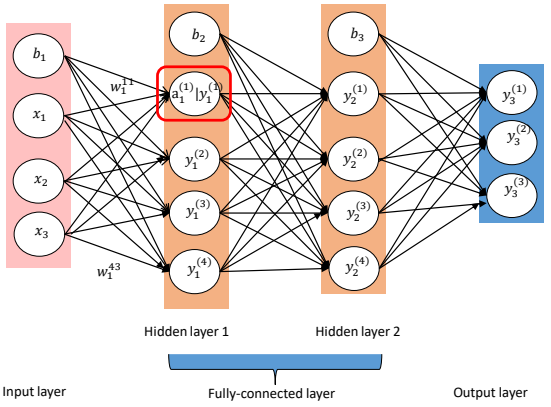
- ***X*** indicates the input data.
- ***W*** indicates the weight.
- ***B*** is the threshold for activating a neuron, and is referred to as a bias in a feedforward neural network.
- ***Y*** is the output of a neuron, and represents the output after the input is processed by the activation function.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- It is also called the Feedforward Neural Network.

Calculating FNN



The first neuron at hidden layer 1 is expanded, as marked by the red box in the figure. A single neuron is calculated as follows:

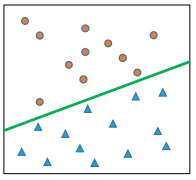
$$a_1^{(1)} = \sum_1^m (x_i * w_1^{1i}) + b_1$$

$$y_1^{(1)} = f(a_1^{(1)})$$

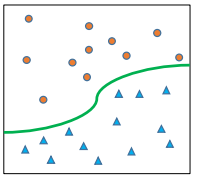
...

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

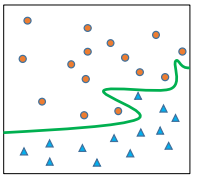
How Do Additional Hidden Layers Impact Neural Networks?



0 hidden layers



3 hidden layers



20 hidden layers

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The more hidden layers, the stronger identification capability of the neural network.

Adding Nodes at a Single Layer

- As we shift from single-layer to the multi-layer perceptrons, we realize that we can increase both the number of layers on a network and the number of network nodes at a single layer.

- Can we enhance network capabilities simply by adding nodes to a single layer?



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

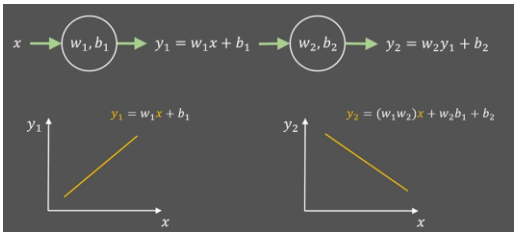
Contents

1. Perceptron
2. **Fully-connected Neural Network and How It Is Trained**
   - Deep Neural Network
   - Common Activation Functions
   - Neural Network Training
   - Optimizers
   - Regularization
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Deep Neural Networks Still Require Nonlinear Activation Functions

- Apart from multi-layer perceptrons, deep neural networks still need nonlinear activation functions to have nonlinear capabilities.
- Use a linear activation function, for example: f(x) = wx +b

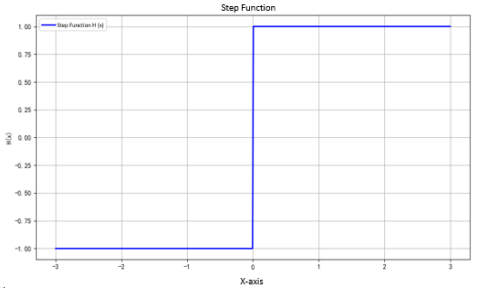


- What are some common activation functions?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Step Function

- Each neuron node in the neural network receives the weighted sum of neurons at the upper layer as the input. After being processed by a nonlinear function, the input value enters the next layer as the input. This nonlinear function is referred to as the activation function.
- The sign function is the activation function used in the multi-layer perceptron and is a step function with the following derivative:

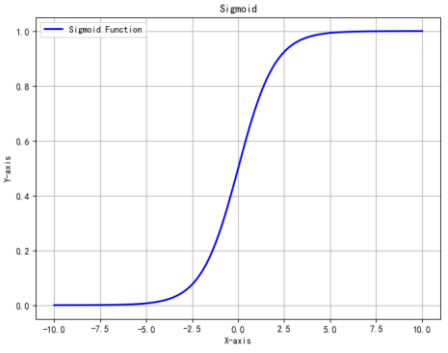


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Sigmoid Function

- The sigmoid function is an activation function frequently used in neural networks.
- For an input with a domain in R, the sigmoid function maps it to an output within the interval (0,1). Because it compresses any input in the range (-∞,+∞) to a value within the interval (0,1), the sigmoid function is often referred to as a squeezing function.

$$\text{sigmoid}(x) = \frac{1}{1+\exp(-x)}$$

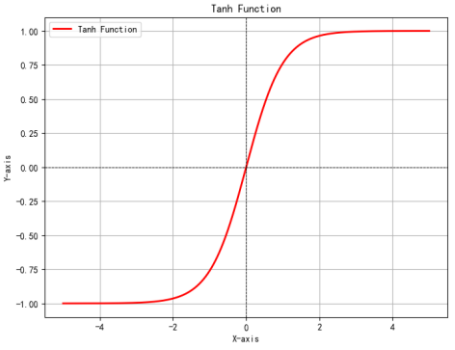


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tanh Function

- Similar to the sigmoid function, the tanh (hyperbolic tangent) function can also compress and convert its input to the interval (-1,1) using the following formula:

$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$



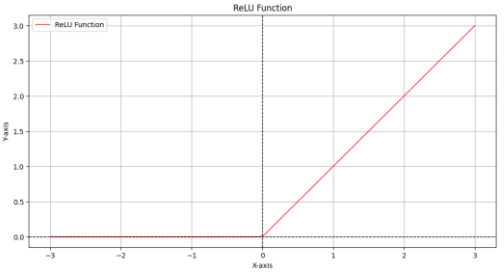
- When the input is near 0, the tanh function is close to linear transformation. It has a similar shape to the sigmoid function. The difference is that the tanh function is centrosymmetric about the origin of the coordinate system.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

ReLU Function

- Rectified linear unit (ReLU) is one of the most popular activation functions because it is easy to implement and performs well in various prediction tasks.
- ReLU provides a simple nonlinear transformation. With a given element *x*, the ReLU function is defined as the maximum value between the element and 0.

$$\mathrm{ReLU}(x) = \begin{cases} max(0, x) & , x >= 0 \\ 0 & , x < 0 \end{cases}$$

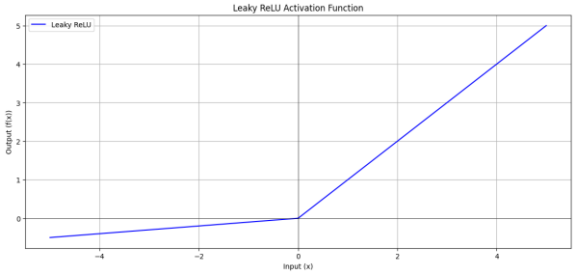


Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Leaky ReLU Function

- The Leaky ReLU function is an enhanced ReLU function. It assigns a non-zero slope to all negative values.

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & , x > 0 \\ \alpha x & , x \leq 0 \end{cases}$$



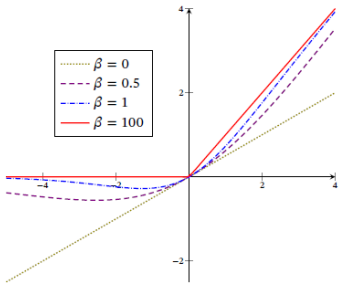
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Swish Function

- The Swish function is a self-gated activation function, and is defined as:

$$\text{swish}(x) = x\sigma(\beta x)$$

- Where σ (.) is the sigmoid function. β is a learnable parameter or a fixed hyper-parameter. σ() ∈ (0, 1) can be regarded as a soft gating mechanism. When σ (βx) is close to 1, the gate is open, and the output of the activation function is approximate to x itself. When σ (βx) is close to 0, the gate closed, and the output is approximate to 0.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

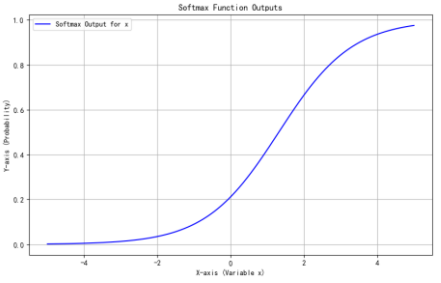
HUAWEI

- When β = 0, the Swish function becomes a linear function $x/2$. When β = 1, the Swish function is approximately linear when $x > 0$, and is approximately saturated and has certain non-monotonicity when $x < 0$. When $\beta \to +\infty$, $\sigma(\beta x)$ tends to be a discrete 0-1 function, and the Swish function approximates to a ReLU function. Therefore, the Swish function can be considered as a non-linear interpolation function between a linear function and a ReLU function, and the degree is controlled by the β parameter.

Softmax Function

- The Softmax function is usually used for multiclass classification in deep learning.
- It converts a K-dimensional vector containing any real number into another K-dimensional vector within the range of (0,1), and the sum of all elements is equal to 1. Therefore, the vector output by the Softmax function can be interpreted as a probability distribution, which is suitable for representing the probabilities of different classifications in the classification model.

$$\sigma(z)_j = \frac{e^{z_j}}{\sum_k e^{z_k}}$$



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

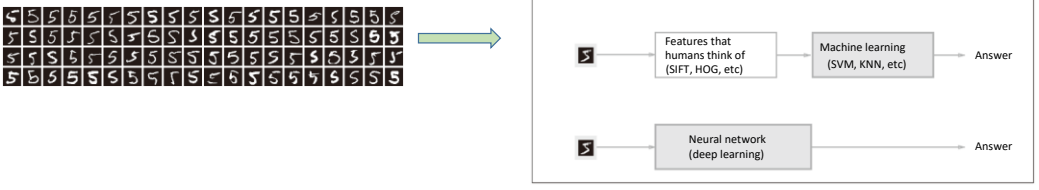
Contents

1. Perceptron
2. **Fully-connected Neural Network and How It Is Trained**
   - Deep Neural Network
   - Common Activation Functions
   - Neural Network Training
   - Optimizers
   - Regularization
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Learning from Data

- The primary difference between a neural network and a multi-layer perceptron is that, when solving nonlinear problems, the neural network can automatically learn how to extract features to form a problem-solving model. This learning ability depends on data.



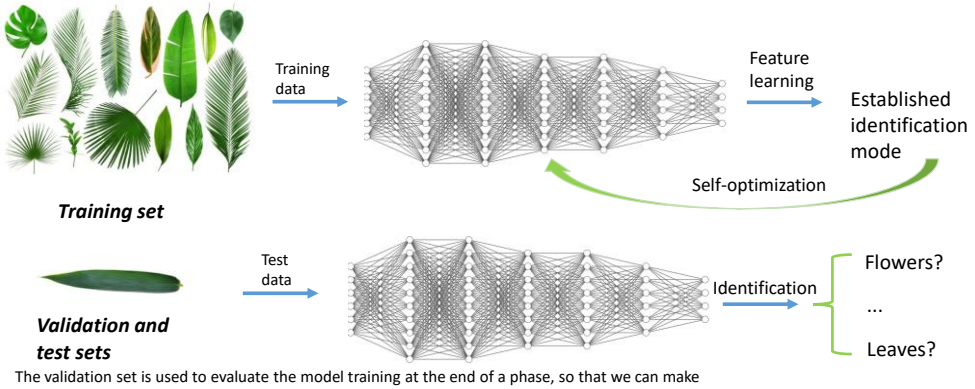
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Data is the foundation of machine learning. Finding answers from data, discovering patterns from data, and telling stories based on data... Without data, machine learning will not be possible at all. Therefore, data is the core of machine learning. This data-driven approach helps us get rid of the human-centered approach.

Data Division

- A dataset is usually divided into a training set, a validation set, and a test set.



The validation set is used to evaluate the model training at the end of a phase, so that we can make adjustments based on the validation results.
The test set is generally data that has never been seen by a formally-working model. The results obtained with the test set directly reflect the model's performance.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=57074
- https://3ms.huawei.com/km/static/image/detail.html?fid=56891

How Does a Neural Network Learn?





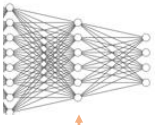




Human learning process

How is the error calculated?





Identification result

True result

Calculation error

Parameter adjustment

Neural network learning process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

Loss Functions

- A loss function is used to measure the difference between the model's predicted value and the actual value in machine learning and deep learning. It quantifies the degree to which a model makes mistakes during training.
- Common loss functions
  - Mean square error: mainly used for regression tasks.

$$E = \frac{1}{2}\sum_{k}(y_k - t_k)^2$$

y_k represents the output of the neural network, t_k represents supervised data, and k represents the number of data dimensions.

  - Cross entropy loss function: mainly used for classification tasks.

$$E = -\sum_{k} t_k \log y_k$$

y_k represents the output of the neural network, and t_k represents the correct solution label.

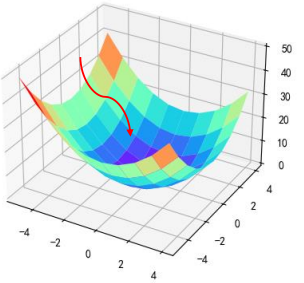
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Gradient Descent

- Neural network learning is a process of finding the optimal parameters. As the parameters are optimized, the neural network's predicted value gradually approaches the real value, and the loss function decreases. Therefore, the optimal parameters allow us to obtain the minimum value of the loss function.
- The gradient is the direction in which the multivariate function value increases the quickest. Therefore, all parameters should be optimized in the reverse gradient direction.

The gradient of the multivariate function $C(W) = f(w_0, w_1, \dots, w_n)$ at $W' = [w_0', w_1', \dots, w_n']^T$ is as follows:

$$\nabla f(w_0', w_1', \dots, w_n') = [\frac{\partial f}{\partial w_0}, \frac{\partial f}{\partial w_1}, \dots, \frac{\partial f}{\partial w_n}]^T|_{W=W'}$$



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Batch Gradient Descent (BGD) Algorithm

- <*X*, *t*> indicates a sample in the training set. *X* is the input value vector, *t* is the target output, *o* is the actual output, *η* is the learning rate, and *C* is the loss function.

  Initialize each w_i to a random value with a smaller absolute value.

  Before the termination condition is met, do as follows:

  - Initialize each Δw_i to zero.
  - For each <*X*, *t*> in the training set, do as follows:
    - Input X to this unit and calculate the output o.
    - For each w_i in this unit: Δw_i+= -η$\frac{1}{n}\sum_x\sum_{d\in D}\frac{\partial C(t_d,o_d)}{\partial w_i}$
  - For each w_i in this unit: w_i+= Δw_i

- This gradient descent algorithm is not used often because the convergence is very slow since all training samples need to be computed every time the weight is updated.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The BGD algorithm uses the entire training set for each learning. Therefore, each update is made towards the correct direction, ensuring convergence to the extrema in the end. A convex function converges to a global extremum, and a non-convex function may converge to a local extremum. However, this algorithm requires longer learning time and more memory resources.

Stochastic Gradient Descent (SGD) Algorithm

- A common variant, Incremental Gradient Descent or Stochastic Gradient Descent, has been developed to solve the issues found in the BGD algorithm. One of its implementations is online learning, which updates the gradient based on each sample.

$$\Delta w_i = -\eta \frac{1}{n} \sum_{x} \sum_{d \in D} \frac{\partial C(t_d, o_d)}{\partial w_i} \Rightarrow \Delta w_i = -\eta \sum_{d \in D} \frac{\partial C(t_d, o_d)}{\partial w_i}$$

- SGD:
  - Initialize each w_i to a random value with a smaller absolute value.
  - Before the termination condition is met, do as follows:
    - Randomly select <X, t> in the training set:
      - Input *X* to this unit and calculate the output *o*.
      - For each w_i in this unit: w_i += -η$\sum_{d \in D} \frac{\partial C(t_d, o_d)}{\partial w_i}$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- This gradient descent algorithm updates the weight based on each sample. Most training samples contain noise. As a result, when the extremum is approximated to, the gradient direction is oriented up and down near the extremum but difficult to converge to the extremum.

Mini-batch Gradient Descent (MBGD) Algorithm

- The MBGD algorithm was designed to address the issues found in the previous two gradient descent algorithms and is now most widely used. It uses a small fixed batch size (BS) of samples to compute Δw_i and update weights.

  Initialize each w_i to a random value with a smaller absolute value.

  Before the termination condition is met, do as follows:

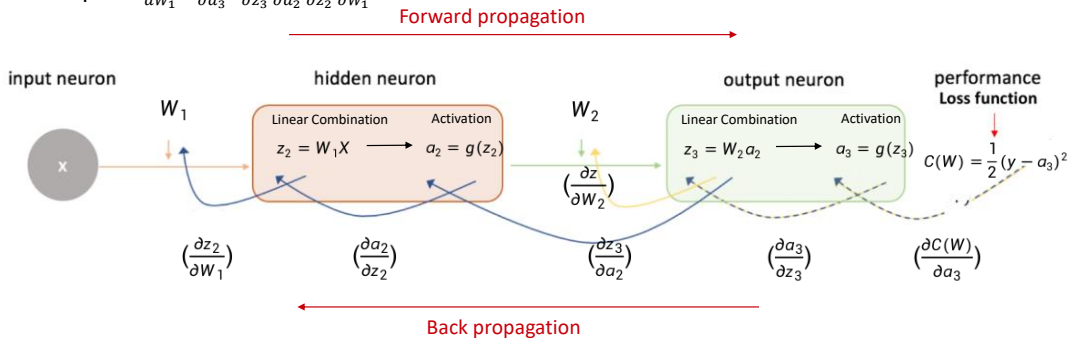
  - Initialize each Δw_i to zero.
  - For each <*X*, *t*> in the next batch of samples (number of samples = BS) obtained from the training set, do as follows:
    - Input *X* to this unit and calculate the output *o*.
    - For each w_i in this unit: Δw_i+= -η$\frac{1}{n}\sum_x\sum_{d\in D}\frac{\partial C(t_d,o_d)}{\partial w_i}$
  - For each w_i in this unit: w_i+= Δw_i
  - If it is the last batch, shuffle the training samples.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- This gradient descent algorithm considers both the efficiency and gradient stability. It may easily overshoot the local minimum and is the most commonly used gradient descent algorithm in actual work. The batch size varies with specific problems and is usually set to 32.

Network Training Process

- Forward propagation: $C(W) = \frac{1}{2}(y - a_3)^2 = \frac{1}{2}(y - (g(W_2 g(W_1 X))))^2$.
- If the parameter W_1 needs to be updated, according to $W_{t+1} = W_t - \eta \frac{dC}{dW}$, compute: $\frac{dC}{dW_1} = \frac{\partial C(W)}{\partial a_3}\frac{\partial a_3}{\partial z_3}\frac{\partial z_3}{\partial a_2}\frac{\partial a_2}{\partial z_2}\frac{\partial z_2}{\partial W_1}$.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- As shown in the figure, the sequence of backpropagation is to multiply the signal LOSS by the local derivative of the node and then send the result to the next node.

Chain Rule

- Composite function

$$z = (x + y)^2 \quad \text{Breakdown} \quad \begin{matrix} z = t^2 \\ t = x + y \end{matrix}$$

- Deriving the composite function: If a function is represented by a composite function, the derivative of the composite function may be represented by the product of derivatives of functions that form the composite function.

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial t}\frac{\partial t}{\partial x} \quad \Rightarrow \quad \frac{\partial z}{\partial x} = \frac{\partial z}{\partial \cancel{t}}\frac{\partial \cancel{t}}{\partial x}$$

$\frac{\partial z}{\partial x}$ derivative (the derivative of z with respect to x) may be expressed by $\frac{\partial z}{\partial t}$ (the derivative of z with respect to t) multiplied by $\frac{\partial t}{\partial x}$ (the derivative of t with respect to x).

- This is the chain rule.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Neural networks propagate forward layer by layer. How can we calculate the gradients of all parameters based on the gradient of the loss function?

- If a function is represented by a composite function, the derivative of the composite function may be represented by the product of derivatives of functions that form the composite function.

Vanishing and Exploding Gradients (1)

- Vanishing gradient: As the number of network layers increases, the derivative value of backward propagation decreases, and the gradient vanishes.
- Exploding gradient: As the number of network layers increases, the derivative value of backward propagation increases, and the gradient explodes.
- Cause: $y_i = \sigma_{(z_i)} = \sigma(w_i x_i + b_i)$, *where* σ is the sig*moid* function.



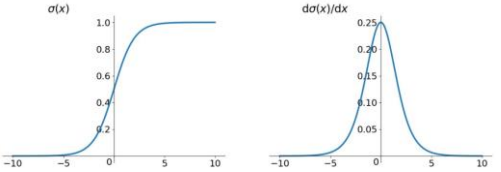
- Backward propagation can be deduced as follows:

$$\frac{\partial C}{\partial W_1} = \frac{\partial C}{\partial y_4}\frac{\partial y_4}{\partial z_4}\frac{\partial z_4}{\partial x_4}\frac{\partial x_4}{\partial z_3}\frac{\partial z_3}{\partial x_3}\frac{\partial x_3}{\partial z_2}\frac{\partial z_2}{\partial x_2}\frac{\partial x_2}{\partial z_1}\frac{\partial z_1}{\partial X_1}$$
$$= \frac{\partial C}{\partial y_4}\sigma'(z_4)w_4\sigma'(z_3)w_3\sigma'(z_2)w_2\sigma'(z_1)x$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Vanishing and Exploding Gradients (2)

- The maximum value of $\sigma'(x)$ is $\frac{1}{4}$:



- The network weight | w | is usually less than 1. Therefore, $|\sigma'(z)\, w| \leq \frac{1}{4}$. When using the chain rule, the value of $\frac{\partial C}{\partial b1}$ becomes smaller when the number of layers increases, resulting in gradient vanishing.
- When the network weight | w | is large, that is $|\sigma'(z)\, w| > 1$, the gradient explodes.
- Solution: Use gradient clipping to prevent the gradient from exploding, and use the ReLU activation function and LSTM neural network to mitigate gradient vanishing.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This problem occurs when the value of w is large. However, this problem rarely occurs when the sigmoid activation function is used. It is because that the value of $\sigma'(z)$ is also related to w ([equation]), unless the input value [equation] of this layer is within a small range.
- Gradient exploding and vanishing are caused by large network depth and unstable network weight update. In essence, they are caused by the chain rule in gradient backward propagation.
- **Gradient clipping** is proposed to solve gradient exploding. The principle is to set a gradient clipping threshold. When being updated, if the gradient exceeds the threshold, the gradient must be limited under the threshold. This prevents gradient exploding.
- Another approach for resolving gradient exploding is weight regularization. l1 regularization and l2 regularization are commonly used. All deep learning frameworks provide APIs for regularization. For example, in TensorFlow, if regularization parameters are set during network setup, you can use the following code to calculate the regularization loss.
- ReLU: The principle is simple. If the derivative of the activation function is 1, gradient vanishing and exploding do not occur, and each network layer has the same update rate. The ReLU function is developed for this purpose.
- For long-short term memory (LSTM) networks, gradient vanishing does not easily occur thanks to the complex gates inside the LSTM. As shown in the following figure, LSTM can memorize the residual memory from previous training through the internal gates when it is

updated. Therefore, it is often used in generative text. Currently, CNN-based LSTM is also available. Try it if you are interested.

Contents

1. Perceptron
2. **Fully-connected Neural Network and How It Is Trained**
   - Deep Neural Network
   - Common Activation Functions
   - Neural Network Training
     - Optimizers
   - Regularization
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Faster Model Training

- Neural network training is very slow when using normal methods.
- To speed up the process, we can use special tools called optimizers.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=57497

Optimizers

- There are various improved versions of gradient descent algorithms. In object-oriented programming, different gradient descent algorithms are often encapsulated into an object called an optimizer.
- There are several reasons for improving algorithms, including:
  - Accelerating algorithm convergence
  - Preventing or overshooting local extrema
  - Simplifying manual parameter setting, especially the learning rate
- Common optimizers include the common SGD optimizer, momentum optimizer, AdaGrad, RMSprop, and Adam.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

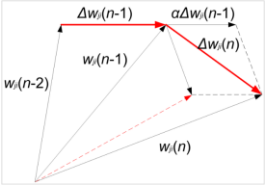
Momentum Optimizer

- Adding the momentum term to Δw_{ji} is a basic way to improve an algorithm. Assume that the weight correction of the n-th iteration is $\Delta w_{ji}(n)$. The weight correction rule is:

$$\Delta w_{ji}^{l}(n) = -\eta \delta_{i}^{l+1} x_{j}^{l}(n) + \alpha \Delta w_{ji}^{l}(n-1)$$

  Where α is a constant ($0 \le \alpha < 1$) called the momentum coefficient, and $\alpha \Delta w_{ji}^{l}(n-1)$ is the momentum term.

- Imagine that a small ball rolls down from a random point on the error surface. The momentum term gives inertia to the ball.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- $\eta \delta_{i}^{l+1} x_{j}^{l}(n)$ indicates the original gradient and direction, **j** represents the **j**-th neuron at the **l**-th layer, and **i** represents the **i**-th neuron at the **(l+1)**-th layer.

Advantages and Disadvantages of the Momentum Optimizer

- Advantages:
  - Enhances the stability of the gradient correction direction and reduces mutations.
  - In areas where the gradient direction is stable, the ball rolls faster and faster (there is a speed upper limit because $\alpha < 1$), which helps the ball quickly overshoot the flat area and accelerates convergence.
  - A small ball with inertia is more likely to roll over some narrow local extrema.
- Disadvantages:
  - The learning rate η and momentum α need to be manually set. Usually, additional experiments are necessary to determine the appropriate values.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AdaGrad Optimizer (1)

- The common characteristic of the SGD algorithm, MBGD algorithm, and momentum optimizer is that they update each parameter using the **same learning rate**.
- According AdaGrad principle, different learning rates should be set for different parameters.

$g_t = \frac{\partial C(t,o)}{\partial w_t}$	Computing gradient
$r_t = r_{t-1} + g_t^2$	Square gradient accumulation
$\Delta w_t = -\frac{\eta}{\varepsilon + \sqrt{r_t}} g_t$	Computation update
$w_{t+1} = w_t + \Delta w_t$	Application update

g_t indicates the t-th gradient, and r_t is the gradient accumulation variable. The initial value of r is 0, which **increases continuously**. η indicates the global learning rate, which needs to be set manually. ε is a small constant that is set to 10^{-7} for numerical stability.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

AdaGrad Optimizer (2)

- According to the AdaGrad optimization algorithm, r continuously increases while the overall learning rate keeps decreasing as the algorithm iterates. This is because we want the learning rate to **slow down** as the number of updates increases. When the model starts learning, we are far away from the optimal solution to the loss function. With more updates, we get closer and closer to the optimal solution, so learning speed can decrease.
- Advantage:
  - The learning rate is automatically updated. As the number of updates increases, the learning rate decreases.
- Disadvantage:
  - The denominator increases continuously, and learning becomes very slow at the end. As a result, the algorithm loses its effectiveness.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

RMSProp Optimizer

- The RMSProp optimizer is an improved AdaGrad optimizer. It introduces an attenuation coefficient to ensure that r **attenuates at a certain rate** in each round.
- The RMSProp optimizer addresses the fact that the AdaGrad optimizer ends the optimization process too early. It is suitable for handling unstable targets and is especially effective for RNNs.

$g_t = \frac{\partial C(t,o)}{\partial w_t}$	Computing gradient
$r_t = \beta r_{t-1} + (1-\beta) g_t^2$	Square gradient accumulation
$\Delta w_t = -\frac{\eta}{\varepsilon + \sqrt{r_t}} g_t$	Computation update
$w_{t+1} = w_t + \Delta w_t$	Application update

  - g_t indicates the *t*-th gradient, and *t* is the gradient accumulation variable. The initial value of *r* is 0, which **may not increase and needs to be adjusted with a parameter**. β indicates the attenuation factor. η indicates the global learning rate, which needs to be set manually. ε is a small constant and is set to 10^{-7} for numerical stability.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Adam Optimizer (1)

- Adaptive Moment Estimation (Adam) was developed based on AdaGrad and AdaDelta. It maintains two additional variables, m_t and v_t, for each variable to be trained.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- Where t represents the t-th iteration, and g_t is the calculated gradient. m_t and v_t are moving averages of the gradient and square gradient, respectively. From a statistical perspective, m_t and v_t are estimates of the gradient's first moment (the average value) and the second moment (the uncentered variance), respectively, hence the name of the method.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Adam Optimizer (2)

- If m_1 and v_1 are initialized using the zero vector, m_t and v_t are close to 0 during the initial iterations, especially when β_1 and β_2 are close to 1. To solve this problem, we use $\hat{m}_t$ and $\hat{v}_t$.

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

- Adam's weight update rule is as follows:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

- Although the rule requires manually setting η, β_1, and β_2, the setting process is much simpler. According to experiments, the default settings are $\beta_1 = 0.9$, $\beta_2 = 0.999$,$\epsilon = 10^{-8}$, η=0.001. In practice, Adam converges quickly. When convergence saturation is reached, η can be reduced. After several rounds of reduction, a satisfying (local) extrema is obtained. Other parameters do not need to be adjusted.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Perceptron
2. **Fully-connected Neural Network and How It Is Trained**
   - Deep Neural Network
   - Common Activation Functions
   - Neural Network Training
   - Optimizers
   - Regularization
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

End of Model Learning

- When does the model finish learning?
- When is the model ready for real use?
- Can we say the model is perfect once it has learnt all training data features?



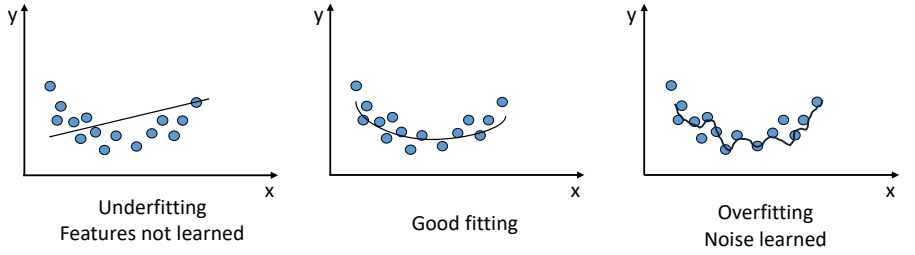
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=64716

Overfitting

- Problem description: The model performs well with the training set, but poorly with the test set.
- Root cause: There are too many feature dimensions, model assumptions, and parameters, too much noise, but very little training data. As a result, the fitting function predicts the training set almost perfectly, but the prediction result for a new test data set is unsatisfactory. Training data is overfitted without considering the model's generalization capability.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Overfitting is caused by insufficient data and complex model. In deep learning, small training data size and high model complexity are two common problems. They result in high prediction accuracy with training data, but low accuracy with test data. This is how overfitting occurs.
  - Obtain more data: Obtain more data from data sources and perform data augmentation.
  - Use a proper model: Reduce the number of network layers and the number of neurons to limit the fitting capability of the network.
  - Dropout
  - Regularization: Restricts weight increase during training.
  - Limit the training time. Pass the evaluation test.
  - Increase the noise: Input + Weight (Gaussian initialization)
  - Data cleansing/pruning: Rectifies or deletes incorrect labels.
  - Using multiple models: Bagging uses diverse models to fit training sets of different parts, while Boosting uses only simple neural networks.

L_1 Regularization

- Add the norm constraint L_1 to model parameters, that is,

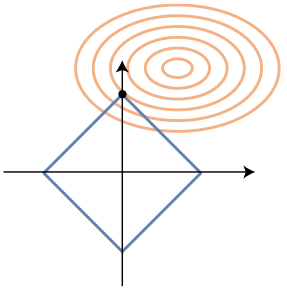
$$\tilde{J}(w; X, y) = J(w; X, y) + \alpha \|w\|_1$$

- If a gradient method is used to resolve the value, the parameter gradient is:

$$\nabla \tilde{J}(w) = \alpha sign(w) + \nabla J(w)$$

- The parameter optimization method is:

$$w_{t+1} = (w - \varepsilon \alpha sign(w)) - \varepsilon \nabla J(w)$$



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- In special cases, for secondary optimization, the parameter recursive formula is $w_i = sign(w_i^*)\max(|w_i^*| - \frac{\alpha}{\lambda_i}, 0)$, assuming that the corresponding Hessian matrix is a diagonal matrix. According to this formula, the parameter value is reduced to 0 if $|w_i^*| < \frac{\alpha}{\lambda_i}$. This is different from L_2 regularization. Compared with the L_2 optimization method, L_2 does not directly reduce the parameter value to 0 but to a value close to 0.

L_2 Regularization

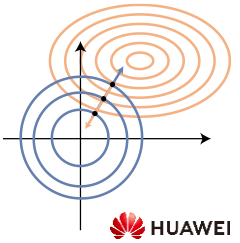
- Add norm penalty term L_2 to prevent overfitting.

$$\tilde{J}(w; X, y) = J(w; X, y) + \frac{1}{2}\alpha\|w\|_2^2$$

A parameter optimization method can be inferred using an optimization technology (such as a gradient method):

$$w = (1 - \varepsilon\alpha)w - \varepsilon\nabla J(w)$$

where ε is the learning rate. Compared with a common gradient optimization formula, this formula multiplies the parameter by a reduction factor.



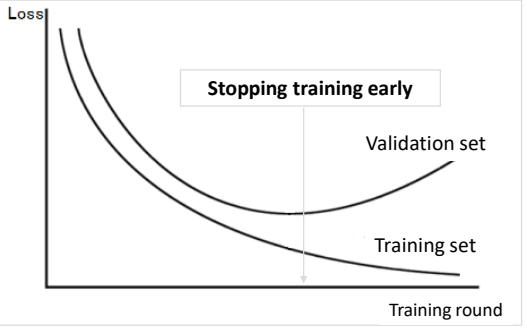
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Add a regular term Ω (θ)=12||w||22 to the target function to make the weight closer to the origin.
- If J is a secondary optimization formula, the model parameter may be represented as $\widetilde{w}_i = \frac{\lambda_i}{\lambda_i+\alpha} w_i$, that is, adding a control factor to the original parameter, where λ is an eigenvalue of the parameter Hessian matrix. Therefore:
  - When $\lambda_i \gg \propto$, the penalty factor has a small effect.
  - When $\lambda_i \ll \propto$, the corresponding parameter value is reduced to 0.

Stopping Training Early

- You can insert a data test on the validation set during the training. If the data loss of the verification set increases, the training is stopped early.

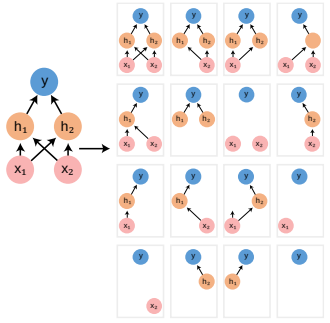


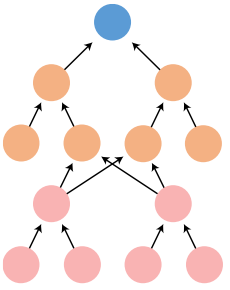
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Generally, a relatively large number (n) of iterations is set. First we need to save the current model, including the network structure and weights, and train the model for num_batch times (that is, an epoch) to obtain a new model. Then, use the test set as the input of the new model for testing. If the test error is greater than the last test error, we do not stop the test immediately. Instead, we should continue to conduct the training and test for several epochs. If the error does not decrease, we stop the model training when we believe that the minimum error is achieved in the last test.
- The common practice is to record the best test set accuracy p so far. If the accuracy does not exceed p in m consecutive periods, it may be considered that p does not increase anymore, and iteration may be stopped early (early stopping).
- As shown in the figure, the test error gradually decreases in the first several epochs. However, it slightly increases in a certain epoch. This indicates that overfitting has occurred.
- Overfitting is unwanted. We can prevent it through early stopping.
- Early stopping is to stop training before the test error starts to increase, even if the training has not converged, that is, the training error has not reached the minimum.

Dropout

- Dropout is a common simple regularization method, which has been widely used since 2014. Simply put, dropout randomly discards some inputs during training. Parameters corresponding to the discarded inputs are not updated.
- Dropout is an integration method. It combines all sub-network results and obtains sub-networks by randomly dropping inputs.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- When the samples of each test round are input into the neural network for training, a probability p is set, so that each neuron has a specific probability of dying and being excluded from network training.
- The process is as follows: 1. Delete neurons (temporarily dead) in the hidden layer of the network at a specific probability p, and keep the input and output neurons unchanged. 2. Propagate the input x forward through the modified network, and propagate back the loss result through the modified network. After this process is performed on a small batch of training samples, update corresponding parameters (w, b) on neurons that are not deleted with the SGD algorithm. 3. Then, repeat the process. Restore the deleted neurons. At this point, the deleted neurons remain unchanged, and the undeleted neurons are updated. Randomly select a subset of half size from the neurons at the hidden layer, and temporarily delete it. Remember to back up the parameters of the deleted neurons. For a small batch of training samples, propagate the input forward, backpropagate the loss, and then update the parameters (w, b) according to the SGD algorithm (The parameters of the undeleted neurons are updated, and the parameters of the deleted neurons remain the same as what they are before the deletion.)
- The sampling probability of each entry is 0.8 for the input and 0.5 for the hidden layers.
- Advantages:
  - Compared with weight decay and norm constraints, this strategy is more effective.
  - It is computationally cheap and simple and can be used in non-deep-learning models.
  - However, it is less effective when the training data is insufficient.
  - Stochasticity is not necessary or sufficient to achieve the normalizing effect of dropout. Invariant shielding parameters can be constructed to obtain good solutions.

- In addition to the preceding methods, we can also use semi-supervised learning, multi-task learning, early stopping, parameter sharing, ensemble methods, and adversarial training.

Contents

1. Perceptron
2. Fully-connected Neural Network and How It Is Trained
3. **Convolutional Neural Network**
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Convolutional Neural Network

- A CNN is a feedforward neural network. Its artificial neurons may **respond to units within the coverage range**. CNN excels at image processing. It includes a **convolutional layer**, a **pooling layer**, and a **fully-connected layer**.
- In the 1960s, Hubel and Wiesel studied the cortex neurons that cats use to sense their environment and select their direction. They found that their unique network structure could simplify feedback neural networks. They then proposed the CNN.
- CNN has become a hot research topic in many scientific fields, especially in pattern classification. The network is widely used because it can avoid complex image pre-processing, directly using original images instead.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

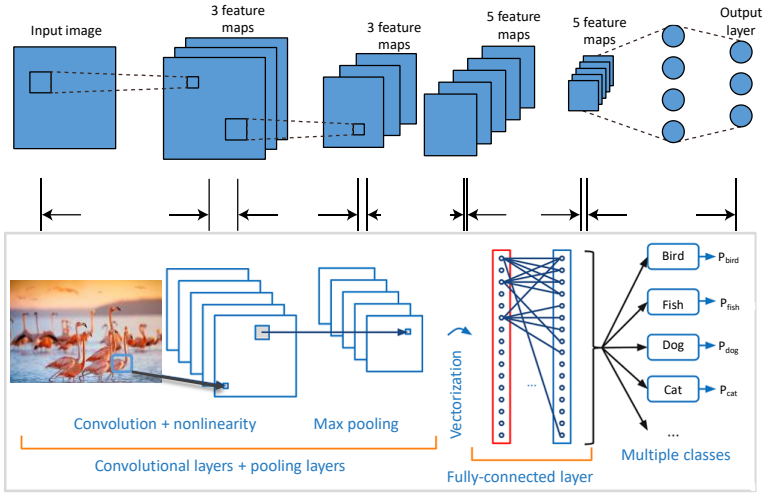
- A filter matrix is a set of fixed weight and can be seen as a constant filter (kernel). The convolution (adding each element, weighted by the kernel) is performed between an image (data from different data windows) and a kernel. This type of network is called CNN.
- Local receptive field: It is generally considered that human perception of the outside world is from local to global. Spatial correlations among local pixels of an image are closer than those among pixels that are far away. Therefore, each neuron does not need to know the global image. It only needs to know the local image and then the local information is combined at a higher level to generate global information. The idea of local network connection is also inspired by the biological visual system structure. The neurons in the visual cortex receive local information (respond to stimuli of certain regions).
- Parameter sharing: One or more filters can be used to scan the input images. The parameters of the filter are weights. At the layers scanned by the same filter, each filter uses the same parameters to perform weighted computation. Weight sharing means that the parameter values of each filter do not change when the filter scans the entire image. For example, if we have three feature filters and each filter scans the entire image. During the scanning process, the parameter values of the filters do not change. In other words, all elements of the image share the same weights.

Main CNN Concepts

- **Local receptive field:** It is believed that humans perceive the outside world from local to global. **The spatial correlations among an image's local pixels are closer than those among the pixels that are far away**. As such, each neuron does not need to know the global image; it only needs to know the local image. Then, the local information is combined at a higher level to generate global information.
- **Parameter sharing:** One or more convolution cores may be used to scan input images. Parameters carried by the convolution cores are weights. In a layer scanned by convolution cores, each core uses the same parameters for weighted computation. Weight sharing means that, when each convolution core scans an entire image, its **parameters are fixed.**

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

CNN Architecture



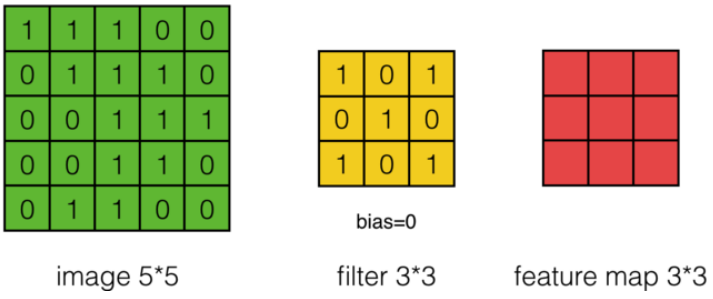
66 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Input layer: inputs data.
- Convolutional layer: composed of several convolutional units. The parameters of each convolutional unit are obtained by optimizing the backward propagation algorithm. The purpose of convolution computation is to extract different input features. The first convolutional layer may extract only some low-level features such as edges, lines, and angles. A multi-layer network can extract more complex features based on the low-level features.
- Rectified linear units layer (ReLU layer): uses ReLU f(x) = max(0, x) as the activation function.
- Pooling layer: partitions features obtained from the convolutional layer into some areas and outputs the maximum or minimum value, generating new features with a smaller spatial size.
- Fully-connected layer: integrates all local features into global features to compute the final scores for each type.
- Output layer: outputs the final result.
- Image on the left: http://image.huawei.com/tiny-lts/v1/images/0281bccdff8137c18a163beba5a2650c_4715x2927.jpg

Computing a Single Convolution Kernel (1)

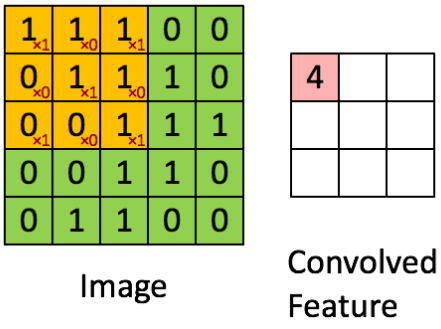
- Description of convolution computing



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Computing a Single Convolution Kernel (2)

- Result of convolution computing

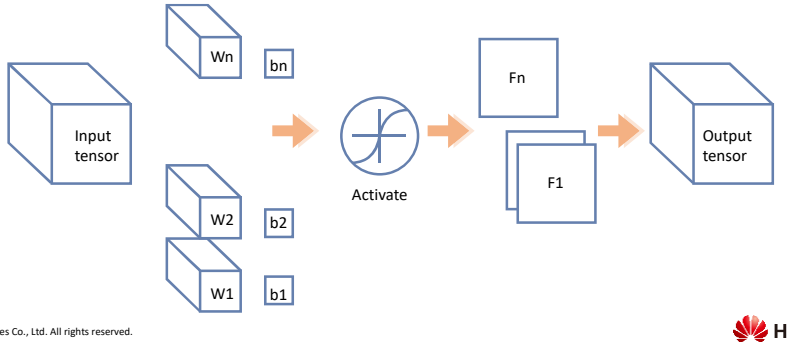


Hanbingtao, 2017, CNN

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Convolutional Layer

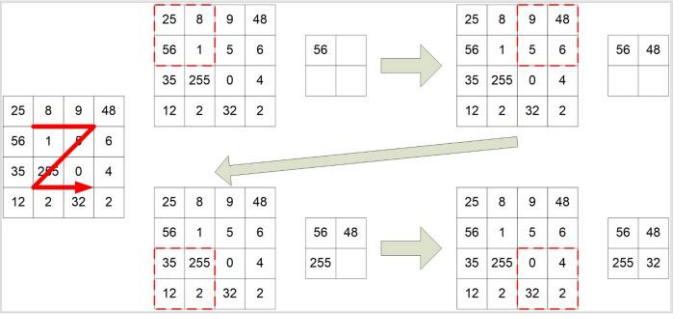
The basic CNN architecture is multi-channel convolution that consists of multiple single convolutions. The output of the upper layer (or the original image of the first layer) is used as the input of the current layer. It is then convolved with the layer's convolution kernel and serves as this layer's output. Each layer's convolution kernel is the weight to learn. Similar to the fully-connected layer, after the convolution is completed, the result should be biased and activated through activation functions before being input to the next layer.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Pooling Layer

Pooling combines nearby units to reduce the size of the input at the next layer, **reducing dimensions**. Commonly used pooling methods include max pooling and average pooling. In max pooling, the maximum value within a small square region is selected as the representative for that area. In average pooling, the average value of the small region is chosen as the representative, with the side length of this region equal to the pool window size. The following figure shows **max pooling** where the pool window size is 2.



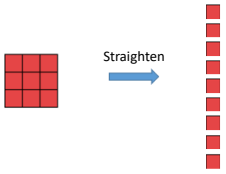
70 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The actual classification networks are feed forward networks that are formed by interconnected convolutional and pooling layers. The pooling layer has the following functions:
  - Invariance: Max pooling ensures invariance within a certain range because the maximum value of a region is the last output value regardless of its location.
  - Reducing the input size for the next layer: Pooling effectively reduces the size of the input data for the next layer, the number of parameters, and computation workload.
  - Obtaining fixed-length data: By properly setting the pooling window size and stride, we can obtain fixed-length outputs from variable-length inputs.
  - Increasing the scale: The features of the upper layer can be extracted from a larger scale.
  - Preventing overfitting: Pooling simplifies the network and reduces the fitting precision. Therefore, it can prevent overfitting (note the possible underfitting).

Fully-connected Layer

- The fully-connected layer is essentially a classifier. The features extracted on the convolutional and pooling layers are straightened and placed at the fully-connected layer to output and classify results.



- Generally, the Softmax function is used as the activation function of the final fully-connected output layer to combine all local features into global ones and compute the score of each class.

$$\text{softmax}=\frac{e^{z_j}}{\sum_k e^{z_k}}$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Perception
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. **Model Architecture Based on the Recurrent Neural Network**
   - RNN
   - LSTM
   - Seq2Seq
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recurrent Neural Network

- The recurrent neural network (RNN) is a **neural network that captures dynamic information in sequential data** through periodical connections of hidden layer nodes. It can classify sequential data.
- Unlike FNNs, the RNN can keep a context state and even store, learn, and express related information in context windows of any length. Unlike traditional neural networks, it is not restricted by the space boundary but can **extend time sequences**. In other words, a side exists between the hidden layer of the current moment and the hidden layer of the next moment.
- The RNN is widely used in scenarios related to sequences, such as videos consisting of image frames, audio consisting of clips, and sentences consisting of words.

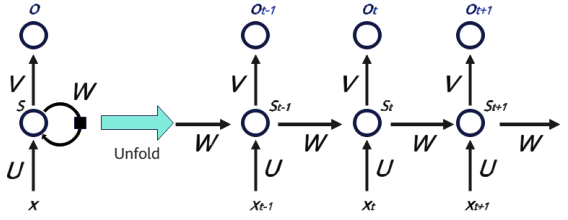
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

RNN Architecture

- X_t is the input of the input sequence at time *t*.
- S_t is the memory cell of the sequence at time *t* and caches previous information:

  $S_t = \sigma(UX_t + WS_{t-1})$.
- S_t traverses multiple hidden layers, and then traverses the fully-connected layer V to obtain the final output O_t at time *t*:

  $O_t = softmax(S_tV)$.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Back-Propagation Through Time

- BPTT:
  - Conventional backward propagation is an extension of the time sequence.
  - There are two errors in a memory cell at time step *t*: the partial derivative of the error C_t at time step *t* with respect to the memory cell and the partial derivative of the error at the next time step *t+1* with respect to the memory cell at time step *t*. These two errors need to be combined.
  - The longer the time sequence, the more likely will the loss of the gradient from the last time step to the weight gradient at the first time step contribute to the vanishing or exploding gradient problem.
  - The total gradient of the weight *w* is the accumulation of the gradients across all time steps.
- BPTT steps:
  1. Compute the output of each neuron (forward);
  2. Compute the error δ_j of each neuron (backward); and
  3. Compute the gradient of each weight.
- Updating weights using the SGD algorithm

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- z_t is the value before the memory cell enters the activation function. $\delta_t = \frac{\partial C_t}{\partial z_t} + w^T \times f_t'(z_t) \times \delta_{t+1}$

RNN Problems

- $S_t = \sigma(UX_t + WS_{t-1})$ extended on the time sequence.
- $S_t = \sigma\left(UX_t + W\left(\sigma\left(UX_{t-1} + W\left(\sigma\left(UX_{t-2} + W(\dots)\right)\right)\right)\right)\right)$
- While the standard RNN architecture solves the problem of information memory, **the information about long-term memory attenuates**.
  - Long-term information needs to be saved in many tasks. For example, a hint at the beginning of a speculative fiction may not be explained until the end.
  - RNN may not be able to save long-term information due to the limited memory cell capacity.
  - We expect that memory cells can remember key information.

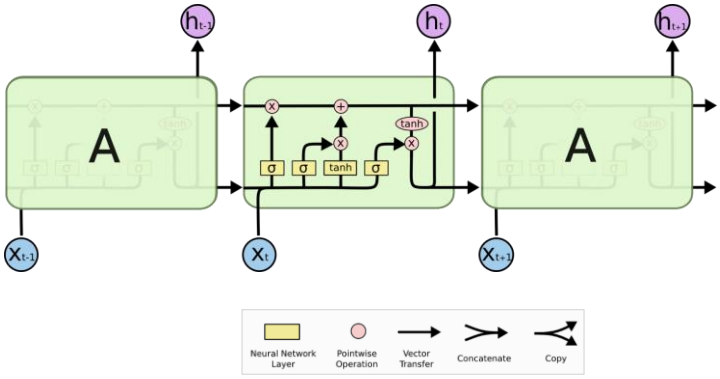
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Perception
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. **Model Architecture Based on the Recurrent Neural Network**
   - RNN
   - LSTM
   - Seq2Seq
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

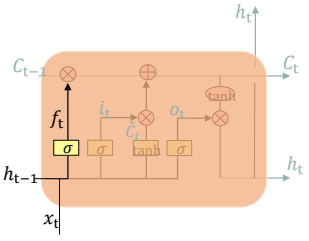
LSTM



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Forget Gate of LSTM

- The first step in LSTM is to decide the information to be discarded from the cell state.
- The decision is made through the **forget gate**. This gate reads h_{t-1} and x_t, and outputs a numeric value in the range from 0 to 1 for each digit in the cell state C_{t-1}. The value 1 indicates that the information is completely retained while the value 0 indicates that the information is completely discarded.

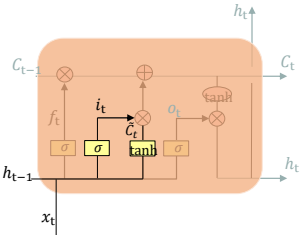


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Input Gate of LSTM

- This step is to determine the information to be stored in the cell state.
- It consists of two parts:
  - The Sigmoid layer is called the input gate layer, which determines the value to be updated.
  - A candidate value vector is created at the tanh layer and is added to the state.



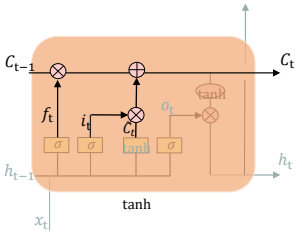
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Updating LSTM Information

- Time to update the earlier cell state: C_{t-1} is updated as C_t.
- We multiply the earlier state by f_t and discard information we decide to discard. $i_t * C_t$ is added. This is a new candidate value, which changes with the expected update of each state.

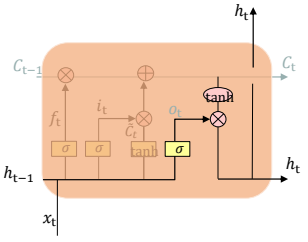


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Output Gate of LSTM

- Run a Sigmoid layer to determine which part of the cell state will be output.
- Then, we process the cell state through tanh (to obtain a value between −1 and 1) and multiply the value by the output of the Sigmoid gate. In the end, we output only the part we decide to output.

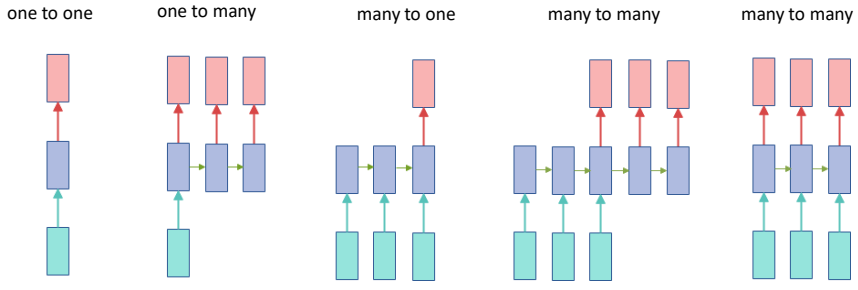


$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * tanh(C_t)$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

RNN Types



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

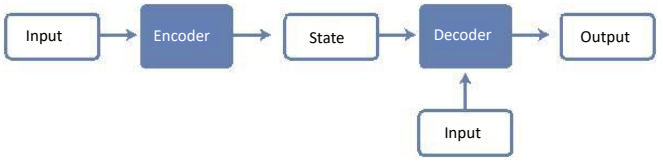
Contents

1. Perception
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. **Model Architecture Based on the Recurrent Neural Network**
   - RNN
   - LSTM
     - Seq2Seq
5. Transformer Architecture
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Encoder-Decoder Architecture

- Machine translation is a core task tackled by a sequence transduction model. Both the input and output of the machine translation are sequences of variable lengths. An architecture with two main components is needed to handle them.
  1. Encoder: Uses a variable-length sequence as input and converts it into an encoding state with a fixed shape.
  2. Decoder: Maps the fixed-shape encoding state to a variable-length sequence.
- This is referred to as the encoder-decoder architecture:



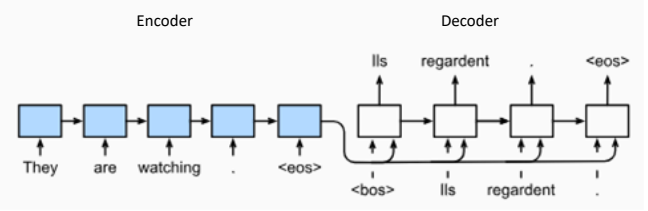
- Models such as RNN and LSTM can be selected for the encoder and decoder.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Both the RNN and the LSTM may be selected for the encoder and the decoder.

Seq2Seq

- Sequence-to-sequence (Seq2Seq): A sequence is input and another sequence is output. This is a typical encoder-decoder model. The encoder encodes the input data, and the decoder decodes the encoded data. It is also a solution to the time sequence problem.



- The encoding part is marked blue. The first <eos> indicates the end of encoding. The decoding part is left white. <bos> is the first input, indicating the start of decoding. The second <eos> indicates the end of decoding.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Encoder Calculation

- The encoder converts the variable-length input sequence into a context variable *c* with a fixed shape, and encodes information about the input sequence in the context variable.
  - It is assumed that the input sequence is *x1*, ..., *xt*, where *xt* is the tth lexical element in the input text sequence. At a time step *t*, the RNN converts the input feature vectors *xt* and *ht-1* (that is, the hidden state of the previous time step) of the lexical element *xt* into *ht* (that is, the hidden state of the current step). A function *f* is used to describe the transformation performed by the recurrent layer of the RNN:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$$

  - The encoder converts the hidden states of all time steps into context variables by using the selected function:

$$\mathbf{c} = q(\mathbf{h}_1, \ldots, \mathbf{h}_T)$$

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Decoder Calculation

- The context variable *c* output by the encoder encodes the entire input sequence *x1*, ..., *xT*. For the output sequence *y1*, *y2*, ..., *yT'* from the training dataset, at each time step *t'* (distinct from the time step *t* of the input sequence or encoder), the probability that the decoder outputs *yt'* depends on the previous output subsequence *y1*, ..., *yt'-1* and the context variable *c*. In other words, the probability is P (*yt'-y1*, ..., *yt'-1,c*).
- At any time step *t'* on the output sequence, the RNN uses the output *yt'-1* from the previous time step and the context variable *c* as its input. It then converts them and the previous hidden state *st'-1* to the hidden state *st'* at the current time step. Thus, a function g can be used to represent the transformation of the decoder's hidden layer:

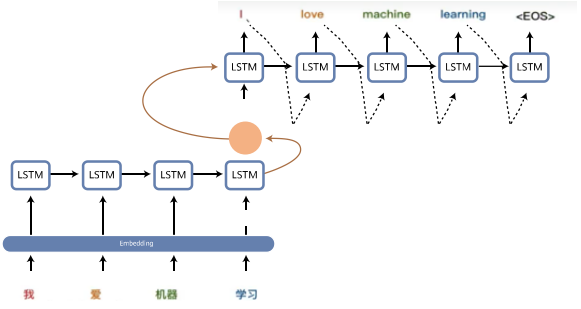
$$\mathbf{s}_{t'} = g(y_{t'-1}, \mathbf{c}, \mathbf{s}_{t'-1})$$

- After obtaining the decoder's hidden state, we can use the output layer and softmax operation to calculate the conditional probability distribution P(yt'1y1,…,yt'−1,c) of the output *yt'* at the time step *t'*.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Seq2Seq Application — Machine Translation





Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Seq2Seq Challenges

- Although Seq2Seq can solve the sequence problem, it still has the following challenges:
  - There is only one vector c between the encoder and the decoder to transfer information, and the length of c is fixed.
    - The semantic vector cannot completely represent information of the entire sequence. Since c has a fixed length, if the input data is too long, it will be compressed to a specified length, and information will be lost. Longer input means greater compression and higher information loss.
    - Long sentences are prone to information loss. In the case of *c* obtained through RNN encoding, earlier information tends to be gradually lost over time.
  - RNN-based Seq2Seq models do not support parallel computing, so the efficiency is low.
    - When calculating data at moment t, the RNN needs to output data at the previous moment, leading to low parallelism.
  - CNN-based Seq2Seq has many parameters, resulting in complex training.
  - During inference, a word can be predicted only after the previous word is output, slowing the inference.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Perception
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. **Transformer Architecture**
6. Basic Foundation Model Architecture

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Addressing Seq2Seq Challenges

- Storing all information in sequence is inefficient. Is there any way to resolve this problem?
- Think about how humans observe things. Generally, humans have a wide field of view. They glance all information within their field of view, quickly capture important information, and ignore the rest. Can this mechanism be applied to the sequence model?



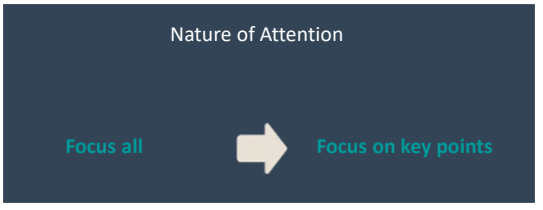
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=61858

Attention Mechanism

Attention is scene and from human cognitive capabilities. For example, when people observe a scene or deal with an event, they usually pay attention to the salient objects in the scene and want to grasp the main contradiction when dealing with an event. The Attention mechanism enables people to focus on important aspects while ignoring minor things, increasing their efficiency.





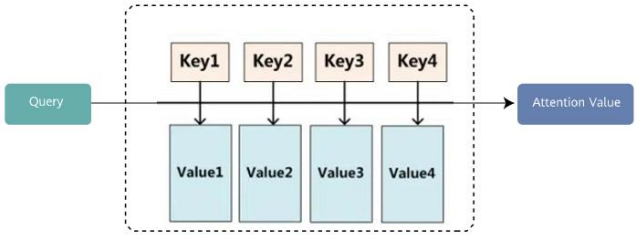
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=67539
- smartvi

Attention Principle

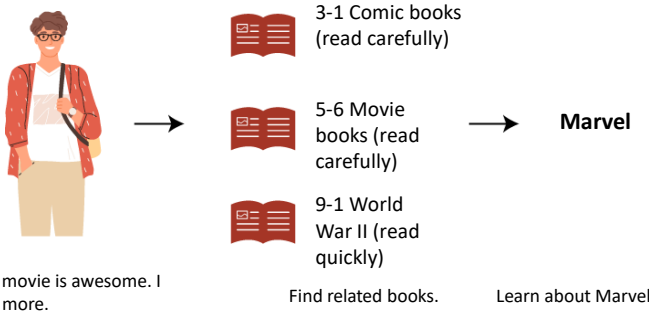
- Attention can be used without the encoder-decoder framework. The following figure shows the principle of attention without the encoder-decoder framework.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Understanding Attention in Reality

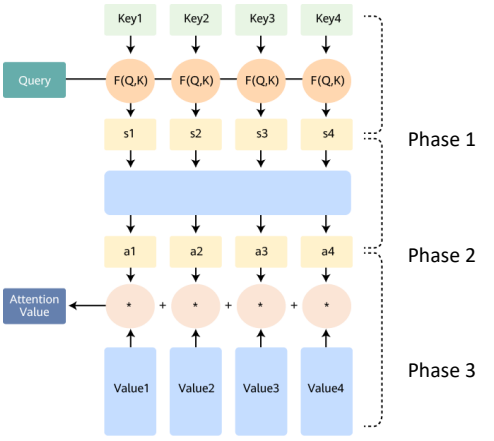
- There are many books (values) in a library (source). Numbers (keys) are assigned to the books to facilitate searches. When we want to know about Marvel (query), we can watch cartoons and movies and even read books related to World War II (Captain America).



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Attention Computing Process

- Step 1: Calculate the similarity between the Query and each Key to obtain a score s.
- Step 2: Convert the score s to a probability distribution in the range of [0,1] through softmax conversion to obtain a readily available weight.
- Step 3: Use [a1, a2, a3... an] as the weight matrix to perform weighted summation on values to obtain the final Attention value.



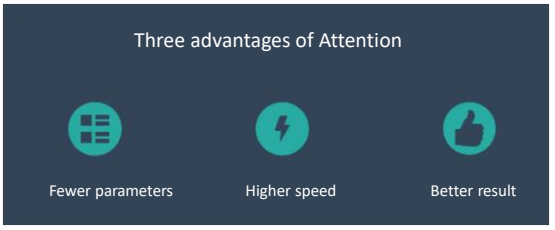
96 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- From the preceding modeling, we can feel that the idea of Attention is simple: **"weighted summation"**. Refer to an inappropriate analogy. Humans learn a new language in four stages: rote learning (learning grammar by way of recitation) -> outlining points (grasping key words in sentences) -> generalizing (being able to understand the context and the relationship behind the language in complex conversations) -> reaching the peak of perfection (immersive practice).
- This is similar to the development of Attention. The RNN era is a period of rote learning. The Attention model has evolved into a transformer, which has excellent expression learning capabilities, then to GPT and BERT. A wealth of practical experience has accumulated through multi-task large-scale learning

Advantages of Attention

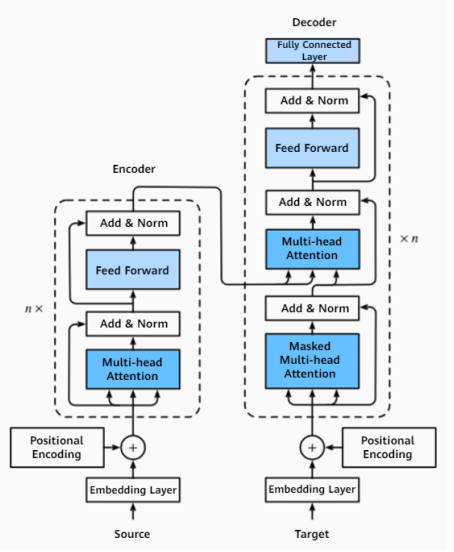
- Fewer parameters: Compared with CNN and RNN, this model is less complex and has fewer parameters. As such, it needs less computing power.
- Higher speed: Attention addresses the fact that RNN cannot perform parallel computing. The steps in the Attention mechanism do not depend on the previous step. As such, the Attention mechanism supports parallel processing, just like CNN.
- Better result: Before Attention was introduced, the model would struggle to remember older information, just like people with a short memory cannot remember things from the past.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Transformer

- In a 2017 paper titled *Attention Is All You Need,* Google proposed a Transformer model to process sequence-related problems based on the Attention (self-attention mechanism) structure.
- Transformer has supported many NLP tasks, such as text classification, machine translation, and reading comprehension.
- The Transformer model uses a whole new formula and rejects the CNN and RNN structures. It uses the Attention mechanism to automatically capture the relative associations at different positions of the input sequence, which is conducive to processing long texts. In addition, the model enables a high degree of parallelism, speeding up training.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Self-Attention

- The Attention mechanism takes effect between the target element and all source elements. In the encoder-decoder framework of a common task, the input source and output target are different. For example, for English-Chinese machine translation, the source is an English sentence, and the target is a translated Chinese sentence.

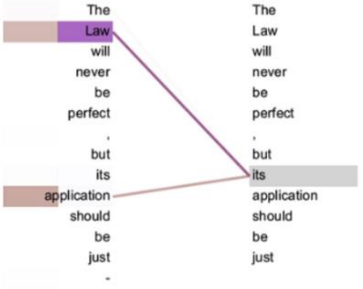
I love machine learning. ⟹ 我爱机器学习

- Transformer uses the self-attention mechanism. As the name implies, self-attention refers to the attention mechanism used between source elements or target elements, rather than between the source and the target. The two mechanisms use the same computing process, but the object to be computed is different.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example of Self-Attention

- A variant of the attention mechanism, the self-attention mechanism is less dependent on external information and outperforms in capturing the internal correlation of data or features.
- When applied to text, the self-attention mechanism primarily addresses the long-distance dependency problem by calculating the relationships between words.



If you want to know what "its" refers to and which words are relevant to "its" in this sentence, you can use "its" as a query, this sentence as a key and a value to calculate the value of attention. Through self-attention, we find that "its" is most relevant to "law" and "application."

- The self-attention mechanism focuses more on its own relationships. Therefore, the encoder and decoder of the self-attention mechanism can be used separately, which further generates different types of network structures.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

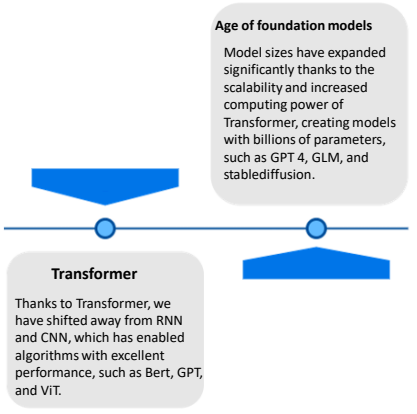
Contents

1. Perception
2. Fully-connected Neural Network and How It Is Trained
3. Convolutional Neural Network
4. Model Architecture Based on the Recurrent Neural Network
5. Transformer Architecture
6. **Basic Foundation Model Architecture**

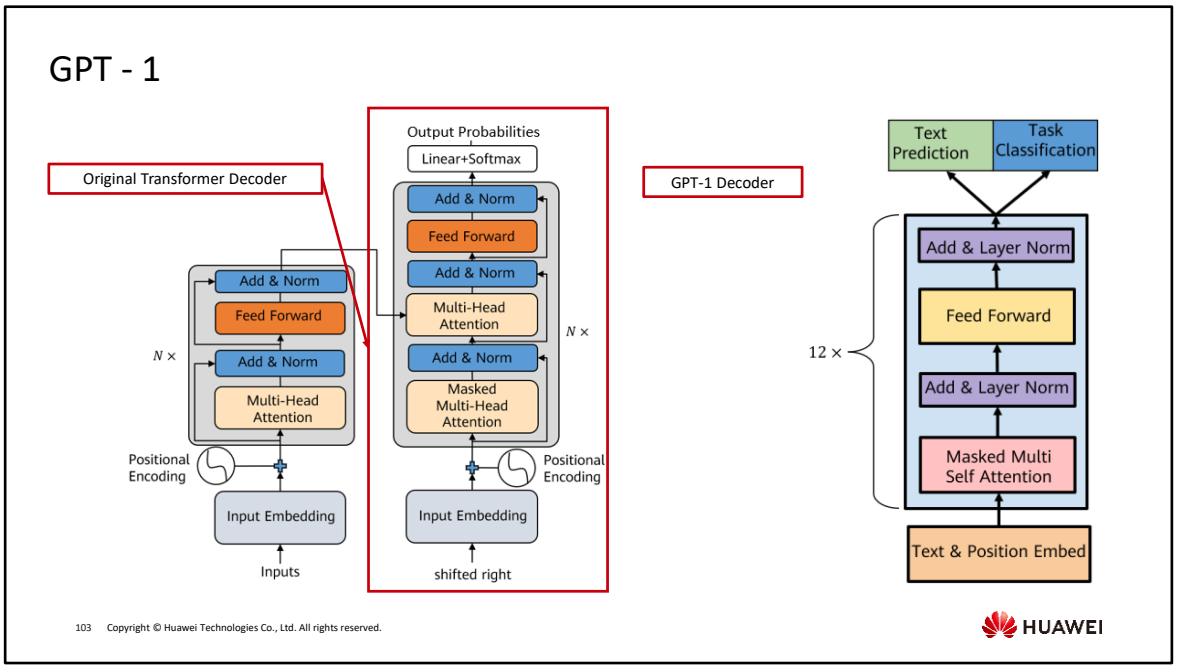
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

State of Foundation Models

- With higher computing capabilities and more data, we can train larger and deeper neural networks. Foundation models, such as the GPT series, LLaMA, and GLM, are trained with powerful generation and generalization capabilities based on massive amounts of data and computing resources.
- These models achieve excellent performance on a single task but can also quickly adapt to new tasks and data through fine-tuning.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.



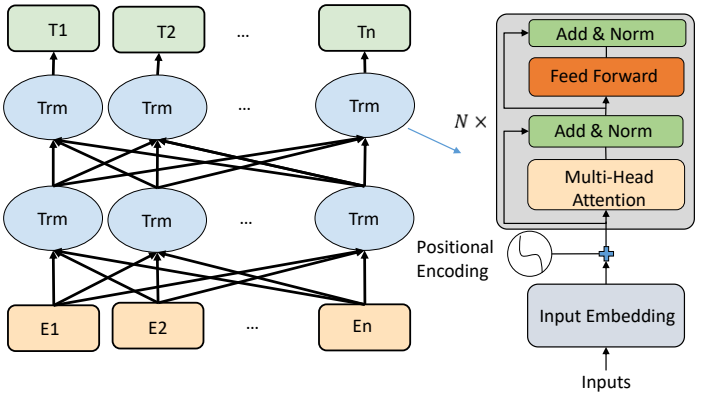
- GPT-1, the first version of OpenAI, is called GPT.
- Using Transformer instead of LSTM is because that, although pre-training helps capture some language information, the LSTM model limits its prediction capability in the short term. However, Transformer enables us to capture longer-term language structures while achieving better generalization performance in other different tasks.

• GPT-1 uses the decoder framework of Transformer.

- GPT-1 retains the masked multi-attention layer and feed forward layer of the decoder and expands the network scale. The number of layers is increased to 12. GPT-1 also increases the number of dimensions of Attention to 768 (originally 512) and the number of heads of Attention to 12 (originally 8). The number of hidden layers at the Feed Forward layer is increased from 2048 to 3072. The total number of parameters reaches 117 million.
- For the positional encoding part, GPT-1 is sharply different from a common transformer. The positional encoding of a common transformer is represented by cosine + sine, while GPT-1 uses random initialization similar to that of a word vector. In addition, update is performed during training, that is, each location is considered an embedding to be learned.

Bert

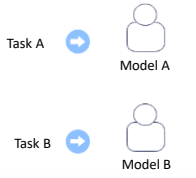
- Released in 2018, BERT is a bidirectional model that analyzes the context of a complete sequence and then performs predictions. This model is trained on a plain text corpus and Wikipedia using 3.3 billion words and 340 million parameters.
- BERT can answer questions, predict statements, and translate text.



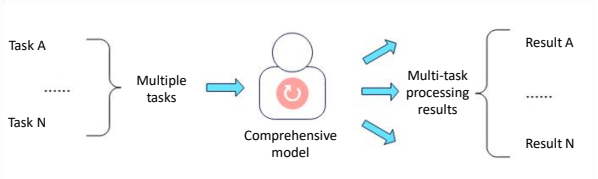
Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Multi-domain Task Processing

- In the past, neural network models were used to cope with single work challenges. Each model was used for only one or several tasks.



- Can one model process tasks in multiple domains?



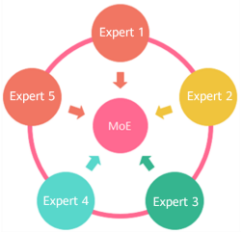
 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

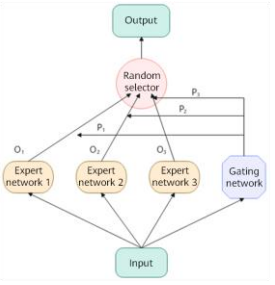
HUAWEI

- In actual data distribution, there are many natural subsets, such as different domains, topics, languages, and modalities. When a single model is used for learning and the model capacity is small, different subsets interfere with model fitting, causing slow model training and difficult generalization.
- For a conventional learning model, training is intended to enable a final model to execute a plurality of tasks in different scenarios. However, when the model updates the weight of a scenario, such training also affect the weights of the model for another scenarios. The mutual impact of the weights is called interference effect. The stronger the **interference effect**, the **slower the learning speed** of the model and **the poorer the generalization** of the model.

MoE

- Mixture-of-Experts (MoE) is a model design strategy that combines multiple models ("experts") to achieve better prediction performance.
- MoE can effectively improve the capacity and efficiency of a foundation model. Generally, MoE uses a gating mechanism along with a set of output mechanisms to combine and balance expert selection, ultimately determining the final prediction for each expert. The expert selection mechanism chooses specific experts for prediction based on the gating mechanism's output.





 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Model size is one of the key factors for improving model performance. This is why today's foundation models are resultful. With a limited computing resource budget, training a larger model with fewer training steps is usually better than training a smaller model with more steps.
- A significant advantage of MoE is that effective pre-training can be performed with far less computing resources than what the Dense model requires. This means that you can significantly scale up your model or dataset under the same computational budget. Especially in the pre-training phase, the hybrid expert model can usually achieve the same quality level more quickly than the Dense model.

Quiz

1. (Single-choice question) Which of the following is not a deep learning neural network? ( )

   A. CNN

   B. RNN

   C. LSTM

   D. Logistic

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 1.D

Quiz

2. (Multi-choice question) Which of the following are components of the CNN? ( )

A. Activation function

B. Convolution kernel

C. Pooling

D. Fully-connected layer

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 2.ABCD

Quiz

3. (True or false) Compared with the RNN, the CNN is more suitable for image recognition. ( )

   A. True

   B. False

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- 3.A

Quiz

4. (True or false) Transformer uses the encoder-decoder architecture. Therefore, it has features similar to Seq2Seq. ( )

   A. True

   B. False

 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- B

Summary

- This chapter discusses the training process of deep neural networks, including multiple network architectures developed from conventional artificial neural networks. Finally, it describes the infrastructure for building large models.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Artificial Intelligence Development Framework



HUAWEI

Foreword

- Recent decades have seen explosive growth in the research and application of deep learning. This has given rise to the third tide of Artificial Intelligence (AI) development, achieving particular success in image recognition, speech recognition and synthesis, autonomous driving, and machine vision. These advances require more advanced algorithms and underlying frameworks.
- Ongoing development in deep learning frameworks has made it easier to use extensive computational resources to train neural network models on large datasets. This section describes the use of AI frameworks and the process of developing AI applications.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completing this course, you will be able to understand:
  - Mainstream development frameworks
  - How AI applications are developed
  - How to use the PyTorch framework

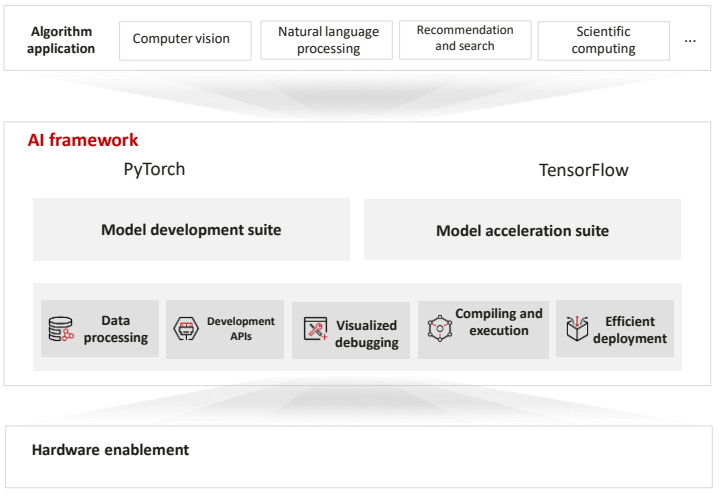
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **AI Framework**
   - Functions of AI Development Frameworks
   - Introduction to Mainstream AI Development Frameworks
2. Basics of AI Development Frameworks
3. PyTorch
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Development Framework



01. Data pre-processing

Pre-processes various data from audio, video, images, and text (data conversion and enhancement).

02. Development APIs

Provides developers with integrated, full-process AI development APIs, accelerating research and development.

03. Debugging and tuning

Provides visualized debugging and tuning capabilities to help view and trace model training.

04. Compiling and execution

Compiles user code and implements high-performance training through full-stack hardware-software synergy.

05. Inference deployment

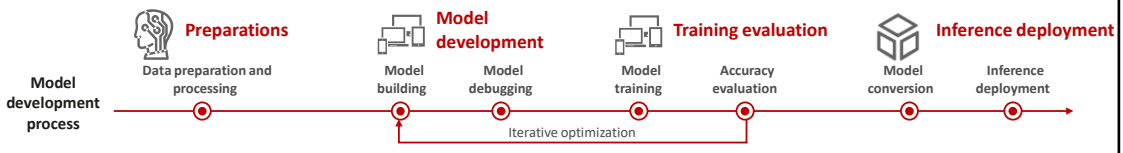
Implements model delivery and computing scheduling, supports quick deployment for industry customers, and maximizes the computing power of AI chips.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The emergence of deep learning frameworks lowers the requirements for developers. Developers no long need to compile code starting from complex neural networks and back-propagation algorithms. Instead, they can use existing models to configure parameters as required, where the model parameters are automatically trained. Moreover, you can add self-defined network layers to the existing models, or select required classifiers and optimization algorithms.

Functions of the AI Development Framework



Accelerates scientific research and development

- Provides easy-to-use and extensive programming APIs
- Allows developers to complete model development with ease
- Unifies data and other APIs, making scientific research and experiments easier

Enables stable and high-performance training

- Enables full-stack software-hardware synergy for high-performance training
- Provides functions such as automatic differentiation and automatic parallelization
- Optimizes model compilation and training

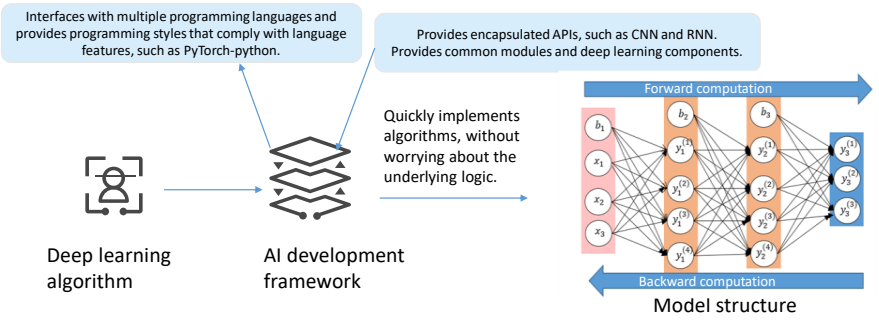
Makes inference deployment easier

- Provides hardware access capabilities
- Simplifies technical details of the underlying hardware
- Solves difficulties in model adaptation and deployment

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Front-end Programming Interface

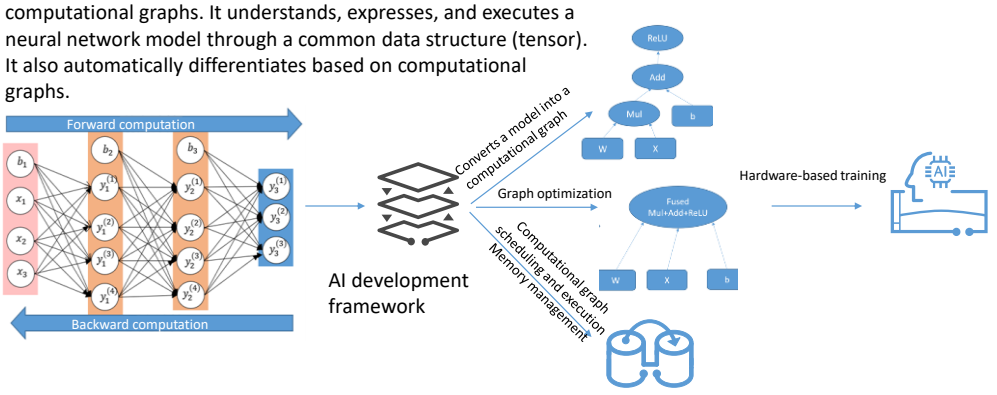
With the AI framework, developers can focus on the service logic of applications without worrying about the underlying mathematical and computing details.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Computing Optimization and Scheduling

The AI framework abstracts model structure computing into computational graphs. It understands, expresses, and executes a neural network model through a common data structure (tensor). It also automatically differentiates based on computational graphs.

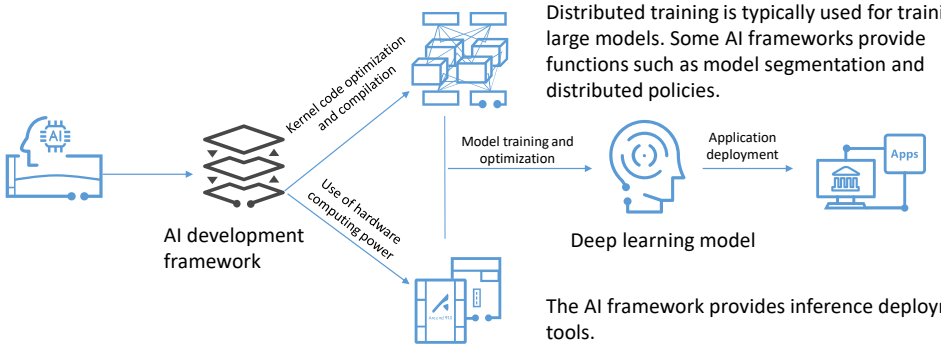


The AI framework usually further optimizes the computational graph to improve the computational efficiency.
It also schedules the graph and manages the memory through the computational graph execution process.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- After the model structure is built, the AI framework converts the model into a computational graph and further optimizes the computational graph before execution.

Model Training and Deployment



Distributed training is typically used for training large models. Some AI frameworks provide functions such as model segmentation and distributed policies.

The AI framework provides inference deployment tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

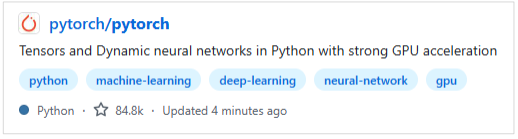
Contents

1. **AI Framework**
   - Functions of AI Development Frameworks
   - Mainstream AI Development Frameworks
2. Basics of AI Development Frameworks
3. PyTorch
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Frameworks

- Today, there are many AI development frameworks, each with its own unique characteristics and serving different domains and scenarios.
- PyTorch is a popular development framework for foundation models.



PyTorch
TensorFlow
PaddlePaddle
JAX
OneFlow
MXNet

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

PyTorch

- PyTorch is a machine learning computing framework released by Meta. It was developed based on Torch, a scientific computing framework supported by many machine learning algorithms. Torch is a tensor operation library similar to NumPy, which is highly flexible. However, NumPy is less popular because it uses the niche programming language Lua. This is why PyTorch was developed.
- PyTorch is a tensor library optimized for deep learning using GPUs and NPUs. It is one of the most common deep learning frameworks.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

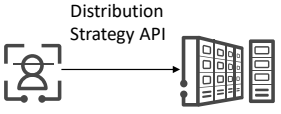
PyTorch Characteristics

- Python first: PyTorch does not simply bind Python to the C++ framework. Instead, it directly supports Python access at a granular level. Developers can use PyTorch as easily as using NumPy or SciPy.
- Dynamic neural network: By default, PyTorch uses dynamic computational graphs. Programs can dynamically build or adjust computational graphs during execution.
- Easy debugging: PyTorch can generate dynamic graphs during execution, and developers can stop the interpreter in the debugger and view the output of a specific node.
- Rich ecosystem: PyTorch has an active community with many documents, tutorials, models, and tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

TensorFlow

- TensorFlow is an end-to-end open-source machine learning platform designed by Google. Currently, the main version is TensorFlow 2.X.
- TensorFlow provides multiple levels of APIs. For example, you can use advanced Keras APIs to quickly build and train models.
- For large deep learning training tasks, you can use the Distribution Strategy API (for distributed training across processing units) to perform distributed training on different hardware configurations without changing the model definition.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

TensorFlow Production Tools

- TensorFlow.js: a library for developing and training machine learning models in JavaScript. It can be deployed on browsers or Node.js.
- TensorFlow Lite: an open-source deep learning framework for on-device inference. It can be used to deploy machine learning models on mobile and IoT devices.
- TensorFlow Extended (TFX): an end-to-end platform for deploying production machine learning pipelines. When you are ready to move trained models from research to production, use TFX to create and manage a production pipeline.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

JAX

- JAX is a high-performance numerical computing library developed by Google. It combines NumPy-like APIs, the automatic differentiation function of Autograd, and the XLA compiler of TensorFlow, implementing efficient computing across CPUs, GPUs, and TPUs.
- Compared with TensorFlow 2.x, it provides:
  - Functional programming APIs.
  - Higher computing efficiency.

```
pip install jax                 # Linux, Windows, and macOS
pip install -U "jax[cuda12]"    # GPU
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Advantages and Disadvantages of JAX

- Advantages:
  - Simplicity and ease of use: JAX is designed to minimize encapsulation and redundant operations, making its APIs simple and easy-to-use.
  - High performance: JAX delivers excellent computing performance through the XLA-based optimization and accelerated execution on GPUs and TPUs.
  - Flexibility: The functional programming style and composable function transformations of JAX make it highly flexible and easy to integrate with other libraries and frameworks.
- Disadvantages:
  - Weak community ecosystem: Compared with deep learning frameworks such as PyTorch, JAX has a relatively small community ecosystem and fewer documentation resources.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

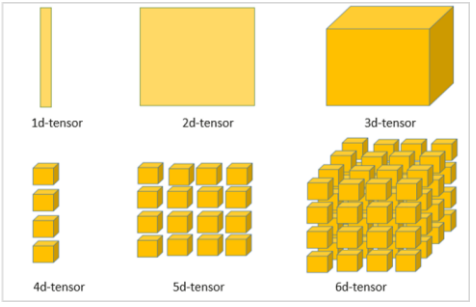
Contents

1. AI Framework
2. **Basics of AI Development Frameworks**
   - Basics
   - Computational Graph
3. PyTorch
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tensor

- A tensor is the most basic data structure in an AI development framework. All data is encapsulated in tensors.
- Definition: a multidimensional array
  - Zero-order tensor: scalar
  - First-order tensor: vector
  - Second-order tensor: matrix



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

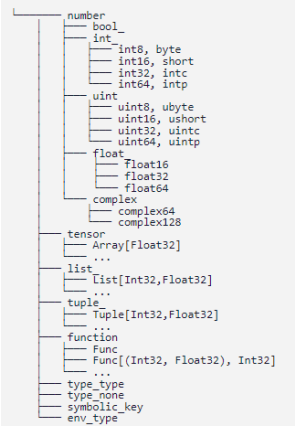
Tensor Attributes

Tensor Attribute	Function
Shape	Size of each tensor dimension, for example, [N,C,H,W].
Rank or dimension (dim)	Number of tensor dimensions; the value is 0 for a scalar and 1 for a vector.
Data type (dtype)	Data type of a tensor, for example, bool, uint8, int16, float32, and float64.
Storage position (device)	Position of the device where the tensor is located, for example, NPU or GPU.
Name	Tensor identifier.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Types

- Currently, AI development frameworks support the following data types: numeric (such as float32, float16, int8, and bf16), bool, and complex. (Some frameworks may also support data types of certain hardware.)
- Data types and programming languages can be used across different AI development frameworks.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Loading

- AI development frameworks provide data loading modules, which can directly load some open-source datasets, such as MNIST, CIFAR-10, CIFAR-100, VOC, COCO and ImageNet.
  - For example, torchvision.datasets.CIFAR10 in PyTorch can directly load CIFAR-10 data. (The data is automatically downloaded if it does not exist locally.)

```
# Download and load the training set (when it is not stored locally).
# Transform is the transformation operation performed on data.
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Format After Loading

- A loaded dataset usually takes the form of a multidimensional tensor.
  - Sequence data is usually a 3-dimensional tensor in the [batch_size, sequence_length, feature_length] format.
    - For example, sentence embedding in NLP is expressed as [batch size, sentence length, word embedding dimension].
  - Image data is usually a 4-dimensional tensor in the [N,H,W,C] format. N indicates the batch data size, H and W indicate the pixel height and width of the image, and C indicates the number of channels.
    - In PyTorch, the default format of received image data tensors is [N,C,H,W].
    - In TensorFlow, the default format of received image tensors is [N,H,W,C].

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

TFRecord

- If a dataset has a lot of data, it can be more efficient to serialize the data and store it in a group of files that can be linearly read. This method is particularly suitable for streaming data over the network. It is also useful for pre-processing when buffering any data.
  - Example: TFRecord in TensorFlow.
  - These data formats reduce the disk I/O and network I/O overhead.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model File

AI development frameworks can execute inference tasks on different hardware platforms using trained models.

Inference model files support two types of data: training parameters and network models.

File Format	Description	Application Scenario
Checkpoint (.ckpt)	It uses the Protocol Buffers format and stores all parameter values on the network by default. You can also configure it to store the model structure and training status.	It is generally used to resume training after a training task is interrupted or to fine-tune a task after training.
ONNX (.onnx)	The Open Neural Network Exchange (ONNX) is a general expression for machine learning models.	It is normally used for model migration between different frameworks or on the inference engine (TensorRT).
bin (.bin)	This is a common binary format used to save and load various models and data.	A PyTorch model needs to be converted into a common binary format.
.pt or .pth	This is the default model file format of PyTorch, which is used to save and load a complete PyTorch model, including the model structure and parameters.	A complete PyTorch model needs to be saved and loaded. For example, the optimal model needs to be saved during training, or the trained model needs to be loaded during deployment.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

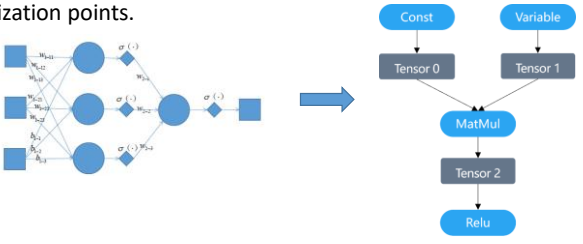
Contents

1. AI Framework
2. **Basics of AI Development Frameworks**
   - Basics
   - Computational Graph
3. PyTorch
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Computational Graph

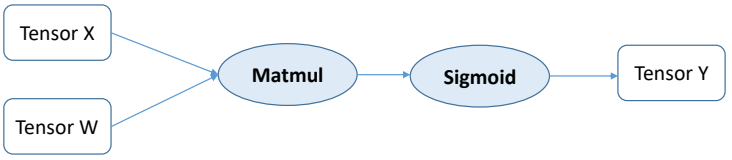
- A computational graph provides an intuitive way to describe the computation process in a deep learning model. Each node represents a computation operation, for example, addition, multiplication, and activation function. An edge represents a flow of data between these operations. This structured representation method makes complex computation easy to understand and analyze.
  - Quickly performs automatic differentiation.
  - Helps find network parallelism and optimization points.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Composition of the Computational Graph

- A computational graph contains tensors (basic data structure) and operators (basic operation units).
- The operator nodes in a graph are connected with directed edges, which indicate the state of each tensor and dependencies between the operations.



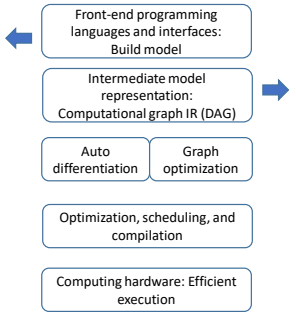
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

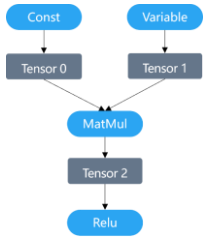
- A node represents a variable or operation. The input nodes represent input data or parameters, and the intermediate node and the output node represent the calculation result.
- The nodes are connected with edges, indicating the data flow or computation dependencies. Each edge usually connects output from one operation to input of another operation.

From Front-end Language to a Computational Graph

- A computational graph is a way to represent mathematical functions with the graph theory language. It is a standardized method used by deep learning frameworks to express neural network models.

```
class _DenseLayer(nn.Module):
   def __init__(
      self, num_input_features: int, growth_rate: int, bn_size:
int, drop_rate: float, memory_efficient: bool = False
   ) -> None:
      super().__init__()
      self.norm1 = nn.Linear()
      self.relu1 = nn.ReLU(inplace=True)
      ...
   def forward(self, x):
      ...
```





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Dynamic/Static Computational Graph

- Currently, mainstream deep learning frameworks can generate two types of computational graphs, static and dynamic.

	Static Graph	Dynamic Graph
Execution	When the program is compiled and executed, the graph structure of the neural network is generated first, followed by the computation operations involved in the graph.	The program is executed in the coding sequence. During forward execution, the reverse execution diagram is dynamically generated based on the backpropagation principle.
Advantages	The compiler uses technologies like graph optimization to further optimize the execution graph, achieving higher execution performance.	The compiler delivers and executes the operators on the neural network one by one, making it easy to build and debug neural network models.
Disadvantages	Not user-friendly for developers and difficult to debug.	Inefficient and difficult to optimize.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Dynamic Computational Graph

- A dynamic computational graph is constructed during running.
- Advantages:
  - Instant execution: Operations are computed in real time. The graph structure can be adjusted anytime as required.
  - Flexibility: It is easy to process various data sizes. It supports complex control flow operations.
  - Easy to debug: Due to on-demand computation, developers can easily use debugging tools of programming languages like Python to check for errors.
  - Research-friendly: Researchers can try different ideas and quickly iterate models during experiments.
  - Intuitiveness: Dynamic computational graphs are easier to understand and use for beginners.
- Disadvantages:
  - The graph may cause extra overhead during running, resulting in lower performance than static computational graphs.
  - Due to the dynamic graph structure, these graphs cannot be optimized as thoroughly as static computational graphs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Using Dynamic Graphs in TensorFlow

```
import tensorflow as tf
import time

# Data initialization
x = tf.Variable(tf.random.normal([512, 1024])) # Batch size 512
W = tf.Variable(tf.random.normal([1024, 1024]))

# Performance test configuration
iterations = 1000

# Execution timing
start_time = time.perf_counter()
for _ in range(iterations):
    _ = tf.matmul(x, W)  # Immediate execution mode
dynamic_duration = time.perf_counter() - start_time

print(f"[Dynamic Graph] {iterations} iterations took {dynamic_duration:.4f} seconds")

# [Dynamic Graph] 1000 iterations took 1.2873 seconds
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Static Computational Graph

- A static computational graph is defined before running.
- Advantages:
  - Predefinition: The entire computational graph needs to be defined before execution.
  - Optimization: Because the graph structure is fixed, the framework can be further optimized, such as memory allocation and parallel computing, to improve the runtime performance.
  - Deployment-friendly: Static computational graphs are suitable for production environments and mobile devices because their runtime performance is usually higher.
  - Portability: Static computational graphs can be exported into a format independent from the framework and can be easily deployed in different environments.
- Disadvantages:
  - These graphs require more complex coding, especially when complex control flows are involved.
  - Because they are defined before running, debugging may be less intuitive than dynamic computational graphs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Using Static Graphs in TensorFlow

```
import tensorflow.compat.v1 as tf
import time
tf.disable_v2_behavior()
# Computational graph construction
x = tf.placeholder(tf.float32, shape=[None, 1024], name='input')
W = tf.Variable(tf.random_normal([1024, 1024]), name='weights')
y = tf.matmul(x, W) # Matrix multiplication

# Performance test configuration
iterations = 1000
input_data = tf.random.normal([512, 1024])  # Batch size 512

# Execution timing
with tf.Session() as sess:
    sess.run(tf.global_variables_initializer())

    start_time = time.perf_counter()
    for _ in range(iterations):
        sess.run(y, feed_dict={x: input_data.eval()})
    static_duration = time.perf_counter() - start_time

print(f"[Static Graph] {iterations} iterations took {static_duration:.4f} seconds")
```

[Static Graph] 1000 iterations took 0.3214 seconds

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Computational Graphs in TensorFlow and PyTorch

PyTorch	TensorFlow
The dynamic graph mode is used by default.	In Tf2.x, the dynamic graph mode is used by default. You can also run tf.enable_eager_execution () to manually enable the dynamic graph mode.
You can use the torch.jit.trace and torch.jit.script functions to improve the efficiency of static graphs.	In Tf1.x, the static graph mode is used by default. In Tf2.x, you can use tf.function to implement static graphs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Framework
2. Basics of AI Development Frameworks
3. **PyTorch**
   - Basic Modules
   - Implementation of the LeNet Network Structure
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common PyTorch Modules

Common PyTorch Module	Function
torch	The core PyTorch module, which provides functions such as tensor operations and linear algebra.
torch.nn	Constructs neural networks and provides the network layer, loss function, and activation function.
torch.utils.data	Provides tools for data loading and pre-processing.
torch.autograd	PyTorch's automatic differentiation engine, which supports gradient calculation and is mainly used for gradient backpropagation after the loss function is obtained.
torch.optim	Provides multiple optimization algorithms for model parameter updates and optimization.
torch.device	Specifies the device on which tensors and models should run, allowing switching between the CPU and the GPU.
torch.distributions	Provides a series of classes to enable PyTorch to sample different distributions and generate computational graphs for the probabilistic sampling process.
torch.jit	PyTorch's instant compiler module, which can convert dynamic graphs into static ones for optimization and serialization.
torch.random	Provides a series of methods to save and set the status of the random number generator, which helps debug the structure and performance of neural networks.
torch.onnx	Defines the deep learning model description files exported and loaded by PyTorch in the ONNX format, enabling model exchanges with other deep learning frameworks.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

torchvision

torchvision is a library specifically designed for computer vision in the PyTorch ecosystem.

torchvision Module	Function
torchvision.datasets	Provides multiple widely used image and video datasets, such as MNIST, CIFAR10/100, Fashion-MNIST, ImageNet and COCO. These datasets can be directly used to train and evaluate visual models such as image classification, object detection, and semantic segmentation.
torchvision.transforms	Supports various data augmentation and pre-processing operations, including cropping, rotation, flipping, normalization, resizing, and color conversion. These operations are critical to training robust deep learning models, and can improve the quality and diversity of training data.
torchvision.models	Encapsulates many pre-trained model structures, such as AlexNet, VGG, ResNet, Inception series, DenseNet, and SqueezeNet, as well as models for object detection and semantic segmentation tasks. These models can be directly loaded for transfer learning or extended as the basic network structure.
torchvision.utils	Provides a number of practical methods, for example, saving tensors as image files and creating image grids to visualize multiple samples, to help visualize and analyze experiment results.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Loading

- PyTorch data is loaded through Dataset and DataLoader. Dataset defines the data format and how data is transformed. DataLoader continuously and interactively reads batch data.
  - torch.utils.data.Dataset is the base class for constructing datasets. By inheriting this class and rewriting the __init__, __len__, and __getitem__ methods, you can provide your own data and labels.
  - torch.utils.data.DataLoader is an iterator used to load datasets. It provides functions such as batch loading, data disordering, and multi-process data loading. You need to pass a Dataset object when using it.

```
# Define a Dataset. The following uses the open-source data in PyTorch as an example.
trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
# Use DataLoader to load the dataset.
trainloader = DataLoader(trainset, batch_size=64, shuffle=True)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Other Methods of Constructing Datasets

- torchvision.datasets.ImageFolder: a class used to load image datasets from a folder. It assumes that all files are stored in folders, and files in each folder belong to the same category.
- Other open-source datasets:

API Name	Description
torchvision.datasets.MNIST	A class used to load the MNIST handwritten digit dataset. You only need to specify the data's root directory and some other parameters to automatically download the dataset.
torchvision.datasets.CIFAR10	CIFAR10 is another class in the torchvision library for loading the standard dataset (CIFAR-10). This dataset contains 60,000 32 x 32 color images divided into 10 categories. Each category includes 6000 images.
torchvision.datasets.CIFAR100	Similar to CIFAR10, CIFAR100 includes 100 categories, and each category includes 600 images. This dataset is also used for image classification tasks.
torchvision.datasets.ImageNet	ImageNet is a large image dataset containing millions of images in thousands of categories.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Construction

- torch.nn.Module is a model construction class in PyTorch. It is the base class of all neural network modules and can be inherited to define models.
- When defining a model, reload the __init__ and forward functions, which are used to create model parameters and define forward computation (forward propagation), respectively.

```
import torch
from torch import nn

class MLP(nn.Module):
 # Declare a layer with model parameters. Here, two fully connected layers are
declared.
 def __init__(self, **kwargs):

   super(MLP, self).__init__(**kwargs)
   self.hidden = nn.Linear(784, 256)
   self.act = nn.ReLU()
   self.output = nn.Linear(256,10)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Forward Computation

```
class MLP(nn.Module):
  # Declare a layer with model parameters.
  def __init__(self, **kwargs):
      ...

 # Forward computation of the model, that is, how to compute and
return the required model output based on the input x.
  def forward(self, x):
      o = self.act(self.hidden(x))
      return self.output(o)
```

- You do not need to define a backpropagation function. The system automatically generates the backward function required for backpropagation by automatically calculating the gradient.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Two-dimensional Convolutional Layer

- torch.nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True, padding_mode='zeros', device=None, dtype=None):
  - A class for implementing the two-dimensional convolution operation.
  - in_channels: number of image input channels. For example, for a color RGB image, the value of in_channels is 3.
  - out_channels: number of output channels (number of convolution kernels) after convolution.
  - kernel_size: size of the convolution kernel.
  - stride: convolution stride.
  - padding: number of zero padding layers.

```
# Define a two-dimensional convolutional layer
conv_layer = nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, bias=True)

 # Input data (Assume that the number of input images is batch_size, the number of input channels is in_channels, and the size is height x width.)
input_tensor = torch.randn(batch_size, in_channels, height, width)

# Pass through the convolutional layer
output_tensor = conv_layer(input_tensor)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Max Pooling

- torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False):
  - Implements two-dimensional max pooling.
  - kernel_size (int or tuple): size of the pooling window.
  - stride (int or tuple, optional): stride during pooling. The default value is kernel_size.
  - padding (int or tuple, optional): number of zero padding layers on each edge of the input feature map. The default value is 0.

```
conv_layer = nn.Conv2d(in_channels, out_channels, kernel_size, stride=1, padding=0, bias=True)

 # Input data (Assume that the number of input images is batch_size, the number of input channels is in_channels, and the size is height x width.)
input_tensor = torch.randn(batch_size, in_channels, height, width)

# Pass through the convolutional layer.
output_tensor = conv_layer(input_tensor)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recurrent Neural Network

- torch.nn.RNN(input_size, hidden_size, num_layers = 1, nonlinearity = 'tanh', bias = True, batch_first = False , dropout = 0.0, bidirectional = False, device = None, dtype = None):
  - Defines the basic recurrent neural network (RNN).
  - input_size: dimension of the input feature.
  - hidden_size: dimension of the hidden layer feature.
  - num_layers: number of RNN layers.
  - bias: Indicates whether to use the bias.
  - nonlinearity: nonlinear activation function. The value is **tanh** or **relu**. The default value is **tanh**.
  - batch_first: If the value is **True**, the first dimension of the input and output is the batch size.

```
# Define the RNN model.
rnn = nn.RNN(input_size=10, hidden_size=20, num_layers=1,
batch_first=True)

# Create input data.
inputs = torch.randn(5, 3, 10) # The sequence length is 5, batc_size is
3, and the input feature dimension is 10.
h0 = torch.zeros(1, 3, 20) # Initially the model is in hidden state, and
the shape is [num_layers, batch_size, hidden_size].
# Forward Propagation.
output, hn = rnn(inputs, h0)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

PyTorch Activation Functions

- Use the activation function class in the nn module and the activation functions in nn.functional.

Activation Function	Calling Method	Calling Method Description

Sigmoid	nn.Sigmoid()
	F.sigmoid(input)
		The following uses Sigmoid as an example:
		Use nn.Sigmoid () to construct the Sigmoid layer (instantiated object) and then call this layer.
		Or, use F.sigmoid (input) to directly apply the Sigmoid function to the input.
		
		import torch
		import torch.nn as nn
		import torch.nn.functional as F
		# Create an input tensor.
		input_tensor = torch.tensor([[-1.0, 0.0, 1.0], [-2.0, -1.0, 0.0]])
		# Create a Sigmoid layer instance.
		sigmoid_layer = nn.Sigmoid()
		
		# Transfer the input data to the Sigmoid layer.
		output_tensor = sigmoid_layer(input_tensor)
		
		# Directly apply the Sigmoid activation function to the input data.
		output_tensor = F.sigmoid(input_tensor)

Tanh	nn.Tanh() F.tanh(input)	
ReLU	nn.ReLU() F.relu(input)	
LeakyReLU	nn.LeakyReLU(negative_slope=0.01) F.leaky_relu(input, negative_slope=0.01)	
ELU	nn.ELU() F.elu(input)	
PReLU	nn.PReLU() F.prelu(input)	
Softmax	nn.Softmax(dim=None) F.softmax(input, dim=None)	

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Loss Functions

- PyTorch provides multiple loss functions.

Function Name	Calling Method	Description
L1 loss (average absolute error)	torch.nn.L1Loss(reduction='mean')	Calculates the average of the absolute differences between the predicted value and the actual value.
Mean square error loss	torch.nn.MSELoss(reduction='mean')	Calculates the average square of the differences between the predicted value and the actual value.
Cross-entropy loss	torch.nn.CrossEntropyLoss(weight=None, reduction='mean')	Combines LogSoftMax and NLLLoss for multiclass classification.
Binary cross-entropy loss	torch.nn.BCELoss(weight=None, reduction='mean')	Calculates the binary cross entropy between the target value and the predicted value for binary classification.
Connection timing classification loss (CTCLoss)	torch.nn.CTCLoss(blank=0, reduction='mean')	Automatically aligns unaligned data for serialized data training.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

PyTorch Optimizers

Optimizer Name	Calling Method	Description
SGD	torch.optim.SGD(params, lr=0.01, momentum=0, dampening=0, weight_decay=0, nesterov=False)	params: parameters to be optimized; lr: learning rate; momentum: momentum factor, which is used to accelerate the convergence of SGD in the related direction and suppress oscillation; dampening: damping coefficient of the momentum, which is used to reduce the amplitude of the momentum update; weight_decay: weight decay coefficient, which is used for L2 regularization
Adam	torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0, amsgrad=False)	params: parameters to be optimized; lr: learning rate; betas: coefficient used to calculate the running average of gradients and gradient squares; weight_decay: weight decay coefficient
AdaGrad	torch.optim.Adagrad(params, lr=0.01, lr_decay=0, weight_decay=0, initial_accumulator_value=0, eps=1e-10)	params: parameters to be optimized; lr: learning rate; lr_decay: learning rate decay; weight_decay: weight decay coefficient; initial_accumulator_value: initial value of the accumulator
AdamW	torch.optim.AdamW(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08, weight_decay=0.01, amsgrad=False)	params: parameters to be optimized; lr: learning rate; betas: coefficient used to calculate the running average of gradients and gradient squares; eps: item added to the denominator to improve the stability of value calculation; weight_decay: weight decay coefficient; amsgrad: indicates whether to use the AMSGrad variant
RMSProp	torch.optim.RMSprop(params, lr=0.001, alpha=0.99, eps=1e-08, weight_decay=0, momentum=0, centered=False)	params: parameters to be optimized; lr: learning rate; alpha: coefficient of the smoothing term; weight_decay: weight decay coefficient; momentum: momentum factor; centered: indicates whether to calculate the centralized RMSprop

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Adding Loss Functions and Optimizers for Training

```
# Instantiate a model structure.
model = MLP()
# Create a loss function.
criterion = nn.CrossEntropyLoss()
# Create an optimizer.
optimizer = optim.SGD(model.parameters(), lr=0.01,momentum=0.9)
# Input data into the model.
output = model(trainset)
# Calculate the loss.
loss = criterion(output, target)
loss.backward()
optimizer.step()
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Framework
2. Basics of AI Development Frameworks
3. **PyTorch**
   - Basic Modules
   - Implementation of the LeNet Network Structure
4. AI Application Development Process

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Building a Complete Neural Network (1)

- Use PyTorch to build a complete network LeNet and use MNIST for training.
  - Build a LeNet network.
    - It has two convolutional layers. The first one uses six 5x5 convolution kernels with a stride of 1 and a padding of 2, and outputs six feature maps. The second one uses 16 5x5 convolution kernels with a stride of 1 and no padding, and outputs 16 feature maps.
    - The average pooling layer is used to reduce the dimensions of the feature maps.
    - ReLU is used after the convolutional layers, and sigmoid or softmax (at the output layer) is used after the fully connected layer.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

# Define the LeNet network structure.
class LeNet(nn.Module):
 def __init__(self):
     super(LeNet, self).__init__()
     # Define a two-dimensional convolutional layer.
     self.conv1 = nn.Conv2d(1, 6, kernel_size=5, stride=1, padding=2)
     self.conv2 = nn.Conv2d(6, 16, kernel_size=5, stride=1)
     # Define the fully connected layer.
     self.fc1 = nn.Linear(16*5*5, 120)
     self.fc2 = nn.Linear(120, 84)
     self.fc3 = nn.Linear(84, 10)
     # Define the pooling layer.
     self.pool = nn.AvgPool2d(kernel_size=2, stride=2)
     # Activation function
     self.relu = nn.ReLU()
     self.sigmoid = nn.Sigmoid()
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Building a Complete Neural Network (2)

Conv1 (the first convolutional layer)
| - - Six 5 x 5 convolution kernels with a stride of 1
| - - Output feature maps: 28 x 28 x 6
Pool
| - - 2 x 2 pooling window, with a stride of 2
| - - Output feature maps: 14 x 14 x 6
Conv2 (the second convolutional layer)
| - - 16 5 x 5 convolution kernels, with a stride of 1
| - - Output feature maps: 10 x 10 x 16
Pool
| - - 2 x 2 pooling window, with a stride of 2
| - - Output feature maps: 5 x 5 x 16
Conv3 (the third convolutional layer, which can be considered a fully connected layer)
| - - 120 5 x 5 convolution kernels, with a stride of 1
| - - Output feature maps: 1 x 1 x 120 (full connection)
FC1 (the first fully connected layer)
| - - Input: 120 neurons
| - - Output: 84 neurons
FC2 (the second fully connected layer/output layer)
| - - Input: 84 neurons
| - - Output: 10 neurons (corresponding to 10 numerical categories)
OUTPUT (output layer)

Define the LeNet forward computation process.

```
# Define the LeNet network structure.
class LeNet(nn.Module):
    def __init__(self):
        ...

    def forward(self, x):
        x = self.conv1(x)
        x = self.relu(x)
        x = self.pool(x)
        x = self.conv2(x)
        x = self.sigmoid(x)
        x = self.pool(x)
        x = x.view(-1, 16*5*5)
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        x = self.relu(x)
        x = self.fc3(x)
        return x
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Building a Complete Neural Network (3)

- Define the dataset with data from the open-source dataset MNIST.
- Define the data pre-processing mode using transforms.Compose. Then, use the transform parameter to apply it to the dataset.

```
# Data preprocessing
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

# Load the MNIST dataset.
trainset = torchvision.datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

testset = torchvision.datasets.MNIST(root='./data', train=False, download=True,
transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=1000, shuffle=False)
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Building a Complete Neural Network (4)

- Initialize the network, loss function, and optimizer.
- Use the for loop for training.
  - Conduct a test after the training is complete. The testing process is not elaborated here.

```
# Initialize the network, loss function, and optimizer.
net = LeNet()
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
# Train the network.
for epoch in range(10):  # Train 10 epochs.
    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        inputs, labels = data

        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        if i % 100 == 99:  # Print the loss every 100 batches.
            print(f'[Epoch {epoch+1}, Batch {i+1}] loss: {running_loss/100:.3f}')
            running_loss = 0.0
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Building a Complete Neural Network (5)

- Save the trained model.
  - In PyTorch, saving and loading a model involves two main parts: the model's state dictionary and architecture. The model's state dictionary contains model parameters (such as weights and biases), and the model architecture defines the model's hierarchy.

```
# Save the model parameters.
torch.save(net.state_dict(), 'lenet_mnist.pth')

# Re-instantiate the model as a model object when loading the model.
net = LeNet()
net.load_state_dict(torch.load('lenet_mnist.pth'))
net.eval () # Switch to the eval mode if inference is required.
```

```
# Save the entire model.
torch.save(net, 'lenet_mnist.pth')

# Load the model.
net = torch.load('lenet_mnist.pth')
model.eval () # Switch to the eval mode if inference is required.
```

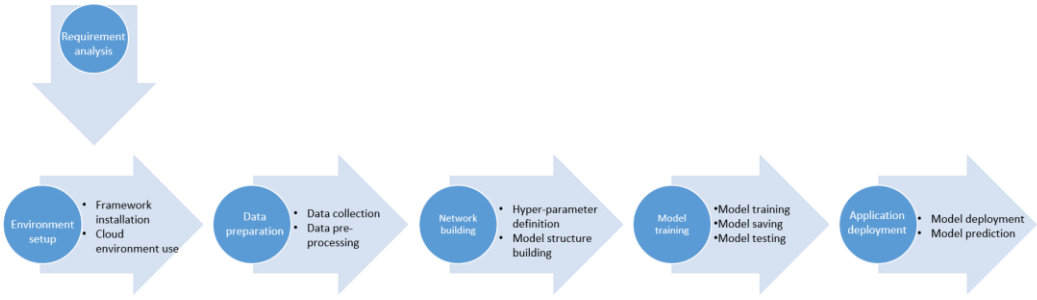

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Framework
2. Basics of AI Development Frameworks
3. PyTorch
4. **AI Application Development Process**

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Application Development Process



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

ResNet-50 Application for Image Classification

- Image classification is the most basic computer vision application and falls into the supervised learning category. For example, the application should determine the type of a given image, such as a cat, a dog, an airplane, or a car. Here we introduce how the application uses the ResNet-50 network to classify flowers in a flower dataset.

Which flowers are these?





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Environment Setup

- Choose a proper environment (Python 3 in this example), obtain the command, and install it according to the guide.
  - The following uses PyTorch as an example.

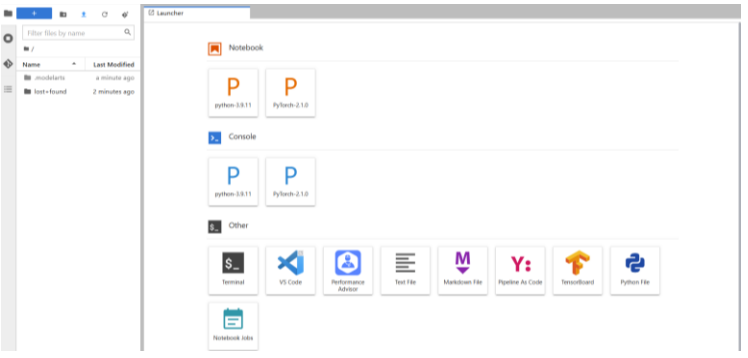


https://PyTorch.org/get-started/locally/

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Cloud Environment Preparation (Optional)

- If no development environment is available, use ModelArts for development, model training, or deployment.
  - ModelArts helps users quickly create and deploy models and manage the entire AI workflow.

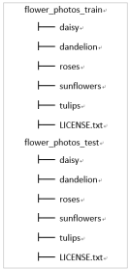


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

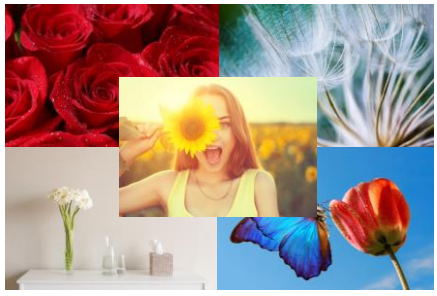
- Compared with a local environment, cloud environment provides plenty of computing power and storage space, making it a good choice for development.

Data Preparation

- The flower image dataset is used as the training data. It is an open-source dataset and contains a total of 3670 images of five flower types: daisies (633 images), dandelions (898 images), roses (641 images), sunflowers (699 images), and tulips (799 images). The images are stored in five folders. The file structure consists of two parts, flower_photos_train and flower_photos_test.



File structure



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Pre-processing

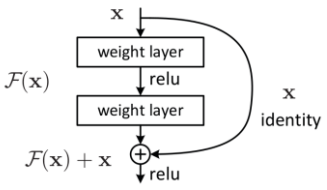
- To improve the model's accuracy and ensure the generalization capability, you need to perform data augmentation and standardization before using the data to train the model.
- Load and process the dataset, including:
  - Reading the dataset
  - Defining parameters required for data augmentation and processing
  - Generating data augmentation operations according to the parameters
  - Processing the generated dataset

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- This application uses open-source datasets, so they do not need to be processed. If collected service data is used, you need to clean and sort the data.

Introduction to ResNet

- Kaiming He from Microsoft Labs proposed ResNet50 in 2015 and won the first place in the ILSVRC 2015 image classification competition.
- The Residual Network is a ResNet highlight. It effectively mitigates degradation and enables deeper network structure design.



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
		3×3 max pool, stride 2				
conv2_x	56×56	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Training

- Start model training after pre-processing the data and building the network model.
- There are two ways to train a model:
  - Start training from scratch based on the dataset (for PyTorch experiments).
  - Fine-tune the dataset on a trained model.
    - Pre-trained model: ResNet model file trained on the ImageNet dataset.
    - Modify the last-layer parameters of the pre-trained model. (The pre-trained model is the 1001 classification task trained on the ImageNet dataset, and the task is the 5 classification task for implementing the flowers dataset.)
    - Perform training based on the training set.
- Parameter tuning: You can adjust different hyper-parameter combinations during training.
- For details about the code, see the lab guide.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

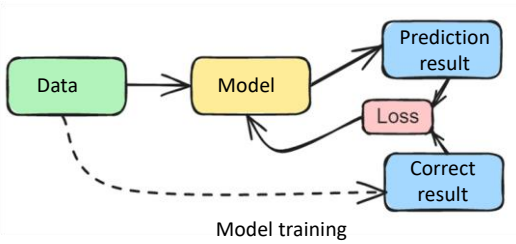
Model Loading and Prediction

- Test the test data based on the trained model weight.
  - After PyTorch training is complete, perform the inference test (torch.no_grad) when gradient calculation is disabled.
    - In the case of inference after deployment, you need to enable the eval mode (model.eval ()).
  - Load the weight file and perform the prediction.
    - To load the model weight, you need to create the model's instance first.

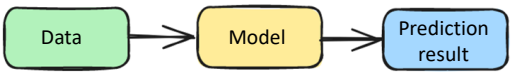
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Deployment

- The figure on the right shows the simplified model training and inference process. Note that model inference does not require updating the model weight parameters. As such, the hardware requirements are lower than those for training.
- Models are usually deployed on dedicated inference servers. After optimization, such as model compression, models can also be deployed on devices such as mobile phones and cameras.



Model training



Model inference

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Compression

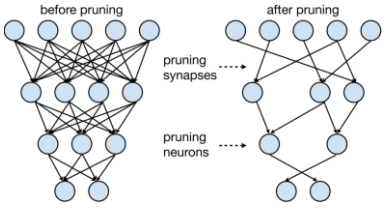
- Increasing the number of model parameters has the following impact on the model.
  - Inference performance
    - When the hardware memory access bandwidth is limited, the data transfer overhead increases with the increased weight.
    - When there is not enough video RAM capacity, the model can only be deployed on the CPU, lowering the inference performance.
  - Deployment cost
    - A larger weight requires more storage capacity, computing resource overhead, and video RAM capacity. As such, more devices are required for model deployment.
- Currently, devices with large video RAM and high memory access bandwidth are expensive. As such, model compression is essential for deploying and promoting AI applications on consumer devices.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- As the batch size and sequence length increase, the KV cache also increases, causing great pressure on memory access.

Model Compression Method - Model Pruning

Pruning is an important model compression technology. The basic idea is to remove less important weights and branches from a model and sparse the network structure to obtain a model with fewer parameters. However, pruning may also cause model performance deterioration. As such, you must find a balance between model size and performance. Neuron connections in a neural network are mathematically represented as a weight matrix. This means that pruning changes some elements in the weight matrix to zero elements. The following figure shows the pruning process. The purpose is to remove less important synapses or neurons.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

Model Compression Method - Knowledge Distillation

- Knowledge distillation is a model compression technology based on transfer learning.
  - A large complex network usually provides good performance, but also contains a lot of redundant information, which increases the computational complexity and resource consumption. Distilling extracts useful information from a complex network and migrates it to a smaller network. After learning, the small network can deliver similar performance to the large complex network while requiring fewer computing resources. The complex network can be seen as a teacher, and the small network as a student.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common Deployment Tool

- Llama.cpp
  - Implemented with C++ code; helps implement LLM inference with minimal setup and superb performance on various hardware devices (on-premises and on the cloud) without the need for an AI framework.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Summary

- This section describes AI development frameworks, including the basics of common AI frameworks, and PyTorch framework. It also touches on the process of developing AI applications.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Multiple-choice question) Which of the following hardware devices can be used for PyTorch training?
   A. CPU
   B. GPU
   C. TensorFlow
   D. CUDA

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Answer: AB

Recommendations

- https://pytorch.org/

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

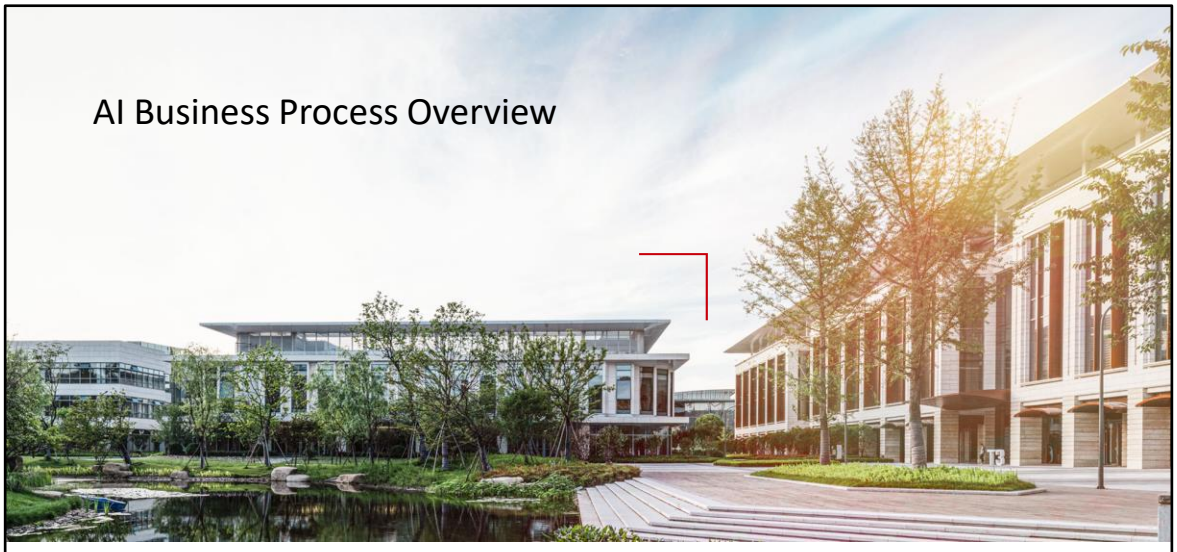
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright©2025 Huawei Technologies Co., Ltd.
All Rights Reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



AI Business Process Overview



HUAWEI

Foreword

- This chapter describes the AI business construction process, how to train a large model (the entire training and fine-tuning process), and how to efficiently use a large model.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon completion of this course, you will be able to:
  - Understand the overall data development process o the AI business.
  - Understand the overall process of foundation model training.
  - Understand how to efficiently use large models

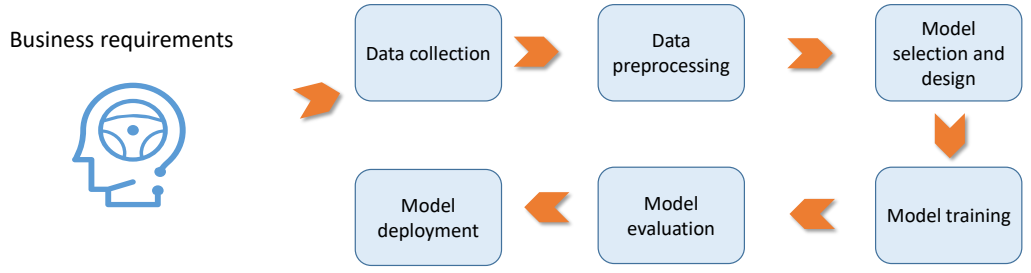
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **AI Business Process**
2. Large Model Business Process
3. Use of Large Models and Prompt Engineering

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI Business Development Process



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- https://support.huaweicloud.com/function-modelarts/index.html#

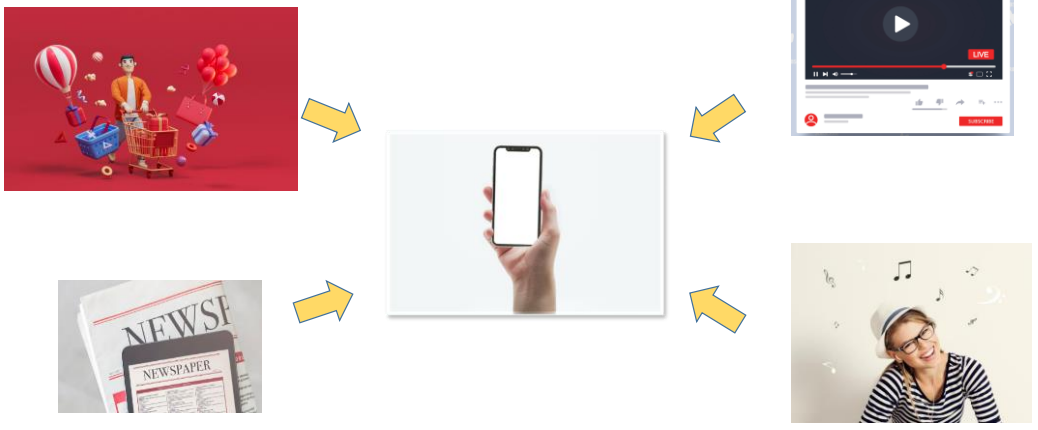
Requirement Analysis



- What is the purpose?
- What are the problems to be solved?

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Recommendation Algorithm Field



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- The recommendation system algorithms include **collaborative filtering, content-based recommendation, knowledge-based recommendation, and hybrid recommendation**.
- **E-commerce product recommendation**: product recommendation on AI algorithms.
- **Personalized news push**: news content push based on user interest. News reading habits and user profiles are crucial. Common methods include content-based recommendations, collaborative filtering, and deep learning sequence models.
- **Video and music content recommendation**: video recommendation systems on platforms such as YouTube and iQIYI. Video metadata and content features are considered along with user behavior data. The algorithms include deep learning embedding models and multi-modal fusion models, such as YouTube DNN and DeepFM.

Computer Vision (CV)









Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Security**
- **Target recognition**, including identity authentication, social media filtering, and security management. Data sets such as Labeled Faces in the Wild (LFW) and MS-Celeb-1M are used. Algorithms include convolutional neural network (CNN), FaceNet, and ArcFace.
- **image recognition and classification**, including object recognition and medical image analysis. Data sets such as ImageNet, Common Objects in Context (COCO), and ChestX-ray14 are used to train and test models. Deep learning frameworks such as ResNet, VGG, and Inception series models are used.
- **Autonomous driving**, including road sign recognition, pedestrian detection, and obstacle avoidance. KITTI, Cityscapes, and Waymo Open Dataset are popular for autonomous driving. These datasets use object detection methods like YOLO, SSD, and Faster R-CNN. They also use 3D sensing methods like PointNet and RangeNet++, which work with lidar data.
- **Image generation**

Natural Language Processing (NLP)



9 Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Chatbot**, including customer service assistant and smart speaker. The training dataset may contain many dialog records and encyclopedia entries. Common algorithms are the sequence-to-sequence (Seq2Seq) model, Transformer architectures like BERT and GPT, and reinforcement learning methods.
- **Sentiment analysis**, including analysis of the sentiment tendency of user comments and social media posts. The IMDb movie review dataset and Twitter sentiment analysis dataset are commonly used benchmark datasets. Common algorithms include the bag of words (BOW) model, TF-IDF, and deep learning models such as TextCNN, LSTM, and bidirectional LSTM.
- **Machine translation**, that is, real-time translation between different languages. The Workshop on Machine Translation (WMT) provides multiple language pair datasets for training translation models. Google's transformer model and its variants, like Transformer-XL and BERT-based architectures, are the mainstream algorithms in this field.
- **Speech recognition and generation**, including converting speech into text or text into speech. They are the basis for voice assistants and automatic speech translation systems.

Data Preparation (1)

- Data collection
  - Recommendation system: Collect users' historical behavior data, which may include users' purchase history, browsing records, and rating data. Public datasets can be used, for example, the MovieLens movie rating dataset.
  - CV: Obtain diversified image or video data in related fields to cover various objects, scenarios, and conditions that the model should recognize. Consider using public datasets like ImageNet, COCO, or PASCAL VOC as the starting point.
  - NLP: Use crawlers to obtain related text from the Internet, or obtain text from public datasets, books, news, and social media platforms. In addition, questionnaires are also an effective way to collect specific data.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Preparation (2)

- Data cleansing
  - This process removes incorrect and unnecessary information, such as duplicate images, irrelevant images, duplicate text items, spelling errors, and user data that is duplicated or abnormal.
- (Optional) Feature engineering
  - Extraction of useful features from raw data might be needed based on the requirements of the recommendation system. Such features include user personal information, object attributes, and user behaviors. The purpose of feature engineering is to create features that can help the model better understand data, improving recommendation accuracy.
  - Currently, CV and NLP do not require this operation.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Preparation (3)

- Data labeling:
  - For supervised learning tasks, you need to label each data item with the correct label. This can be performed manually or using automated tools. The quality and accuracy of labeling are crucial to the training effect of the model.
- Dataset splitting:
  - Split the cleansed and labeled data into training, validation, and test sets. The training set is used to train the model, the validation set is used to adjust model parameters and select the best model, and the test set is used to evaluate the model performance.
- Preprocessing:
  - Preprocess the data, such as normalization and encoding, to make it easier for the model to process.
- Data analysis and evaluation:
  - After creating a dataset, perform thorough data analysis to ensure its quality, diversity, and balance. In addition, you need to evaluate whether the dataset meets the expected objectives and prepare to support model training.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

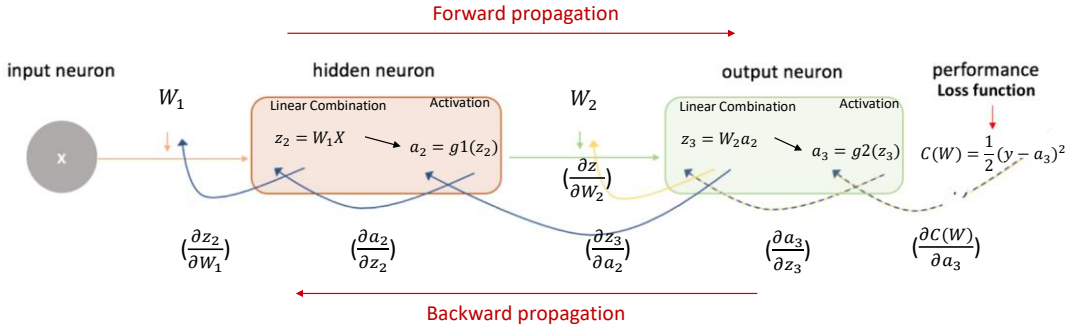
Model Selection and Design

- Select a proper deep learning model architecture based on the task type and data characteristics. For example:
  - Image recognition task: CNN, such as VGG, ResNet, and YOLO
  - Text processing task: recurrent neural network (RNN), long short term memory (LSTM), gated recurrent unit (GRU), and Transformer (BERT and GPT)
  - Deep neural network-based recommendation algorithm and attention mechanism-based recommendation algorithm
- Design the model hierarchy, including the input layer, hidden layer (convolutional layer, recurrent layer, self-attention layer, and so on), output layer, and auxiliary layers such as the potential pooling layer and normalization layer.
- Hyperparameter selection:
  - Determine the hyperparameters of the model, such as the learning rate, batch size, regularization strength, number of layers, number of nodes, and dropout ratio.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Training

- Forward propagation: Calculates the loss function, that is, the difference between the prediction and the actual label.
- Backward propagation: Optimizes the parameters in the model based on the loss function, so that the next prediction is as close as possible to the actual value.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Evaluation (1)

- Classification task evaluation metrics
  - Accuracy: It is one of the most basic evaluation metrics. It measures the ratio of the number of samples that are correctly predicted by the model to the total number of samples.
  - Precision: In a classification task, precision is the proportion of true positive predictions among all positive predictions made by the model.
  - Recall: For binary classification or multi-class classification, precision is the proportion of positive samples predicted by the model, reflecting the model's ability to predict positive samples.
  - F1-Score: The F1-Score is the harmonic mean of precision and recall. It combines precision and recall. A higher F1 value indicates that the model achieves a better balance between precision and recall.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

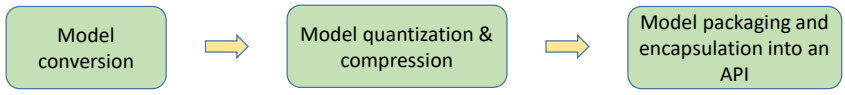
Model Evaluation (2)

- Regression task evaluation metrics
  - Mean squared error (MSE): It is the average of the squared differences between the predicted value and the actual value. It is used to measure the prediction error.
  - Root mean squared error (RMSE): It is the square root of MSE. The unit is the same as that of the original observation value, which more intuitively reflects the difference between the predicted value and the actual value.
  - Mean absolute error (MAE): It is the average of the absolute differences between the predicted value and the actual value. Compared with MSE, MAE is more robust to outliers.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Model Deployment

- Model conversion to the middleware format
- Quantization: FP32 to FP16 and INT8
- Model encapsulation: Encapsulate the model as an API for rollout



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

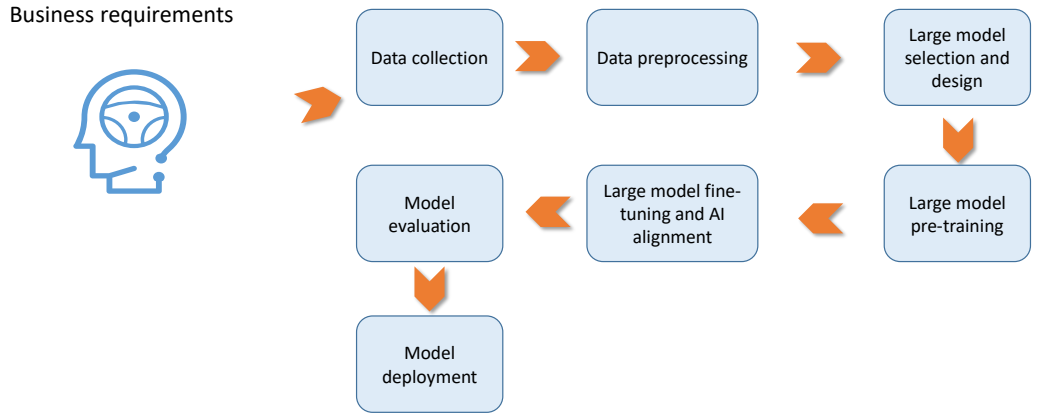
- The trained model file format in PyTorch is PTH, while ONNX is an exchange format developed by AI companies.

Contents

1. AI Business Process
2. **Large Model Business Process**
   - Pre-training
   - Model Fine-tuning
   - RHLF
   - Model Evaluation
3. Use of Large Models and Prompt Engineering

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Large Model Business Development Process



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- https://support.huaweicloud.com/intl/en-us/function-modelarts/index.html

Large Models Are More Complex with More Parameters

- Large models often have billions or even more parameters and can be hundreds of gigabytes or larger.
  - Large models based on Transformer use the self-attention mechanism, which brings a large number of parameters. The self-attention mechanism allows the model to independently consider each position in the input sequence and flexibly capture long-distance dependencies, which further increases the demand for model parameters.
  - For complex, high-dimensional data distribution, more parameters help the model fit better, reduce bias, and improve training performance. In particular, when processing tasks such as natural language, the model needs enough capacity to understand and generate various sentence structures and meanings, so as to better cope with the richness and complexity of language.
- For example, it is estimated based on documentation that GPT-3 has about 175 billion parameters. LLaMA2 by Meta AI comes in three sizes: 7 billion (7B), 13 billion (13B), and 70 billion (70B) parameters.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Large Models Require a Large Data Volume

- Large models have many more parameters than traditional ones, so they need bigger training datasets. Learning from more data helps these models improve.
- The following table shows the training data distribution of several typical large models.

Data		GPT3-175B	LLaMA-65B	GLM-130B
Data description	Data source	CommonCrawl Books1 and 2 WebText2 Wikipedia	CommonCrawl C4 Github Wikipedia Books ArXiv StackExchange	Pile, Chinese Wudao, Corpora ...
Data processing	Filtering			
	Deduplication			
Training data	Ratio	60%, 22%, 8%, 8%, and 3%	67%, 15%, 4.5%, 4.5%, 2.5%, and 2.0%	.....
	Tokens	300B	1.4T	400B

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- Explain that datasets need to be cleansed to remove errors and duplicates. The original GPT3 data was 45 TB, but only 570 GB remains after filtering. The ratio of used data varies according to the data quality.

Diversity of Large Model Training Data

- Training large models needs large-scale, high-quality, and multimodal datasets. This data typically needs to be collected from various domains and sources, and can be texts, images, audio, and videos.



E2E model development pipeline

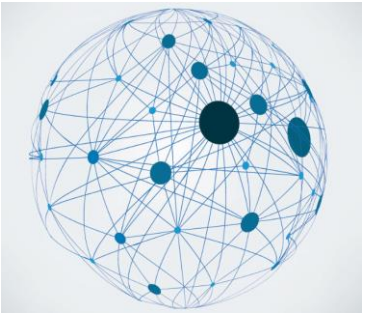
An E2E tool chain for model development, training, and inference enables seamless collaboration of DataOps, MLOps, and DevOps, improving development efficiency by 50%.











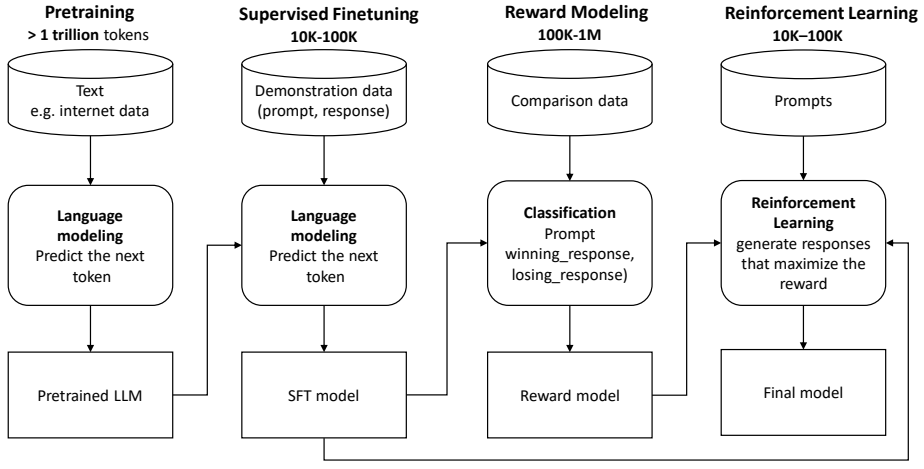
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Large Models Have Better Capabilities

- **Better learning capability**: Large models have more parameters and a more complex structure, which allows them to fit data better and learn more effectively. This allows them to process bigger datasets and find deeper patterns, leading to better performance on many tasks.
- **Better generalization capability**: Large models generalize better, performing well on new data. This makes them adaptable for real-world uses and less likely to overfit.
- **Stronger multi-task processing capability**: Large models can handle multiple tasks at once, eliminating the need for separate models for each task. This improves flexibility and efficiency and lowers maintenance costs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

LLM Training Process



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- During pre-training, we use huge amounts of data from sources like web pages, Wikipedia, books, GitHub, papers, and Q&A sites. This data contains hundreds of billions to trillions of words. We train the model using a supercomputer with thousands of powerful GPUs and fast networks. It takes dozens of days to complete this training and build the basic language model (base model). The Basic Model develops the capability to model long texts, enabling the model to generate language. Based on the input prompt, the model can produce text completions.

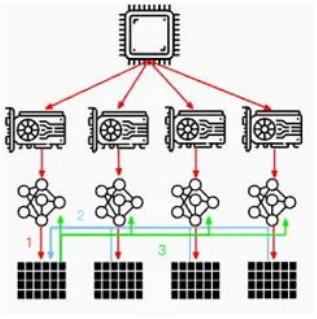
Pre-training Datasets

- Data sources of large models are diverse, mainly including the following:
  - Public data on the Internet: The Internet holds vast amounts of public data, such as texts, images, and videos. Developers can crawl these data on the Internet to build their datasets. Such data may come from sources like news sites, social media, blogs, and forums.
  - Professional databases and data released by institutions: Many professional institutions, databases, and academic organizations release organized and processed datasets. These datasets usually have high quality and reliability. Large models can use these data for training to improve their performance and accuracy.
  - Internal enterprise data: For some customized large models, enterprises may use their internal data to build datasets. Such data may include enterprise business data, user behavior data, and product data, helping model better adapt to the actual needs of enterprises.
  - User-generated content: With the popularity of social media and online platforms, user-generated content (such as comments, ratings, and feedback) has also become an important source of large model datasets. Such data reflects users' real needs and preferences, helping the model better understand and meet their needs.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Data Parallelism

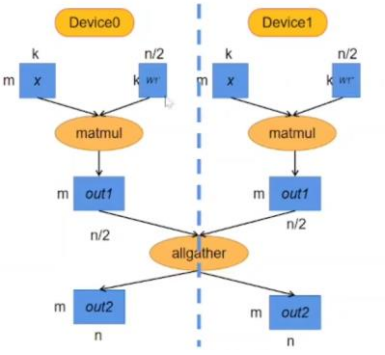
- Data parallelism: The training dataset needs to be split into N parts, each located onto a separate computing node. Each node keeps a full model copy and calculates gradients using its local data. All nodes send their gradients to node 0, which combines them and sends the final result back to the other nodes.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Tensor Parallelism

- In tensor parallel training, we split a tensor into N blocks along one dimension. Each device holds 1/N of the tensor, keeping the computation accurate. This requires extra communication to ensure correct results.



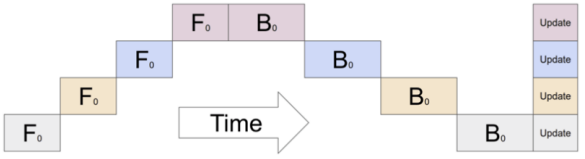
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Row-wise weight partitioning
  - The number of computing units decides how parameters are split. Data also needs to be divided for matrix calculations.
- Column-wise weight partitioning
  - The number of computing units decides how w is split. In this case, input data X does not need to be split.

Pipeline Parallelism

- The following figure shows the pipeline parallelism. Assume that there are four NPUs. The parameters of the large model are divided into four parts by layer and placed on device 0, 1, 2, and 3 in sequence. For a minibatch of data, forward propagation is first performed on device 0 (corresponding to the gray block F0 in the figure). After the calculation is complete, the result of device 0 is sent to device 1 to continue forward propagation (corresponding to the yellow block F0 in the figure). After the forward propagation is complete on the last NPU, that is, device 3, the loss is calculated on device 3 and backward propagation is performed (corresponding to the purple block B0 in the figure). Backward propagation is performed until device 0 is complete. The optimizer parameters are updated on each card, and the model parameters are updated.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Business Process
2. **Large Model Business Process**
   - Pre-training
   - Supervised Fine-tuning
   - Reinforcement Learning
   - Model Evaluation
3. Use of Large Models and Prompt Engineering

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

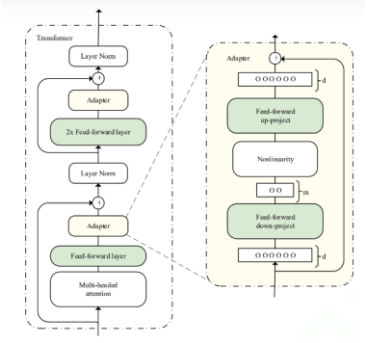
Supervised Fine-Tuning (SFT)

- SFT refers to pre-training a neural network model (the source model) on a source dataset. Then, a new neural network model (the target model) is created.
- The target model copies all design elements and parameters from the source model, except for the output layer.
- These parameters hold knowledge from the source dataset, which works well with the target dataset. The output layer from the source model is removed because it is closely tied to the source dataset's labels.
- During fine-tuning, add an output layer whose output size equals the number of target dataset categories to the target model, and randomly initialize the model parameters of this layer. We train the target model from scratch up to the output layer. The other layer's parameters are fine-tuned using the source model's parameters.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Adapter Tuning

- As computer hardware improves, pre-trained models have more parameters. Fine-tuning the whole model for downstream tasks is costly and time-consuming. Adapters help solve this problem. Adapter adds parameters for downstream tasks into each layer of the pre-trained model. During fine-tuning, only these task-specific parameters are trained, and the main model is frozen. This reduces the computing power costs.

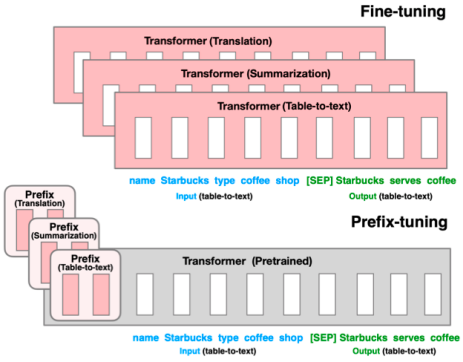


An adapter module is added to each layer or some layers of the pre-trained model. During fine-tuning, the pre-trained model's main part is frozen, and the adapter module learns the knowledge of the downstream task.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Prefix Tuning

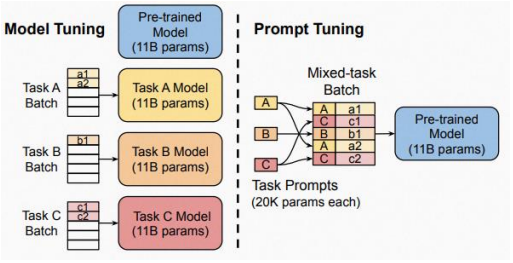
- In prefix tuning, a set of task-specific virtual tokens is added before the input as the prefix. During training, only these prefix token's parameters are updated, while the rest of the Transformer parameters stay fixed.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Prompt Tuning

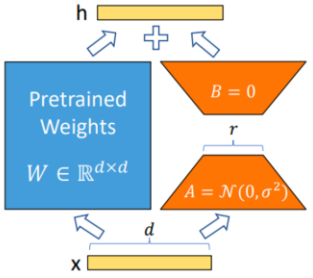
- This method is a simpler form of prefix tuning. It adds prompt tokens only to the input layer, removing the need for an MLP to handle training issues.
- The pre-trained parameters stay fixed, and one or more embeddings are added for each task. These embeddings are then combined with the query and fed into the LLM, with only the embeddings being trained. The left figure shows full-parameter fine-tuning for a single task, while the right figure shows prompt tuning.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

LoRA: Low-Rank Adaptation of Large Language Models

- LoRA's idea is simple. In the figure below, the pre-trained model's matrix parameters are frozen and replaced with matrices A and B. During downstream tasks, only A and B are updated.



- A bypass is added next to the pre-trained language model (PLM) to reduce and then increase dimensions, mimicking the intrinsic rank.
- During training, only the matrices for dimensionality reduction (A) and increasing (B) are updated;
- The model's input and output sizes stay the same. At the end, the parameters of BA and the PLM are merged.
- Matrix A starts with random Gaussian values, and matrix B starts as a zero matrix to keep the bypass matrix at zero initially.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

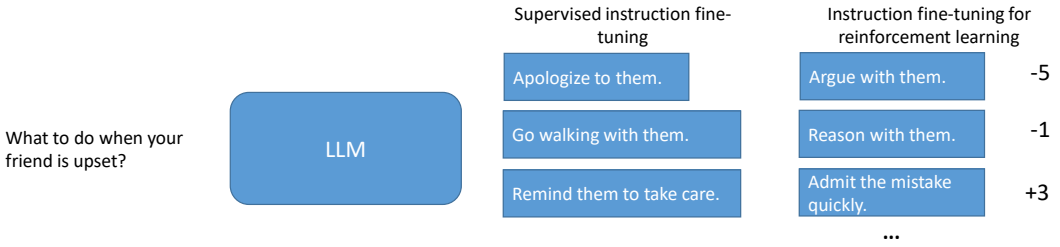
Contents

1. AI Business Process
2. **Large Model Business Process**
   - Pre-training
   - Supervised Fine-tuning
   - RLHF
   - Model Evaluation
3. Use of Large Models and Prompt Engineering

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Alignment

- Use data that humans prefer for training (SFT + RLHF):
  - Authenticity: Is it false or misleading information?
  - Harmlessness: Does it cause physical or mental harm to people or the environment?
  - Usefulness: Does it address the user's task?



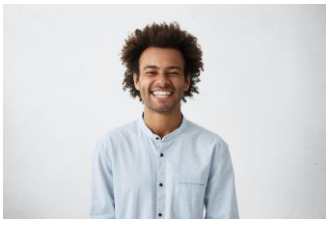
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Why use instruction tuning: Because GPT's outputs can be random, inaccurate, uninteresting, or harmful due to its training on large datasets.
- Why use reinforcement learning: 1. Labeling data is expensive. 2. Reinforcement learning shows if a model works well but doesn't explains how to improve it. Reinforcement learning allows models to explore more freely, overcoming the limits of supervised learning.

RLHF

- Reinforcement learning from human feedback (RLHF) is a method that leverages human feedback to refine a model, enabling effective self-learning.
- By integrating human feedback into the reward function, RLHF ensures that large models can perform tasks more aligned with human targets, aspirations, and needs.





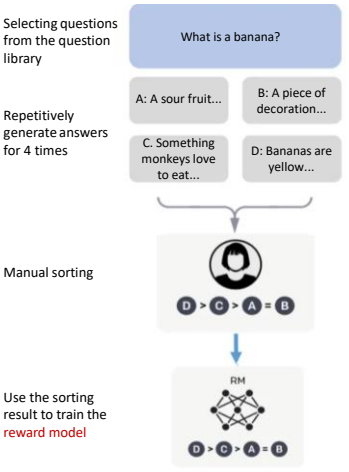
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- RLHF is a standard technology used to make sure LLMs create real, harmless, and helpful content. However, human communication is a personal and creative act, and the value of LLM output largely depends on human values and preferences. Different models are trained in various ways and use different human feedback, leading to varied results. How well each model matches human values depends on its creator.

Reward Model (RM) Training

- Randomly select questions from the dataset. Use mode generated after the supervised instruction fine-tuning (SFT model), and generate multiple responses for each question. Labeling personnel rank the responses based on a thorough evaluation. This process is like getting guidance from a mentor or teacher.
- Then, use the ranking results to train the reward model. Multiple ranking results are paired to form multiple training data pairs.
- The RM model takes an input and gives a score to rate the response's quality. For each training pair, it adjusts the parameters to make sure high-quality responses get higher scores than low-quality ones.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- **Reward model training**
- **Data preparation**: First, prepare a batch of data for training the reward model. This data should include text samples and their human evaluation scores. You can get these scores from crowdsourcing platforms or expert reviews, as they reflect how people perceive text quality.
- **Model construction**: Next, build a neural network as the reward model. You can use a simple multilayer perceptron (MLP) or a more complex model like the Transformer. The key is to ensure the model can effectively capture human evaluation criteria.
- **Training process**: Use the prepared data to train the reward model. The training target is to minimize the error between the model's predicted scores and the actual human evaluation scores. You can achieve this with optimization methods like gradient descent.
- **Validation and tuning**: Validate the model during training to ensure it can accurately predict human evaluations on new data. If the model performs poorly, we may need to adjust its structure or training method.

Reinforcement Learning

- Reinforcement learning considers the overall impact more than supervised learning.
  - Supervised learning gives feedback on individual tokens to help the model give the right answer. Reinforcement learning gives feedback on the whole output instead of individual tokens. Reinforcement learning works better for large language models due to the difference in feedback granularity.
- Reinforcement learning is more effective at addressing hallucination issues.
  - In supervised learning, the model gives a result even if it doesn't know the answer. In reinforcement learning, the model avoids low-scoring answers by using reward scores.
- Reinforcement learning better handles reward accumulation issues in multi-round conversations.
  - Building effective multi-round dialog interactions and determining if the final goal is met is hard with supervised learning. But using reinforcement learning, we can create a reward function that assesses the model's output based on the dialog's coherence and context.

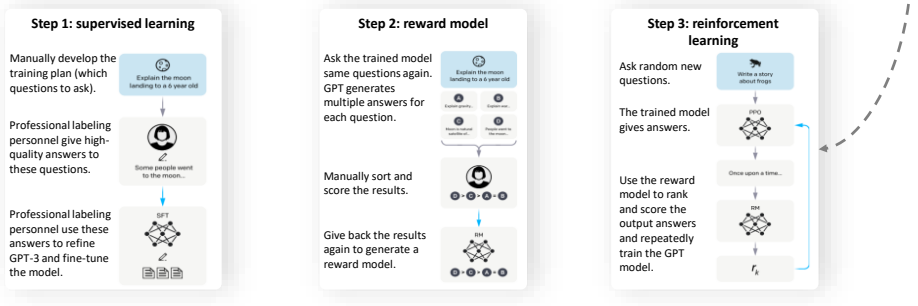
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

RLHF Training Procedure

Fine-tune the pre-trained LLM on specific topics or with instructions and human example corpora.

Collect human-labeled datasets to train a reward model.

Use reinforcement learning, like PPO, to fine-tune the LLM with this dataset and reward model.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Business Process
2. **Large Model Business Process**
   - Pre-training
   - Supervised Fine-tuning
   - RLHF
   - Model Evaluation
3. Use of Large Models and Prompt Engineering

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Why Do We Need Evaluation for Large Models

- A large model is a highly capable function f, which is not fundamentally different from previous machine learning models. So why do we need evaluation for large models? What are the differences between evaluation for large models and previous evaluation for machine learning models?
- Research and evaluation help us better understand the strengths and weaknesses of large models.
  - Most studies show these models match or exceed human performance in many tasks. However, some studies doubt if this success comes from memorizing the training data.
- Research and evaluation can guide the improvement of collaboration between human and large models.
  - Since people are the main users, understanding and evaluating model capabilities is crucial for enhancing human-machine interaction designs.
- Research and evaluation can better plan and coordinate the future development of large models and prevent unknown and potential risks.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- For example, when given only the LeetCode problem number, the model outputs the correct solution. This shows that the training data was contaminated.
- For example, our recent study, PromptBench. It is the first evaluation benchmark form prompt robustness in LLMs. We found that these models often struggle with interference and lack stability. This led us to improve the system's fault tolerance through better prompts.
- Large models keep evolving and becoming more powerful. Can we assess their abilities from an evolutionary viewpoint using well-designed and scientific methods? How can we predict potential risks in advance? These are important research topics.

Evaluation Dimensions

- Large language models can handle many complex natural language tasks, unlike traditional algorithms. Therefore, model evaluation must be performed from multiple dimensions. The evaluation dimensions can be classified into the following three aspects:
  - Knowledge and capability: Large language models have rich knowledge and can handle many tasks, including natural language processing (text classification and information extraction), knowledge Q&A (reading comprehension and open-domain Q&A), natural language generation (text summarization and text creation), logical reasoning, and code generation.
  - Ethics and security: Models are trained based on the 3H principles: helpfulness, honesty, and harmlessness. Harmlessness ensures that the model's responses match human values.
  - Vertical domains, mainly covering complex reasoning capability (such as knowledge reasoning and mathematical reasoning), environment interaction capability (for example, an agent generates actions based on language instructions for home tasks), and problem solving capability in specific domains (such as finance, law, and healthcare).

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Evaluation System

- GLUE and SuperGLUE test a model's natural language understanding. They cover tasks like text classification, inference, question answering, and sentiment analysis. These benchmarks simulate real-world language processing scenarios and set the standard for measuring a model's natural language understanding capabilities.
- GSM8K and MMLU test 57 different disciplines, providing a comprehensive evaluation of the knowledge domain of large models. GSM8K evaluates math problem-solving skills. MMLU uses multiple-choice questions to evaluate a model's understanding and reasoning in many fields.
- BLUE-bench focuses on the biomedical field and tests how well models perform in medical Q&A and related tasks.
- Adversarial Robustness Benchmarks are used to test how well the model resists malicious inputs, evaluating its security.
- LLM Ethics Benchmarks check if large models create content that breaks social ethics, like bias, toxicity, or dishonesty.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Common Evaluation Metrics

- BLEU
  - It is commonly used in machine translation tasks. It checks how many n-grams match between the translated text and the reference text. More matches mean better translation quality. Unigrams are used to measure the accuracy of word-level translation, while higher-order n-grams are used to evaluate the fluency of sentence-level translation.
- NIST
  - It is an improvement on the BLEU method, introducing the concept of information volume for each n-gram, giving more weight to less frequent but important words.
- ROUGE
  - Rouge is an improved version of BLEU that focuses on recall, not precision. It checks how many n-grams from the reference translation appear in the output. Rouge-N and Rouge-L are scores are based on n-grams and the longest common subsequence, respectively.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- https://3ms.huawei.com/km/blogs/details/14813656

Challenges in Large Model Evaluation (1)

- Diverse foundation model capabilities make evaluation challenging
  - It is challenging to make comprehensive and fair evaluations on diverse foundation model capabilities. For example, Stanford evaluates models across 42 scenario tasks in 7 dimensions, SuperCLUE defines 70 scenario tasks, C-Eval selects 52 scenario tasks, and AGIEval chooses 20 standardized test tasks.
- Lack of unified industry standards for large model evaluation
  - Testing standards and methods lack industry-wide uniformity; different evaluation organizations have varying testing subjects, scopes, and results. Public dataset-based testing presents significant challenges: the availability of numerous public datasets increases selection complexity and workload, while it is also difficult to ensure that these datasets do not overlap with the model's training set.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Challenges in Large Model Evaluation (2)

- Lack of evaluation standards for enterprise applications
  - The industry lacks a standard way to evaluate enterprise apps. Different areas focus on different features. For example, customer service values instruction compliance and hallucination suppression, while R&D focuses on code generation.
  - Open-source Internet data cannot be used to evaluate model performance in enterprise services.
  - Professional testing platforms may leak enterprise data.
- Rapid model updates, heavy manual evaluation workload, and repeated work
  - Models update quickly. Manual evaluations are time-consuming, repetitive, and inconsistent.
  - Professional evaluation needs a lot of data preparations. Expert labeling takes a lot of time and effort, making it costly. Also, defining tasks and ensuring coverage are challenges.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Business Process
2. Large Model Business Process
3. **Use of Large Models and Prompt Engineering**
   - How to Use Large Models
   - Prompt Engineering and Basic Methods
   - Advanced Prompting Techniques

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

How to Better Use Large Models



Write clear and specific instructions.

Use symbols to clearly distinguish different parts of the prompt.
Output structured prompts.
Provide a few examples.



...

Give the model time to think.

Develop the steps required to complete the task.
Let the model propose a solution first. Then, compare and assess it.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- Provide as clear and specific instructions as possible to express what you want the model to do. This will guide the model to create content as you expect and reduce the chance of getting irrelevant or wrong answers.
- Most large models have emergent capabilities, can solve relatively complex problems, and have a very broad knowledge coverage.
- They can understand and analyze natural language input from users and provide services and assistance in different fields and tasks.
- However, communicating with large models still requires some skills.

Contents

1. AI Business Process
2. Large Model Business Process
3. **Use of Large Models and Prompt Engineering**
   - How to Use Large Models
   - Prompt Engineering and Basic Methods
   - Advanced Prompting Techniques

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Prompt Engineering

- Prompt engineering involves refining prompt phrases using structured text to guide the LLM to produce desired outputs.
- Researchers can use prompt engineering to improve the capabilities of LLMs in handling complex tasks, such as Q&A and arithmetic inference.
- Developers can use prompt engineering to design and develop powerful engineering technologies to efficiently connect with LLMs or other ecosystem tools.
- Prompt engineering is key for interacting and interconnecting with large language models and understanding what they can do. Users can improve LLM security through prompt engineering. They can also enhance LLM capabilities by using specialized knowledge and external tools.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Zero-Shot Prompting

- Zero-shot prompting is a technique where the user does not provide demonstrations related to the task result, and directly prompts the language model to provide task-related answers.
- Large language models are capable of zero-shot prompting, but their success depends on the complexity of the task and the scope of existing knowledge.

```
Q: <Question>?
A:
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Example of Zero-Shot Prompting

Prompt:

Certain substances in apples may cause allergies in consumers, leading to symptoms such as rashes and difficulty breathing; at the same time, excessive fructose and dietary fiber in apples may cause digestive symptoms such as bloating and diarrhea.

Positive and negative evaluation:

Output:

The description includes potential negative impacts of apples without directly mentioning any positive aspects of apples. The following is an analysis of the positive and negative evaluations of the description:

1. Negative evaluation

- "Certain substances in apples may cause allergies in consumers": This points out the potential health risks apples may pose to certain groups, which is a clear negative evaluation.
- "Leading to symptoms such as rashes and difficulty breathing": It lists the symptoms of allergic reactions in detail, further reinforcing the potential negative health impacts of apples.
- "Excessive fructose and dietary fiber may cause digestive symptoms such as bloating and diarrhea": This again emphasizes the potential side effects of apple consumption, which is a negative evaluation of the potential adverse effects of apples.

In this description, no positive attributes of apples, such as nutritional value or health benefits, are mentioned, so overall it leans towards a negative evaluation.

2. Positive evaluation: Although there is no direct positive evaluation in the description, it can be inferred that, since apples are widely recognized as a healthy food, the mentioned allergens and digestive issues usually do not prevent most people from consuming apples. However, this positive perspective is based on general knowledge rather than what is provided in the description itself.

In summary, the description focuses on the potential risks of apples without explicitly expressing any positive features, so it can be judged as a negative evaluation. It should be noted that, to provide a comprehensive evaluation, the positive aspects such as the nutritional and health benefits of apples are usually also needed to be considered.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- If only simple data judgment is required, the output is complex and contains a large amount of unnecessary information.
- Models of the GPT series are all autoregressive language models. This is to say that such models predict the next word based on the current input, and then combine the prediction result with the input as the model input to predict the next word. This process repeats cyclically.
- Researchers of GPT-3 found that large-scale training data generates an interesting emerging behavior called In-Context Learning (ICL). ICL does not require model parameter adjustment and can achieve excellent results with only a few examples of downstream tasks.
- The pre-trained GPT-3 model does not need to be retrained when migrated to a new task. You only need to provide a task description (optional), then provide several examples (task queries and corresponding answers, organized in pairs), and finally add the query to be answered by the model. After the preceding content is packaged as the model input, the model can correctly output the answer corresponding to the last query.

Few-Shot Prompting

- Although large language models demonstrate impressive zero-shot capabilities, they still perform poorly on more complex tasks when using zero-shot settings.
- Few-shot prompting can be used as a technique to enable contextual learning, providing demonstrations in the prompt to guide the model to achieve better performance. Demonstrations are conditions for subsequent examples, and the model is expected to generate responses.
- Few can be selected randomly.

```
Q: <Question>?
A: <Answer>
Q: <Question>?
A: <Answer>
Q: <Question>?
A: <Answer>
Q: <Question>?
A:
```


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- One-shot few-shot

Example of Few-Shot Prompting

Prompt:

Certain substances in apples may cause allergies in consumers, leading to symptoms such as rashes and difficulty breathing; at the same time, excessive fructose and dietary fiber in apples may cause digestive symptoms such as bloating and diarrhea.

Evaluation: Negative

Bananas are a very beneficial fruit for the human body.

Evaluation: Positive

Durian is sweet and delicious, with very high nutritional value.

Evaluation:

Output:

Positive

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. AI Business Process
2. Large Model Business Process
3. **Use of Large Models and Prompt Engineering**
   - How to Use Large Models
   - Prompt Engineering and Basic Methods
   - Advanced Prompting Techniques

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Chain-of-Thoughts Prompting

- Chain-of-thought (CoT) prompting enables complex reasoning by introducing intermediate reasoning steps. It expands a question step by step and retains the background information of previous questions in each answer. CoT makes a conversation content-rich and coherent.
- CoT prompting mainly has the following two types:
  - Few-shot-CoT
  - Zero-shot-CoT
- Feature
  - Orderliness: CoT prompting requires that questions be broken down into a series of ordered steps. Each step is based on the previous step, forming a clear CoT.
  - Relevance: There must be a close logical connection between each thought step to ensure the coherence and consistency of the entire thought process.
  - Step-by-step inference: The model focuses only on the current question and related information in each step and approaches the final answer through step-by-step reasoning.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Few-Shot-CoT

- In this mode, the model is provided with a small number of examples, each of which includes a high-quality inference process (handwritten or model-generated).

Prompt:

The sum of the odd numbers in this group is even: 4, 8, 9, 15, 12, 2, 1.
A: Add all the odd numbers (9, 15, 1) to get 25. The answer is False.
The sum of the odd numbers in this group is even: 17, 10, 19, 4, 8, 12, 24.
A: Add all the odd numbers (17,19) to get 36. The answer is True.
The sum of the odd numbers in this group is even: 16, 11, 14, 4, 8, 13, 24.
A: Add all the odd numbers (11, 13) to get 24. The answer is True.
The sum of the odd numbers in this group is an even number: 17, 9, 10, 12, 13, 4, 2.
A: Add all the odd numbers (17, 9, 13) to get 39. The answer is False.
The sum of odd numbers in this group of numbers is even: 15, 32, 5, 13, 82, 7, 1.
A:

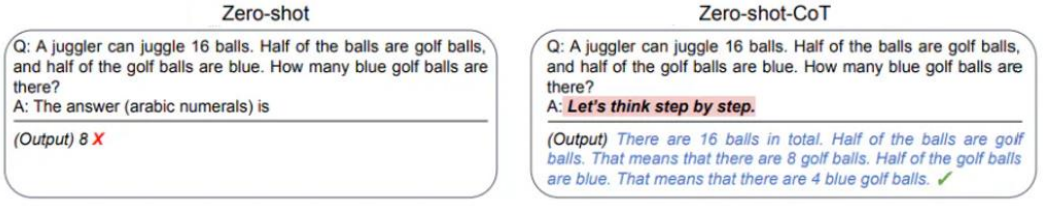
Output:

Add all odd numbers (15, 5, 13, 7, 1) to get 41. The answer is False.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Zero-Shot-CoT

- The zero-shot-CoT method adds the prompt "Let's think step by step" before each answer of the language model to guide the model to think step by step.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Advantages of Zero-Shot-CoT

- Generalization: Zero-shot-CoT uses a single template and can work across multiple inference tasks without needing to design a specific prompt for each task, whereas few-shot prompts require a dedicated example for each task.
- No manual example design: Zero-shot-CoT does not require manual design of multi-step inference examples, reducing the workload of prompt engineering, whereas few-shot prompts necessitate meticulous design of examples for each task.
- Stronger zero-shot inference benchmark: Zero-shot-CoT serves as a stronger zero-shot inference benchmark, providing better performance for difficult multi-step inference tasks, while standard zero-shot prompts perform poorly.
- Better scalability: As the model scale increases, the performance improvement of zero-shot-cot is more obvious, showing better scalability. Few-shot prompts are not very sensitive to the model scale.
- Reduced bias: Zero-shot-CoT reduces the bias introduced by manually designed examples, making the research on language model bias more fair.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. (Multiple-answer question) The chain-of-thought (CoT) prompt implements complex reasoning capabilities by introducing intermediate reasoning steps. Compared with few-shot-CoT, what are the advantages of zero-shot-CoT?
   A. In most cases, the effect is better than that of few-shot-CoT.
   B. No need to manually design examples
   C. Better scalability
   D. High generalization

Copyright © Huawei Technologies Co., Ltd. All rights reserved.
HUAWEI

- BCD

Summary

- This chapter introduces the entire process of AI service and foundation model building, and describes how to use prompt engineering to efficiently use foundation models to obtain more accurate answers.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

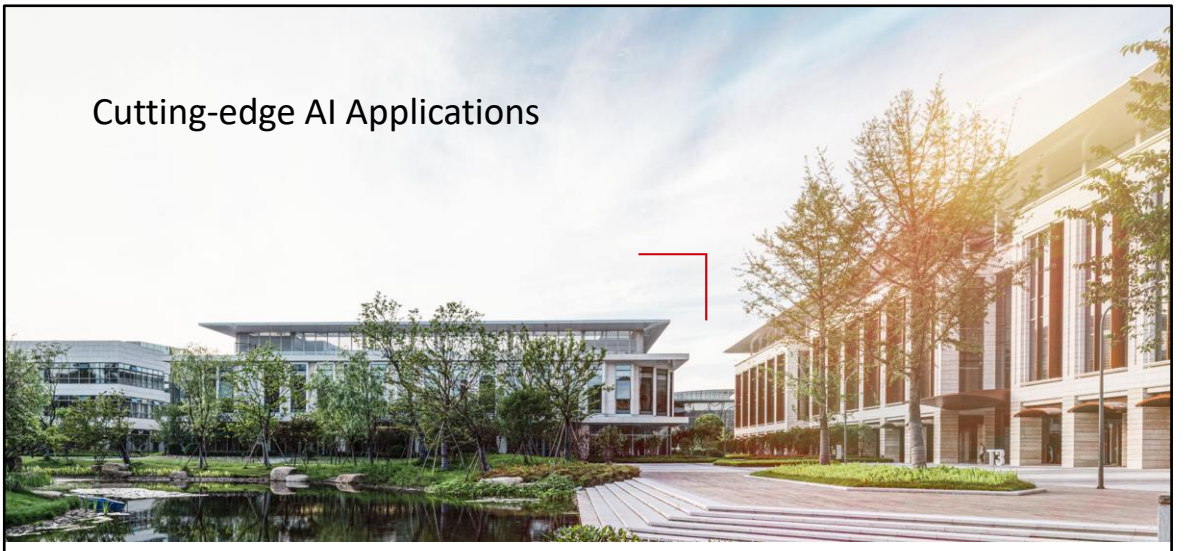
Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.



Cutting-edge AI Applications



HUAWEI

Foreword

- In a time of overwhelming technological advancement, artificial intelligence (AI) is an incredible force for change. From intelligent assistants to autonomous driving, and from weather forecasts to intelligent robots, AI is being applied almost pervasively, making it a new engine for social progress and development.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Objectives

- Upon the completion of this course, you will be able to understand:
  - AI applications in daily life
  - AI applications in cutting-edge fields

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. **Voice Assistants**
2. Smart Homes
3. Intelligent Vehicles
4. Intelligent Recommendation
5. Intelligent Robots
6. AI4Science

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Ubiquitous AI

- In this age of technology, AI has become indispensable to our lives. AI is bringing all kinds of novel conveniences to our lives, ranging from the voice assistants on our smartphones to the automatic management of smart homes, even reaching our healthcare, education, and travel.



Smart home

Voice assistant





Efficient travel

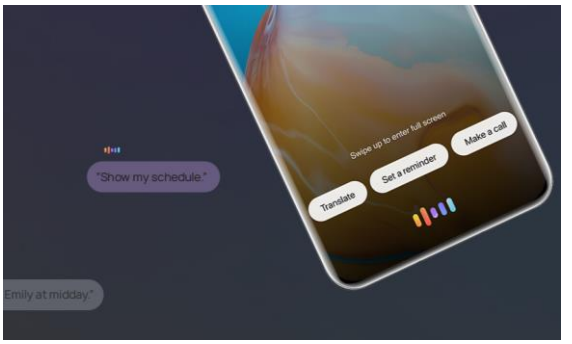
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://consumer.huawei.com/en/emui/celia/

Voice Assistant – Celia

- Celia is a smart assistant. Powered by foundation models, Celia is capable of providing solutions for every scenario.



Natural dialogue-based interaction, making it an efficient and convenient tool

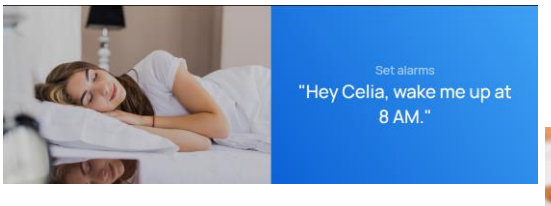
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

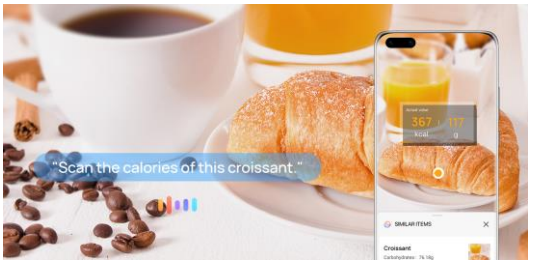
HUAWEI

- https://consumer.huawei.com/en/emui/celia/

Celia – Simplify Your Daily Routines

Wherever you are and whatever you are doing, with the help of Celia, you can get your everyday tasks done quickly and effortlessly. Be productive and enjoy your day!



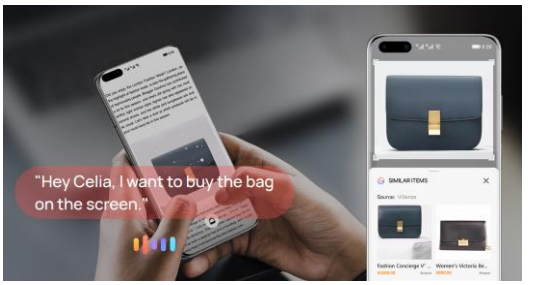


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Celia – Be Fun, Be Smart

Celia makes entertainment easier as well. Quickly find items you want to buy, play your favourite videos and control your music without picking up the phone. Relax and have fun!



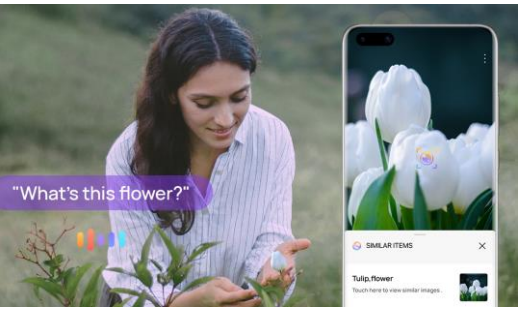


Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Celia – Your Perfect Travel Companion

Celia is also the travel assistant you are looking for. Ask Celia to translate your menu, plan your day, take photos for you and more. Celia is always there to meet your needs.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Voice Assistants
2. **Smart Homes**
3. Intelligent Vehicles
4. Intelligent Recommendation
5. Intelligent Robots
6. AI4Science

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Smart Home

- The smart home seamlessly blends modern technology with home life, redefining our personal spaces with an elevated lifestyle of convenience.
- Through highly advanced IoT, algorithms, and big data analytics, smart homes connect a variety of devices into one automated and tailored intelligent ecosystem, bringing users with convenience, comfort, and security.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://3ms.huawei.com/km/static/image/detail.html?fid=66109

Smart Home – Making Your Life Easier

- AI connects home appliances to create an intelligent ecosystem

When you step into your home, warm lights fade in from the hallway to the living room. The intelligent environment control system automatically adjusts the ambient temperature, humidity, and air quality to suit your preferences. Let the home take care of itself, and relax after a long day.

Awaken the entire house in the flick of a switch

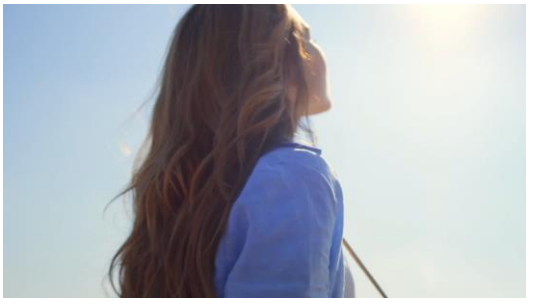


Lighting up your way

At night, corridor lights automatically switch on to match your path, without a need of manual control.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Smart Home - Making Your Life More Comfortable



During the day, sunshades are controlled in groups for adjusting light intensity, making sure rooms are filled with soft light.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Rest Assured at Home and Away



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

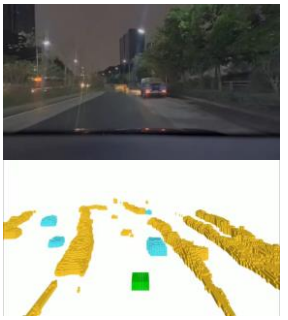
1. Voice Assistants
2. Smart Homes
3. **Intelligent Vehicles**
4. Intelligent Recommendation
5. Intelligent Robots
6. AI4Science

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Qiankun Intelligent Driving

QianKun ADS

With powerful identification and processing capabilities, Qiankun ADS 3.0 reaches an identification rate of 99.9% for unusually shaped obstacles. It can accurately identify whitelisted obstacles such as humans and vehicles, as well as unusually shaped obstacles like cones, water-weighted traffic barriers, soil piles, and stones and gravel.



Qiankun Intelligent Driving
Convergent perception

As the "eye" of an intelligent vehicle, sensors are a core component of an intelligent driving system. They can include laser radars, millimeter-wave radars, and cameras. With data from multiple sensors merged, objects can be detected in reliable, accurate, and robust way, providing all-scenario, all-condition, and all-weather protection.



Intelligent computing platform

Functioning as the "brain" of an intelligent vehicle, the intelligent driving computing platform has powerful performance and can receive complex road condition information from dozens of sensors around the vehicle in real time. Being responsive, it enables the intelligent driving algorithm to perform inference and calculation on complex road condition information and quickly decide upon how to drive. Being stable and reliable, it can run for a long time, which can help reduce a driver's fatigue when going long distances. This all combines for a relaxing and comfortable travel experience.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

HarmonyOS Cockpit

- The HarmonyOS automotive operating system redefines the intelligent cockpit. Celia's voice and visual capabilities make travel smarter. The smart OS links people, audio systems, and devices. Hundreds of automotive applications and in-vehicle audio deliver a new mobile entertainment experience.





Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Contents

1. Voice Assistants
2. Smart Homes
3. Intelligent Vehicles
4. **Intelligent Recommendation**
5. Intelligent Robots
6. AI4Science

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Intelligent Recommendation

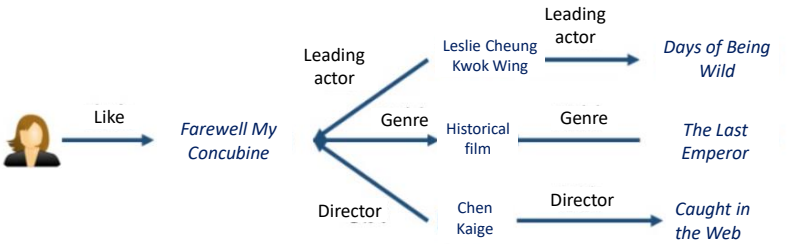
- Intelligent recommendation is a recommender system service built based on AI. AI-powered recommendation can automatically integrate big data and continuously gain insights into end users through data analysis and filtering, empowering enterprises to develop into data-driven decision-making organizations.
- Knowledge graphs are knowledge bases used in intelligent recommender systems. They are essentially structured networks that store relationships between entities. They contain a large amount of background information about target objects and relationships between objects in recommendation systems. Knowledge graphs provide precise profiles of target objects, such as user behavior, interests, and requirements, to implement accurate matching and targeted recommendation, achieving scenario- and task-specific personalized recommendation. This close relationship enables knowledge graphs to shine in intelligent recommendation.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Case Study

- As shown in the following figure, a knowledge graph provides different types of relationship links, introducing more semantic relationships to items. This helps diverge recommendation results, avoid recommendation limitations, and discover user interests at a deeper level. This user may also like the following movies: *Days of Being Wild*, *The Last Emperor*, and *Caught in the Web*. In addition, the graph can connect users' historical records and recommendation results to improve their satisfaction and acceptance of recommendation results and enhance their trust in the recommender system.



Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- In addition, as a type of effective auxiliary information in hybrid recommender systems, knowledge graphs can effectively resolve a range of key problems regarding data sparseness, cold start, and recommendation diversity.

Contents

1. Voice Assistants
2. Smart Homes
3. Intelligent Vehicles
4. Intelligent Recommendation
5. **Intelligent Robots**
6. AI4Science

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

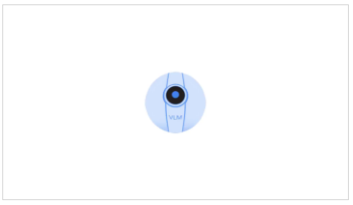
Intelligent Robots

- Humanoid robots are designed to have human-like forms and functions, including sensory perception and cognition.
- They will likely become the most valuable carriers of embodied AI. Combined with rapidly developing general artificial intelligence and foundation models, humanoid robots will enable machines to interoperate and interact with the environments in a more intuitive and intelligent manner. They will be able to perform a wide variety of complex tasks just like humans.



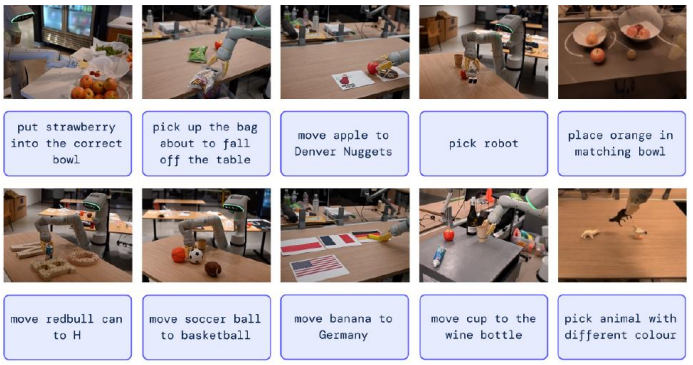
Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Robots Understand Instructions and Things



Vision-language models trained on massive Internet data are directly incorporated into end-to-end robotic control, and such models are referred to as vision-language-action models (VLAs).

The Vision-Language-Action model can significantly improve a robot's generalization capability for new objects. It can interpret commands that do not exist in the robot's training data, for example, placing an object on a specified number or icon. It can also perform basic inference in response to user commands, like picking up the smallest or largest object, or the one closest to a specified object.



RT-2: Vision-Language-Action Models TransferWeb Knowledge to Robotic Control--DeepMind

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- DeepMind explores how to incorporate vision-language models trained on massive Internet data directly into end-to-end robotic control for improving generalization and implementing unexpected semantic inference. We aim to enable an end-to-end trained single model to learn to map the information observed by the robot to actions, while retaining the advantages of massive pre-training on language and vision-language data. To this end, we propose joint fine-tuning of state-of-the-art vision-language models, including robot trajectory data and massive Internet vision-language tasks (for example, visual Q&A). Unlike other practices, we decide to achieve this objective by using a simple and general approach. To incorporate natural language responses and robot actions into the same format, we represent the actions as text labels and add them directly into the training set of the model in the same way as natural language labels. Such model is called a Vision-Language-Action Model (VLA) and instantiated as RT-2. According to our extensive evaluations (6000 evaluation trials), efficient robot strategies were generated and RT-2 acquired a series of unanticipated capabilities from massive Internet training. To be specific, this can significantly improve a robot's generalization capability for new objects, interpret commands that do not exist in the robot's training data (for example, placing an object on a specified number or icon), and perform basic inference in response to user commands (for example, picking up the smallest or largest object, or the one closest to a specified object).

Robots Understand Things Like Human

0. Different mugs, some observed by the robot, others not.





1. Give an instruction for a mug observed by the robot.

RDT-1B: Diffusion Foundation Model for Bimanual Manipulation

2. Give an instruction for mug 1 not yet observed by the robot.





3. Give an instruction for mug 2 not yet observed by the robot.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

HUAWEI

- https://arxiv.org/pdf/2410.07864
- https://rdt-robotics.github.io/rdt-robotics/

Contents

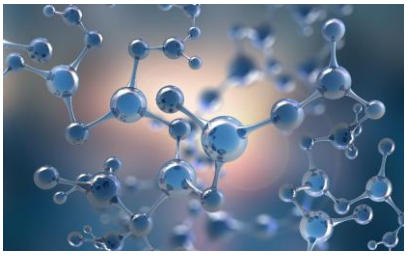
1. Voice Assistants
2. Smart Homes
3. Intelligent Vehicles
4. Intelligent Recommendation
5. Intelligent Robots
6. **AI4Science**

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The Power of AI in Science



On October 8, 2024, the Royal Swedish Academy of Sciences announced that the Nobel Prize in Physics 2024 would be awarded to John J. Hopfield and Geoffrey E. Hinton for their "foundational discoveries and inventions that enable machine learning with artificial neural networks".



On October 9, 2024, the Royal Swedish Academy of Sciences announced that the Nobel Prize in Chemistry 2024 would be awarded to David Baker for his contributions to computational protein design, and to Demis Hassabis and John M. Jumper for their contributions to protein structure prediction using AlphaFold2, an AI model developed by them.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

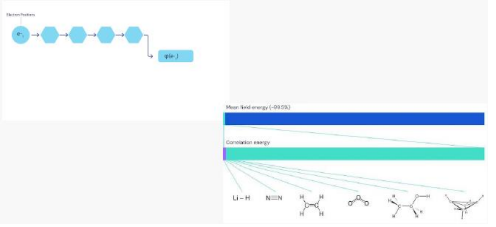
HUAWEI

- The Nobel Prize in Physics was awarded to American scientist John Hopfield and Canadian scientist Geoffrey Hinton for their "foundational discoveries and inventions that enable machine learning with artificial neural networks". The work of these two pioneers in AI has laid a foundation for the development of deep learning and neural networks, and plays a crucial role in the booming development of AI.
- Half of the Nobel Prize in Chemistry was awarded to American scientist David Baker for his contribution to computational protein design, while British scientist Demis Hassabis and American scientist John Jumper shared the other half for their achievements in protein structure prediction. AlphaFold2, an AI model developed by Hassabis and Jumper, has solved a 50-year-old problem by predicting the complex structures of approximately 200 million known proteins. It has been used by over 2 million people worldwide and holds revolutionary significance in fields like drug R&D.
- The awarding of these prizes reflects a new trend in scientific research: AI, as a force that cannot be ignored, is driving a paradigm shift in scientific research. In fields such as physics, chemistry, biology, and medicine, AI has become an important tool for solving long-standing complex scientific problems. It has established a paradigm theoretically capable of solving all scientific problems: starting from practical problems, transforming them into input data that AI can process, and then using deep learning networks to ultimately output results. Many scientists believe that AI will continuously push scientific research beyond traditional frameworks, achieving more profound and broader innovations.

AI + Quantum Mechanics

FermiNet (DeepMind, 2024)

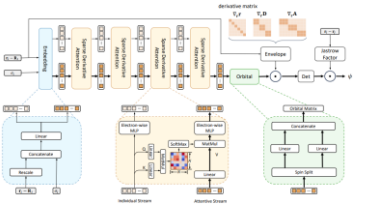
FermiNet is a neural network architecture proposed by DeepMind. It supports a parameterized wave function representation method and can efficiently process multi-electron quantum systems, improving the accuracy and efficiency of electronic structure calculations. FermiNet calculates the energy of atoms and molecules, and still captures 97% of the relevant energy in a molecular system consisting of 30 electrons of Bicyclobutane.



David Pfau, et.al. Accurate computation of quantum excited states with neural networks. Science, 2024

LapNet (Peking University and ByteDance, 2024)

LapNet is a deep learning architecture developed by ByteDance and Peking University for neural network-based variational Monte Carlo (NN-VMC). The team has designed a forward Laplacian computing framework to calculate the Laplacian related to the neural network through an effective forward propagation process (which is a bottleneck of NN-VMC). LapNet can achieve significant acceleration, extending the applicability of NN-VMC to larger systems.



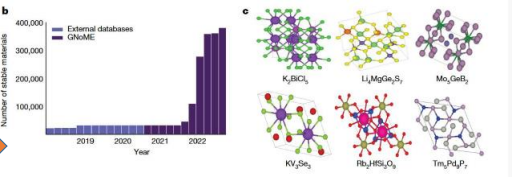
Ruichen Li, et.al. A computational framework for neural network based variational Monte Carlo with Forward Laplacian. Nature M. I., 2024

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

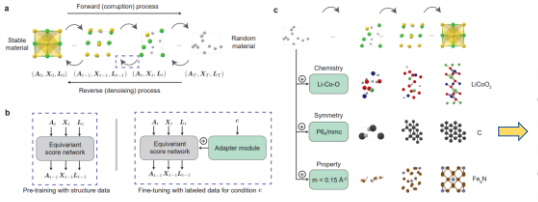
AI + Material Science

GNoME (DeepMind and *Nature*, 2023)

The GNoME tool launched by DeepMind has discovered 2.2 million types of new crystals, among which 380,000 crystals with stable structures are expected to be synthesized. This may drive technological transformations in next-generation batteries and superconductors. GNoME uses a graph neural network (GNN) model to efficiently predict the crystal structure and stability of materials, greatly accelerating the discovery of new materials. Compared with traditional experimental methods, AI models can predict millions of materials in a short time, shortening the research period and significantly saving experimental costs.



AmilMerchant et al. Scaling deep learning for materials discovery, Nature (2023)



Tian Xie et al. MatterGen: a generative model for inorganic materials design. (2024)

MatterGen (Microsoft, 2024)

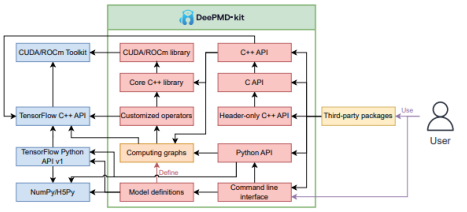
MatterGen is a new generative model developed by Microsoft Research to design inorganic materials with specific properties. MatterGen can generate stable and diverse crystal structures, and refine atomic types, coordinates, and periodic lattices through a new diffusion-based generation process. The model can be fine-tuned based on specific property requirements to generate new materials that meet multiple property constraints, such as chemical composition, symmetry, and mechanical, electronic, and magnetic properties, for directional material design.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

AI + Molecular Dynamics

DeePMD (DP Technology, 2023)

DeePMD-kit is a deep learning package with great potential developed by DP Technology. It integrates first-principles modeling, data-driven methods represented by machine learning, and high-performance computing to process ultra-large-scale molecular dynamics problems with the precision of first principles. DeePMD-kit can significantly improve computing efficiency while improving simulation precision, resolving the contradiction between precision and computing volume in traditional simulation methods.



Weinan E, et. al. DeePMD kit v2: A software package for deep potential models. 2023

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Quiz

1. List some typical AI applications.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

- Smart speaker, text-to-image APP, and the like

Summary

- This chapter describes how AI reshapes our lives, as well as AI applications in fields such as meteorology and robotics.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and organization for a fully connected, intelligent world.

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

HUAWEI